

Python Essencial

Abrindo Portas para o Futuro

Dilermando Piva Jr



Python Essencial

Abrindo Portas para o Futuro

Prof. Dr. Dilermando Piva Junior

FoxTablet

Salto - SP, 2024

Copyright desta edição 2024 [SELO]

Copyright do texto 2024 Dilermando Piva Jr.

Copyright da Capa 2024 Dilermando Piva Jr.

Aviso copyright internacional quando obrigatório em contrato

Nenhuma parte desta edição pode ser utilizada ou reproduzida – em qualquer meio ou forma, seja mecânico ou eletrônico, fotocópia, gravação etc. –, nem apropriada ou estocada em sistema de banco de dados, sem a expressa autorização da editora ou do autor.

O texto deste livro foi fixado conforme o acordo ortográfico da Língua Portuguesa vigente no Brasil desde 1º de janeiro de 2009.

1ª edição, 2024 - Impresso no Brasil

Dados Internacionais de Catalogação na Publicação (CIP) de acordo com ISBD

M000

Piva Jr., Dilermando

Python Essencial: abrindo portas para o futuro / Dilermando Piva Jr. – Salto, SP: Foxtablet, 2024.

139 p. ; il. ; 17cm x 24cm.

Inclui índice.

ISBN: 978-[CÓDIGO-AQUI]

1. Programação. 2. Python. 3. Machine Learning. 4. Ciência de Dados. 5. Automação. 6. Inteligência Artificial. I. Piva Jr., Dilermando. II. Título.

CDD 005.133 CDU 681.3

Elaborado por [BIBLIOTECÁRIO], CRB-[NÚMERO]

Prefácio

Python é, sem dúvida, uma das linguagens de programação mais acessíveis, poderosas e versáteis da atualidade.

Ela é utilizada por uma ampla gama de profissionais, desde desenvolvedores de software até cientistas de dados e entusiastas de programação.

Sua simplicidade de sintaxe e a vasta quantidade de bibliotecas prontas fazem com que Python se destaque como a escolha preferida para quem deseja aprender a programar, assim como para aqueles que já têm experiência em outras linguagens e buscam uma maneira mais eficiente de desenvolver soluções.

O propósito deste livro é justamente **abrir as portas para o futuro** através do domínio de Python. Seja você um iniciante que nunca escreveu uma linha de código ou um desenvolvedor experiente que deseja expandir seus conhecimentos, **Python Essencial** foi pensado para guiar você por essa jornada de aprendizado de forma prática e intuitiva.

A cada capítulo, você será apresentado a conceitos fundamentais, práticas recomendadas e exemplos reais que irão ajudá-lo a aplicar a linguagem no mundo real, seja na criação de aplicativos, automação de processos ou análise de dados.

Por que Python? Simples. Python oferece a combinação perfeita de facilidade de uso com profundidade técnica.

Você pode começar com um "*Hello World!*" e, em pouco tempo, estará construindo soluções complexas e eficientes.

A sua curva de aprendizado é rápida, mas sua capacidade é ilimitada.

Além disso, com o crescente movimento de **inteligência artificial**, **ciência de dados**, e **automação**, Python se tornou a linguagem por excelência para aqueles que querem se destacar nessas áreas.

Com o avanço das tecnologias e a crescente demanda por profissionais capacitados em áreas como **machine learning**, **big data**, e **desenvolvimento web**, o aprendizado de Python não é apenas uma vantagem competitiva, mas uma **necessidade**.

Este livro tem como objetivo fornecer as bases para que você não só compreenda Python, mas possa utilizá-lo como uma ferramenta para resolver problemas, criar novas oportunidades e, quem sabe, abrir as portas para uma nova carreira.

Python Essencial: abrindo portas para o futuro! é mais do que um guia técnico. É um convite para explorar o potencial que a programação pode oferecer.

Ao final deste livro, você terá construído não apenas um conhecimento profundo da linguagem, mas também uma confiança renovada nas suas habilidades para enfrentar qualquer desafio que o mundo da programação possa apresentar.

Boa Leitura!!

Prof. Dr. Dilermando Piva Jr.

Sobre o Autor



Dr. Dilermando Piva Junior é um acadêmico e profissional de destaque na área de **Computação e Ciência de Dados**, com uma sólida carreira que abrange ensino, pesquisa e consultoria em diversas áreas da tecnologia. Ele é doutor na área de **Inteligência Artificial** pela **UNICAMP** (2006) e mestre pela mesma instituição (2005), onde desenvolveu pesquisas pioneiras que aliam algoritmos avançados à tomada de decisão automatizada, áreas centrais de sua especialidade.

Com uma trajetória acadêmica e profissional voltada à inovação em **programação**, Dr. Piva tem vasta experiência em diversas **linguagens de programação**, com foco especial em **Python**, uma ferramenta que ele utiliza e ensina extensivamente em suas atividades de docência e consultoria. Ao longo dos anos, ele se especializou na aplicação de Python em áreas como **desenvolvimento de software, machine learning, ciência de dados e inteligência artificial**, conquistando uma ampla expertise em projetos complexos para empresas de médio e grande porte.

Atualmente, Dr. Piva é **Coordenador do Curso de Ciência de Dados para Negócios** na **Fatec Votorantim**, onde capacita a próxima geração de profissionais de tecnologia, com um currículo fortemente embasado em linguagens de programação e o uso de **Python** em análises de dados e aprendizado de máquina. Ele também atua como professor na **Fatec Sorocaba**, compartilhando sua vasta experiência em **algoritmos, estrutura de dados e aprendizado de máquina**.

Além de sua atuação acadêmica, o Dr. Piva é amplamente reconhecido por suas contribuições à área educacional, sendo autor de várias publicações nacionais e internacionais. Ele é membro do **Comitê Científico da ABED** e participa dos **Conselhos Editoriais** de renomadas revistas científicas, além de ser avaliador ad-hoc do **Conselho Estadual de Educação do Estado de São Paulo (CEE)**. Entre 2009 e 2014, foi **Coordenador Geral do Ensino a Distância** do **Centro Paula Souza**, onde ajudou a consolidar a educação a distância em São Paulo.

Com uma formação acadêmica robusta que inclui um mestrado em **Gerenciamento de Sistemas de Informação** pela **PUC-Campinas** (1996) e pós-graduações em **Big Data** e **Acupuntura**, Dr. Piva continua a explorar novas fronteiras do conhecimento, sempre focado no impacto da tecnologia e da ciência de dados para o futuro.

Mais informações, acesse: <https://piva.pro.br>

Agradecimentos e Dedicatória

Este livro é o resultado de uma jornada que não percorri sozinho. Gostaria de dedicar e expressar meus mais profundos agradecimentos àqueles que me acompanharam, me apoiaram e foram fundamentais na realização de mais esta obra.

À minha esposa, **Adriana Barros Ferreira**, minha parceira incansável, que me ofereceu seu apoio, carinho, amor e compreensão ao longo de todo o processo de criação deste livro. Sua paciência e incentivo contínuo foram essenciais para que eu pudesse seguir em frente. Este livro é tão seu quanto meu.

Também dedico este trabalho aos meus pais, **Tereza Benatti Piva** e **Dilermando Piva**, que, embora não estejam mais fisicamente presentes, me deram as bases de amor, educação e valores que me trouxeram até aqui. A eles, devo tudo o que sou e todas as conquistas que celebro hoje.

Um agradecimento especial ao meu tio, **José Oreste Benatti**, que muitas vezes foi como um segundo pai. Suas orientações, conselhos e apoio incondicional ao longo da minha vida foram inestimáveis. Sua sabedoria e presença marcaram profundamente meu caminho.

A todos vocês, minha eterna gratidão e este livro como símbolo do que o amor, o apoio e o legado podem gerar.

Sumário

1. Bem-vindo(a) ao Mundo da Programação Python.....	11
○ Oportunidades que esperam por Você	
○ Python: A chave para o Futuro	
○ Vamos começar a Codificar!	
○ Preparando-se para a Jornada	
○ Para saber mais...	
2. Configurando o Ambiente	24
○ Por Que Usar o Google Colab?	
○ Foco no Aprendizado	
○ Pronto para Começar?	
○ Passo a Passo para Configurar o Google Colab	
○ Primeiros passos com Python	
○ Hello World! – O primeiro passo de todo programador	
○ Escrevendo o Primeiro Código	
○ O que acabou de acontecer?	
○ O primeiro de muitos passos	
○ Para saber mais...	
3. Variáveis e Tipos de Dados	31
○ Um Ponto Importante: use nomes descritivos e em Inglês	
○ Tipos de Dados	
○ Tipo Texto – String – str	
○ Tipo Inteiro – int	
○ Tipo Decimal / Real – float	
○ Tipo Booleano – bool	
○ Para saber mais...	
4. Operadores	39
○ Operadores Aritméticos	
○ Operadores de Comparação	
○ Operadores de Atribuição	
○ Operadores Lógicos	
○ Operadores de Identidade	
○ Operadores de Associação	
○ Para saber mais...	
5. Estruturas de Dados Integradas	58
○ Listas e seus métodos	
○ Tuplas e seus métodos	
○ Conjuntos e seus métodos	
○ Dicionários e seus métodos	
○ Manipulação de Strings	
○ Para saber mais...	
6. Controle de Fluxo: Comandos Condicionais.....	79
○ Comandos Condicionais	
○ Mas o que são “Blocos de Código”?	

○ Por que a Indentação é Importante? ○ Comando if-else ○ Comando if-elif-else ○ Um exemplo final ○ Para saber mais...	
7. Controle de Fluxo: Comandos de Repetição.....	86
○ Laço while ○ Mas o que é um iterável? ○ Laço for ○ A função range() ○ Comandos break e continue ○ Para saber mais...	
8. Funções.....	96
○ Criando Funções em Python ○ Funções com Parâmetros ○ Funções com Retorno de Valores ○ Funções como Automatizadores de Tarefas ○ Interação com o Usuário: função input() ○ Para saber mais...	
9. Módulos ou Pacotes em Python	105
○ Mas o que são Módulos? ○ O que são Pacotes? ○ Importando Funções Específicas ○ Por que usar Módulos e Pacotes? ○ Como instalar pacotes no Google Colab ○ Para saber mais...	
10. Introdução ao Machine Learning	112
○ O que é Machine Learning? ○ Como funciona e quais os principais tipos de ML? ○ Como as máquinas aprendem? ○ Algoritmos Comuns no Machine Learning ○ Exemplo prático de ML em Python ○ Para saber mais...	
11. Introdução a Ciência de Dados.....	123
○ O que é Ciência de Dados? ○ O pipeline de Ciência de Dados ○ Principais pacotes de Ciência de Dados em Python ○ Pipeline completo de Ciência de Dados c/ Python ○ Para saber mais...	
12. Desenvolvimento de Projetos Reais.....	135
○ Projeto 1: Pedra, Papel e Tesoura ○ Projeto 2: Conversor de Moedas ○ Projeto 3: Calculadora Simples ○ Projeto 4: Gerador de Senhas Aleatórias ○ Projeto 5: Jogo da Força	

- **Projeto 6:** Classificador de SPAM
- **Projeto 7:** Prevendo o Preço de Casas
- **E chegamos ao final dessa Jornada!.....** 165
- O Futuro da Linguagem Python
- O Próximo Passo
- Considerações Finais

Referências Bibliográficas 167

1

Bem-vindo ao Mundo da Programação em Python!

Imagine-se explorando um novo mundo, onde cada linha de código é uma chave que abre portas para oportunidades incríveis!

Este livro é o seu guia nessa jornada, desenhado para ser acessível a todos, seja você alguém que nunca escreveu uma única linha de código em **Python** ou em qualquer outra linguagem, ou um programador que já domina outras linguagens e deseja aprender **Python** de forma rápida e eficaz.

Não se preocupe se você está começando do absoluto zero — este livro vai conduzi-lo passo a passo, desvendando cada conceito com clareza e simplicidade. Desde o básico, você será guiado por exemplos práticos e explicações detalhadas que facilitarão a compreensão de cada tópico.

Se você já tem experiência em outras linguagens de programação, encontrará aqui uma abordagem objetiva e eficaz para migrar seu conhecimento para o universo Python.

Por meio de comparações e exemplos concretos, você verá como aplicar suas habilidades prévias de forma eficiente nesta nova linguagem.

Este livro foi pensado para ser mais do que um manual técnico.

É uma ferramenta prática para o aprendizado de **Python**, capacitando você a transformar ideias em realidade, desde pequenos scripts até projetos mais complexos.

Vamos explorar juntos o poder dessa linguagem e como ela pode abrir novas portas para sua carreira e criatividade.

Oportunidades que Esperam por Você

Antes de mergulharmos no universo Python, vamos dar uma olhada nas oportunidades que você encontrará ao dominar essa linguagem poderosa.



Python se destaca pela sua versatilidade, sendo amplamente utilizado em várias áreas da tecnologia e da ciência, desde desenvolvimento web até automação de tarefas, análise de dados e criação de sistemas inteligentes.

Desenvolvimento Web

Python é amplamente utilizado no **desenvolvimento web**, com frameworks poderosos como **Django** e **Flask**.

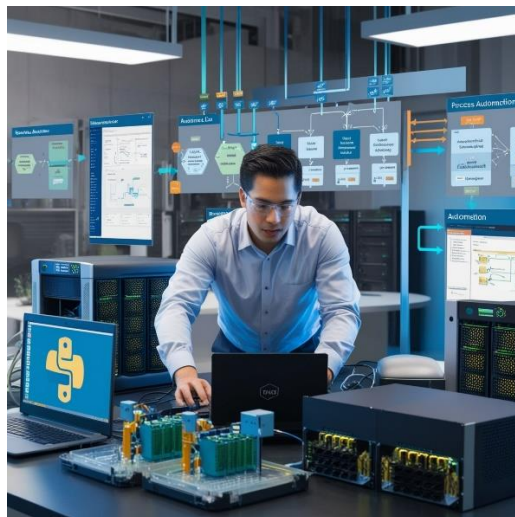


Esses frameworks permitem a criação de aplicações web robustas e escaláveis com eficiência. Exemplos de aplicações incluem:

- **Desenvolvimento de sites dinâmicos:** sites que se adaptam ao comportamento do usuário.
- **Aplicações web escaláveis:** plataformas como redes sociais e e-commerce.
- **Sistemas de gestão de conteúdo (CMS):** criação de blogs, portais de notícias e sites empresariais.

Automação de Processos

Python facilita a automação de tarefas rotineiras, desde scripts simples até processos empresariais complexos.



Áreas onde Python é usado para automação incluem:

- **Automação de TI:** gerenciar e monitorar redes, servidores e bancos de dados.
- **Automação de marketing:** automatizar campanhas de email, coleta de dados e interações com clientes.
- **Automação pessoal:** organizar arquivos no seu computador, renomear lotes de arquivos ou enviar lembretes automaticamente.

Ciência de Dados (Data Science)

Python também é a linguagem preferida no campo da **Ciência de Dados**, onde é utilizada para manipular, analisar e extrair insights valiosos de grandes volumes de dados.

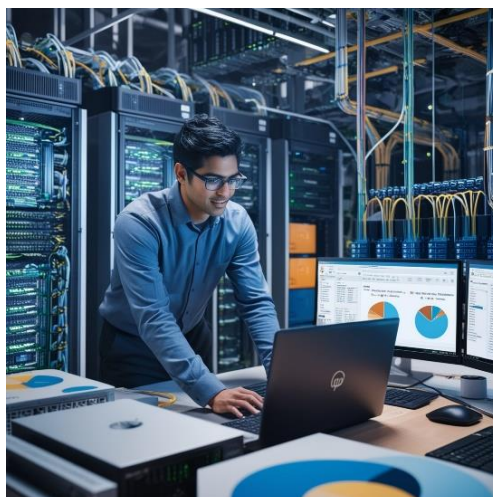


Com bibliotecas como **Pandas** e **NumPy**, Python permite que você:

- **Análise de dados de negócios:** ajudar empresas a tomar decisões baseadas em dados e otimizar operações.
- **Pesquisa científica:** analisar grandes conjuntos de dados em áreas como biologia, física e ciências sociais.
- **Engenharia de dados:** construir e otimizar sistemas para coleta, armazenamento e processamento de dados.

Big Data e Análise de Dados

Com o aumento exponencial do volume de dados gerados diariamente, **Big Data** tornou-se um campo crítico, e Python é uma ferramenta indispensável para lidar com grandes conjuntos de dados.

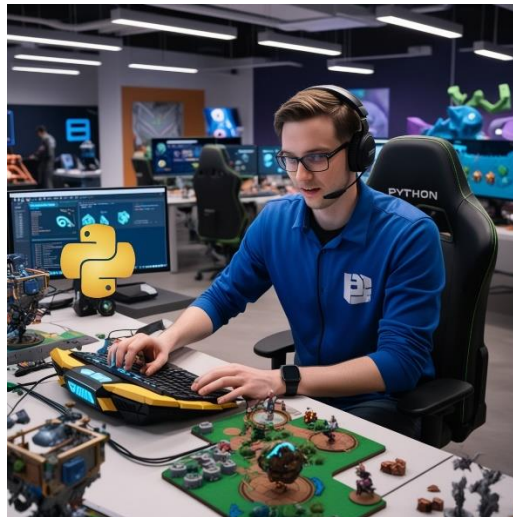


Bibliotecas como **Dask** e **PySpark** permitem:

- **Processamento de grandes volumes de dados:** manipular e analisar conjuntos de dados massivos de forma eficiente.
- **Visualização de dados:** criar gráficos e dashboards interativos com **Matplotlib** e **Seaborn**.
- **Análise de tendências:** prever e identificar padrões em grandes volumes de dados.

Desenvolvimento de Jogos

Python também encontra seu lugar no **desenvolvimento de jogos**, com bibliotecas como **Pygame** permitindo que desenvolvedores criem jogos 2D simples.

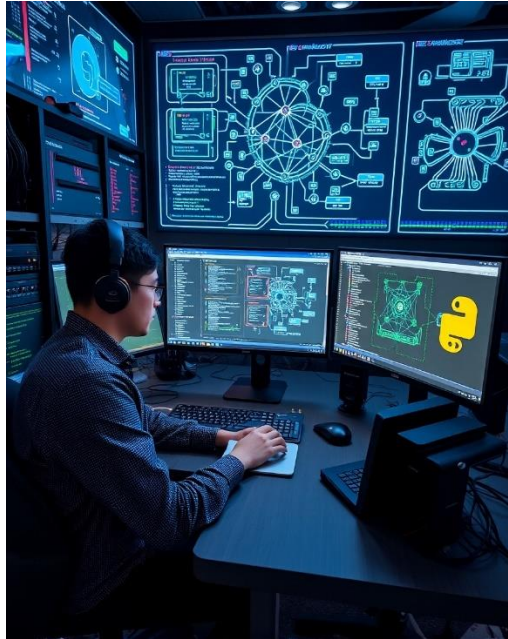


Aplicações incluem:

- **Desenvolvimento de jogos educativos:** para ensinar conceitos interativos.
- **Prototipagem de jogos:** criar versões iniciais de jogos antes de migrar para plataformas mais complexas.
- **Simulações interativas:** ambientes simulados para pesquisa e aprendizado.

Segurança da Informação e Hacking Ético

Python é amplamente usado por especialistas em **segurança da informação** e hackers éticos para automação de testes de penetração, análise de vulnerabilidades e desenvolvimento de ferramentas de segurança.



Aplicações incluem:

- **Desenvolvimento de scripts de segurança:** automatizar a verificação de falhas de segurança.
- **Análise forense:** analisar dados e metadados em arquivos suspeitos.
- **Testes de penetração (pentest):** avaliar a segurança de redes e sistemas.

Inteligência Artificial e Machine Learning

Python também lidera o campo da **Inteligência Artificial (IA)** e do **Machine Learning (Aprendizado de Máquina)**, onde é amplamente utilizado para criar modelos que aprendem com os dados e fazem previsões.



Bibliotecas como **scikit-learn**, **TensorFlow** e **PyTorch** facilitam o desenvolvimento de soluções como:

- **Análise preditiva:** prever tendências de mercado ou comportamento do consumidor.
- **Reconhecimento de padrões:** identificar fraudes em transações financeiras.
- **Personalização:** recomendar produtos ou conteúdos com base no histórico do usuário.

Visão Computacional (Computer Vision)

A Visão Computacional, que envolve permitir que computadores interpretem informações visuais do mundo, é outra área em que Python brilha.



Com a biblioteca **OpenCV**, você pode:

- **Deteção de objetos:** identificar e localizar objetos em imagens ou vídeos.
- **Reconhecimento facial:** verificar identidades com base em características faciais.
- **Veículos autônomos:** ajudar carros a entender e navegar por ambientes complexos.

Inteligência Artificial Generativa (IA Generativa)

A **IA Generativa** permite a criação de novos conteúdos, como imagens, texto ou música, a partir de padrões aprendidos em dados.



Aplicações incluem:

- **Criação de arte digital:** gerar novas obras de arte com base em estilos aprendidos.
- **Geração de texto e música:** produzir histórias, poemas ou composições musicais originais.
- **Criação de modelos 3D:** usar IA para criar modelos tridimensionais detalhados para design e animação.

Estas são apenas algumas das áreas em que o Python desempenha um papel crucial, e dominar essa linguagem pode abrir um mundo de oportunidades para você. Cada uma dessas áreas está repleta de desafios emocionantes e potencial para inovação, tornando o aprendizado de Python uma habilidade valiosa e recompensadora.

Python: A Chave para o Futuro

Se você está mirando nas carreiras mais dinâmicas e em alta demanda da atualidade, aprender **Python** não é apenas uma vantagem — é essencial.

Seja seu objetivo se tornar um **Engenheiro de Aprendizado de Máquina**, um **Cientista de Dados**, ou um **Analista de Dados**, Python será a ferramenta que abrirá as portas para essas oportunidades. Sua versatilidade, simplicidade e poder de processamento fazem dela a linguagem preferida de empresas e profissionais que lideram a transformação digital em todo o mundo.

Python não é apenas uma linguagem de programação; é uma ponte que conecta o conhecimento técnico à inovação, permitindo que você desenvolva soluções inteligentes e eficientes em praticamente qualquer área.

Desde a criação de **modelos de machine learning** que revolucionam a forma como tomamos decisões até o desenvolvimento de sistemas que analisam vastos volumes de dados em tempo real, Python é a espinha dorsal de muitos dos avanços tecnológicos que moldam o futuro.

Além disso, a curva de aprendizado de Python é suave o suficiente para iniciantes, mas sua profundidade é infinita para aqueles que buscam dominar áreas mais complexas.

Com uma vasta gama de bibliotecas e pacotes especializados, Python permite que você desenvolva desde pequenos scripts automatizados até **projetos de inteligência artificial** avançados. Ferramentas como **pandas**, **NumPy**, **scikit-learn**, **TensorFlow** e muitas outras são amplamente usadas por grandes corporações e startups de inovação para resolver problemas de alta complexidade.

Não importa se você está no início da sua jornada ou buscando aprimorar suas habilidades: dominar Python é o investimento mais inteligente que você pode fazer para o seu futuro profissional.

Python não é apenas uma linguagem de programação — é a **linguagem do futuro**, a chave que pode desbloquear carreiras, inovação e oportunidades de crescimento ilimitadas.

Portanto, seja qual for o seu sonho — desenvolver **aplicações de IA**, transformar-se em um especialista em **ciência de dados**, ou criar **soluções automatizadas** que impactam milhões de vidas — Python será seu maior aliado.

Invista no aprendizado desta linguagem hoje, e você estará preparando o terreno para uma carreira de sucesso, em constante evolução e repleta de possibilidades.

Vamos Começar a Codificar!

Chegou a hora de colocar a mão na massa!

Este livro é totalmente **prático e interativo**, e a única maneira de realmente aprender Python é **escrevendo código**.

Ler sobre os conceitos é apenas o primeiro passo, mas o verdadeiro aprendizado acontece quando você começa a codificar por conta própria, testando e ajustando suas soluções.

Acredite, errar faz parte do processo — e não apenas no início. **Você vai cometer erros**, vai encontrar desafios que parecem difíceis de superar, mas cada erro é uma oportunidade de aprendizado. **Erros não são falhas**, eles são parte essencial da jornada de qualquer programador, e a cada correção, você estará mais próximo de dominar Python.

Quando algo parecer complicado ou confuso, não desanime. Isso é totalmente normal!

Programação é como resolver um quebra-cabeça: às vezes, você precisa tentar várias peças antes de encontrar a solução correta.

O segredo é a **persistência**. Cada linha de código que você escreve, cada desafio que supera, fortalece sua compreensão e suas habilidades. Com paciência, prática e curiosidade, você verá como os conceitos vão começar a fazer sentido e, antes que perceba, estará desenvolvendo suas próprias soluções com confiança.

Portanto, o maior conselho que posso dar é: **não tenha medo de tentar**. Experimente, erre, corrija, e tente de novo.

Você não está sozinho nessa jornada — Python foi projetado para ser intuitivo e acessível, mas como toda habilidade valiosa, requer prática. O verdadeiro poder do aprendizado vem de sua **própria experiência**.

Quanto mais você praticar, mais natural o código vai se tornar, e mais longe você poderá ir.

Então, vamos nessa! O importante é **começar**, e estou aqui para guiar você em cada passo desse caminho.

Preparando-se para a Jornada

Antes de avançarmos, precisamos fazer algumas configurações básicas. Tudo o que você precisa é de um laptop e uma boa conexão à internet. Não se preocupe com instalações complexas; faremos tudo da forma mais simples e direta.

Primeiro passo: Configuração do ambiente!

Então, está pronto para começar esta emocionante jornada no mundo da programação em Python?

No próximo capítulo vamos configurar seu ambiente e iniciar a aventura. Boa sorte e divirta-se codificando!



PARA SABER MAIS ...

Ao final de cada capítulo, você encontrará esta seção chamada "**Para Saber Mais...**".

Aqui, nosso objetivo é fornecer recursos adicionais para aprofundar os tópicos abordados e complementar seu aprendizado.

Python é uma linguagem vasta, com inúmeras possibilidades, e nem sempre é possível cobrir tudo de forma exaustiva em um único capítulo. Por isso, esta seção será dedicada a:

- **Expansão de conceitos:** Se algum assunto foi apresentado de forma simples ou introdutória, você encontrará aqui dicas de como se aprofundar no tema.
- **Recursos adicionais:** Recomendações de livros, artigos, cursos online e ferramentas que podem ajudar a ampliar seu conhecimento.
- **Complemento de informações:** Às vezes, surgem aspectos mais complexos de um tópico que podem ser explorados por quem deseja ir além. Aqui, traremos sugestões para que você possa explorar esses pontos com mais detalhes.
- **Atualizações e tendências:** Python está em constante evolução, e novas ferramentas e técnicas surgem o tempo todo. Nesta seção, você encontrará dicas de como se manter atualizado.

Essa seção não é obrigatória, mas é uma oportunidade valiosa para quem quer **ir além e mergulhar mais fundo** nos temas abordados.

Utilize-a como uma bússola para explorar novos caminhos no seu aprendizado de Python e expandir seus horizontes.

Nesse primeiro capítulo falamos sobre a importância da linguagem Python e a facilidade com que podemos aprender essa linguagem. Independente da linguagem de programação, as últimas pesquisas e relatórios das principais Ongs e Empresas de Tecnologia indicam que isso se torna “**Uma necessidade crescente em nosso mundo digital**”.

A programação, independente da linguagem, tornou-se uma habilidade fundamental no século XXI. Mas por quê? Vamos explorar alguns motivos e referências que comprovam essa necessidade crescente:

1. O Mundo é Digital

- **Automação:** A programação permite automatizar tarefas repetitivas, aumentando a produtividade e liberando tempo para atividades mais criativas.
- **Inovação:** A capacidade de criar soluções personalizadas para problemas específicos impulsiona a inovação em todos os setores.
- **Inteligência Artificial:** A IA está transformando diversas áreas, e a programação é essencial para desenvolver e utilizar essas tecnologias.

2. Mercado de Trabalho

- **Demanda alta:** A crescente demanda por profissionais de tecnologia impulsiona o mercado de trabalho. Alguns estudos indicam que haverá um “gap” de mais de 1 milhão de vagas não preenchidas na área de tecnologia nos próximos 5 anos (segundo **Code.org** - <https://code.org/>).
- **Salários competitivos:** Profissionais da área de programação geralmente possuem salários acima da média.
- **Oportunidades diversas:** As possibilidades de carreira são vastas, desde desenvolvimento de software até ciência de dados. (essas duas últimas afirmações foram retiradas de relatórios do **LinkedIn** (<https://www.linkedin.com/>) e da “**Burning Glass Technologies**” (<https://www.burningglassinstitute.org/>)).

3. Desenvolvimento Pessoal

- **Pensamento crítico:** A programação exige a resolução de problemas de forma lógica e estruturada, desenvolvendo o pensamento crítico.
- **Criatividade:** A programação é uma forma de expressão artística, permitindo criar soluções inovadoras e personalizadas.
- **Aprendizado contínuo:** A tecnologia evolui rapidamente, e a programação exige um constante aprendizado e adaptação.

Como referência dessas afirmações, assista ao vídeo do ex-presidente dos Estados Unidos, **Barack Obama**, indicando a necessidade dos jovens aprenderem a programar. Acesse o link: <https://www.youtube.com/watch?v=SfBRhilt2xl>

4. Impacto Social

- **Solução de problemas:** A programação pode ser utilizada para desenvolver soluções para problemas sociais, como desigualdade, pobreza e questões ambientais.
- **Acesso à informação:** A programação permite criar ferramentas e plataformas que democratizam o acesso à informação.

Referências para essa afirmação são encontradas nas organizações sem fins lucrativos, que tem como objetivo promover a educação da programação, tais como: **Code.org** (<https://code.org/>) e a **Girls Who Code** (<https://girlswhocode.com/>)

Em resumo

Aprender a programar não é apenas uma habilidade profissional, mas também uma ferramenta para o desenvolvimento pessoal e para contribuir para um mundo melhor. A capacidade de criar, automatizar e resolver problemas através da programação é cada vez mais valorizada em diversos setores.

2

Configurando o ambiente

Bem-vindo ao primeiro passo da sua jornada de aprendizado em Python!

Antes de começarmos a explorar os incríveis recursos dessa linguagem, é essencial que configuremos nosso ambiente de programação.

A escolha do ambiente certo pode fazer toda a diferença na sua experiência de aprendizado, especialmente se você é iniciante.

Existem muitas opções de IDEs (Ambientes de Desenvolvimento Integrados) disponíveis para programar em Python, como o **PyCharm**, **VS Code**, ou até o **Jupyter Notebook**. No entanto, para simplificar o processo e permitir que você comece a programar sem enfrentar barreiras técnicas, optamos por usar o **Google Colab**.

Esse ambiente é perfeito tanto para quem nunca programou quanto para aqueles que querem um ambiente prático e acessível.

Por Que Usar o Google Colab?

O **Google Colab**, ou **Colaboratory**, é uma ferramenta gratuita oferecida pelo Google que permite escrever e executar código Python diretamente no seu navegador, eliminando a necessidade de instalar softwares ou lidar com configurações avançadas. Tudo o que você precisa é de um navegador e uma conexão com a internet.

Aqui estão algumas das principais vantagens do Google Colab:

- **Acessibilidade:** Com apenas uma conta Google e um navegador, você pode começar a programar imediatamente. Não importa se você está usando um PC, Mac ou Linux, o Colab funciona em qualquer plataforma com acesso à web.
- **Facilidade de Uso:** A interface do Google Colab é amigável e intuitiva, projetada para ser simples de usar. Isso é especialmente útil para iniciantes,

que podem se concentrar no aprendizado da linguagem sem se preocupar com configurações complicadas.

- **Potência de Processamento:** O Colab permite que você acesse **GPUs** (Unidades de Processamento Gráfico) e **TPUs** (Unidades de Processamento de Tensores) gratuitamente. Isso é uma vantagem enorme, especialmente quando você começar a explorar áreas mais avançadas, como **machine learning** ou **ciência de dados**, que exigem maior poder de processamento.
- **Colaboração em Tempo Real:** Como o nome sugere, o Colab facilita a colaboração. Você pode compartilhar seus notebooks com colegas, professores ou amigos com um simples clique. Isso torna o processo de aprendizado colaborativo e interativo, permitindo que várias pessoas trabalhem em um projeto ao mesmo tempo, visualizando e editando o código em conjunto.
- **Integração com o Google Drive:** Seus notebooks podem ser facilmente salvos e organizados diretamente no seu **Google Drive**, facilitando o acesso e a organização dos seus projetos.
- **Recursos Pré-instalados:** O Colab já vem com várias bibliotecas Python populares pré-instaladas, como **NumPy**, **pandas**, **matplotlib**, **TensorFlow**, entre outras. Isso economiza o tempo que você gastaria configurando dependências no seu próprio ambiente local.

Foco no Aprendizado

Com o **Google Colab**, você pode se concentrar inteiramente no **aprendizado de Python**.

Ele elimina qualquer complicação relacionada a instalações, compatibilidade de software ou necessidade de hardware específico. Isso significa que você pode começar a praticar o código desde o primeiro minuto, sem interrupções.

Além disso, por ser uma plataforma baseada na nuvem, todos os seus projetos podem ser acessados de qualquer lugar, permitindo que você continue de onde parou, esteja no seu computador pessoal, no trabalho, ou em um dispositivo móvel.

Pronto para Começar?

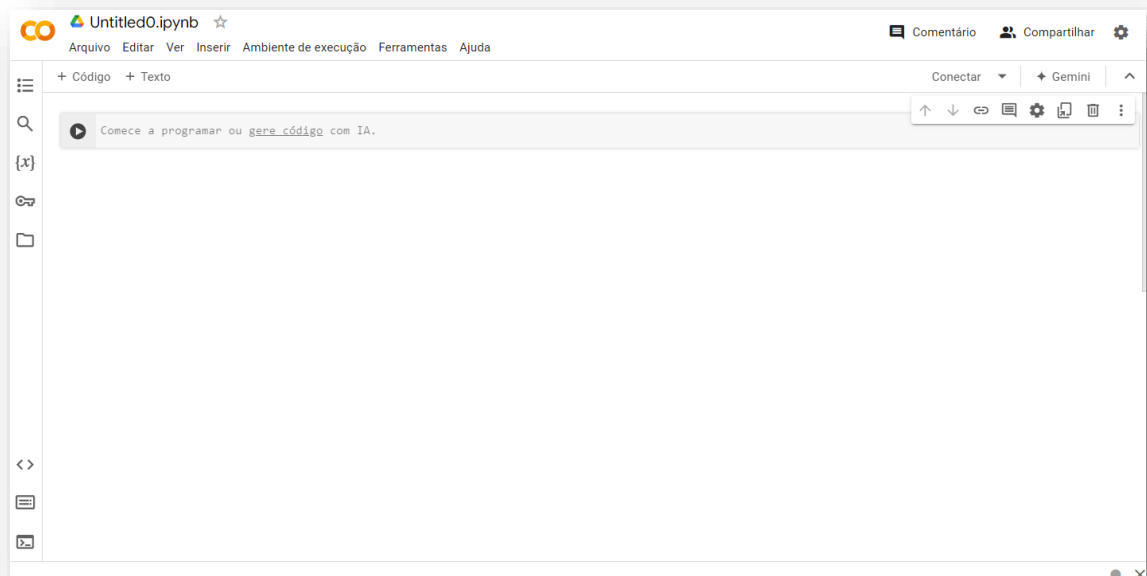
Agora que você conhece as vantagens de usar o **Google Colab**, está pronto para configurar o seu ambiente de programação. Ao longo deste livro, usaremos o Colab para escrever e testar nossos códigos.

Isso garantirá que você tenha uma experiência prática e acessível, aproveitando ao máximo o poder e a simplicidade do **Python**.

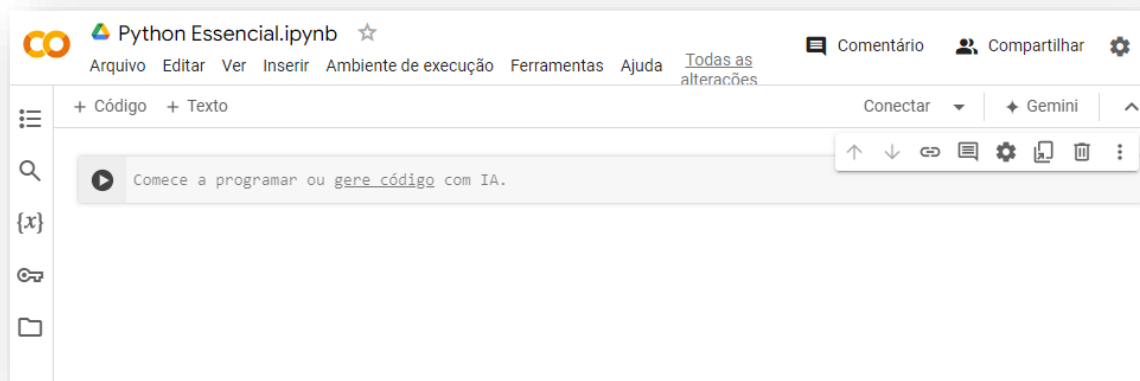
Passo a Passo para Configurar o Google Colab

Agora, vamos configurar seu ambiente de programação em poucos passos simples:

1. **Abra seu navegador favorito.**
2. **Pesquise "Google Colab".**
3. **Clique no primeiro link que aparece nos resultados da pesquisa.**
4. **Clique em "Novo notebook".** (Se você ainda não fez login com seu Gmail, faça isso primeiro. Você deve ter uma conta Google para poder utilizar o Google Colab).



5. **Agora você tem um notebook novo para começar sua jornada de aprendizado em Python.**
6. **Clique em "Untitled.ipynb" no canto superior esquerdo e mude o nome do arquivo para "Python Essencial".**



Pronto! Você configurou com sucesso seu ambiente de programação. Vamos começar a codificar!

Primeiros Passos com Python

Agora que você configurou seu ambiente de desenvolvimento no Google Colab e criou seu primeiro notebook chamado **"Python Essencial"**, chegou a hora de mergulharmos no código!

No **Google Colab**, o código é escrito em **células**, e você pode executar cada célula individualmente.

Isso permite testar e visualizar o funcionamento de cada trecho de código de forma rápida e prática, uma das grandes vantagens dessa ferramenta.

Hello World! – O Primeiro Passo de Todo Programador

Existe uma curiosa "tradição" — ou como muitos preferem chamar, uma **"maldição"** — no mundo da programação: o **primeiro programa** que todo iniciante deve escrever é aquele que imprime a mensagem **"Hello World!"** na tela.

Esse simples programa não faz nada de extraordinário, mas simboliza um **rito de passagem** que muitos programadores ao longo da história seguiram.

Essa tradição remonta aos primeiros dias da programação, quando o objetivo era testar se o sistema e as linguagens de programação estavam funcionando corretamente.

A ideia de imprimir **"Hello World!"** surgiu nos anos 70, com o livro *The C Programming Language*, de **Brian Kernighan** e **Dennis Ritchie**, os criadores da linguagem C. No exemplo do livro, o primeiro programa que eles mostraram foi justamente um código que exibia "Hello, World!". Desde então, essa simples frase

se tornou **um marco simbólico** para todos os iniciantes que começam sua jornada no mundo da programação.

Por que chamamos de **maldição**? Porque programadores experientes costumam dizer que qualquer pessoa que inicia sua jornada sem antes escrever o famoso **"Hello World!"** pode sofrer com bugs e problemas inexplicáveis no futuro. Embora seja apenas uma piada, muitos seguem religiosamente essa tradição.

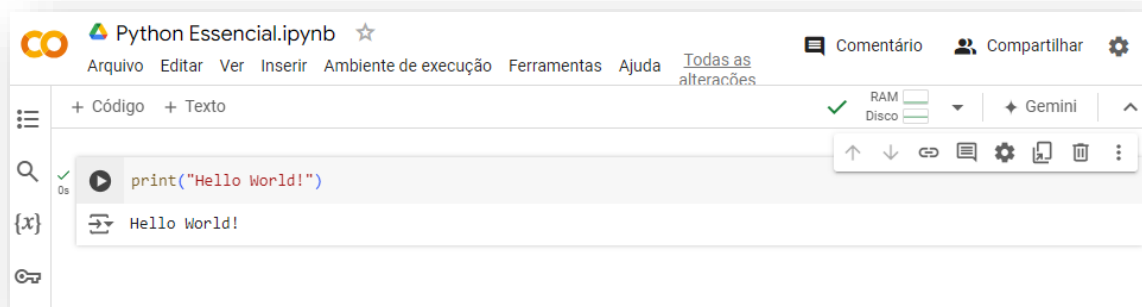
Agora, é a sua vez de participar desse momento histórico e escrever o seu **primeiro programa em Python**.

Escrevendo o Primeiro Código

Na primeira célula do seu notebook, escreva o seguinte código:

```
print("Hello World!")
```

Depois de digitar o código, pressione **Shift + Enter** para executá-lo ou simplesmente clique no ícone 



Se tudo der certo, o notebook irá executar o código e imprimir o texto "Hello World!" logo abaixo da célula.

Parabéns! Você acaba de dar o primeiro passo para se tornar um programador Python.

O Que Acabou de Acontecer?

Vamos analisar o que você acabou de fazer. O comando **print()** é uma função integrada do Python que exibe a mensagem passada entre os parênteses na tela. No nosso caso, a mensagem foi **"Hello World!"**. Python reconhece qualquer texto entre aspas como uma **string** (um conjunto de caracteres), e **print()** cuida de exibir esse texto no output (saída padrão da função, que é o console).

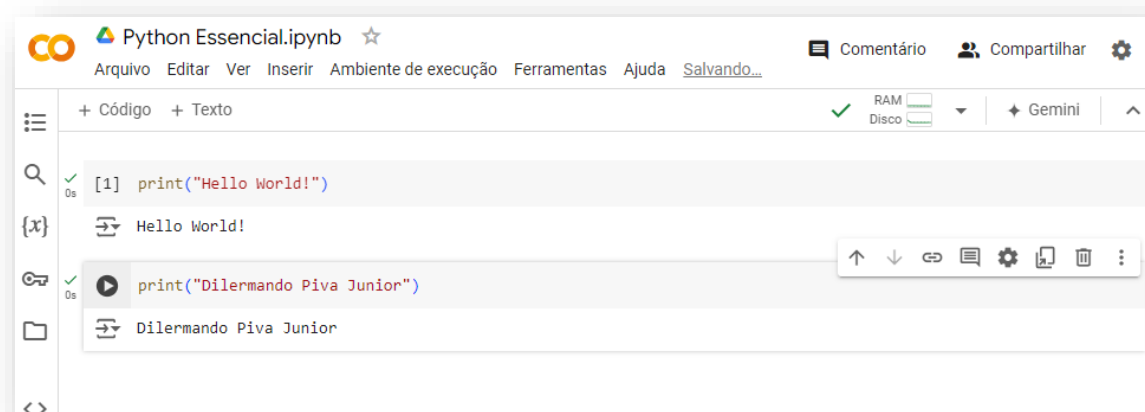
Embora seja um código extremamente simples, ele carrega um significado poderoso: agora, você entrou no mundo da programação.

Agora, vamos usá-la para imprimir seu nome. Digite o seguinte na próxima célula (para ir para a próxima célula clique no botão no canto superior esquerdo:



```
print("Seu Nome")
```

Depois de pressionar **Shift + Enter** ou ► você verá seu nome na saída. Não se esqueça de colocar o seu nome entre aspas simples ou duplas.



O que começou com "Hello World" logo se transformará em scripts e programas mais complexos que você desenvolverá ao longo desta jornada.

O Primeiro de Muitos Passos

Esse pequeno programa é apenas o início de uma jornada emocionante.

A partir de agora, você começará a explorar o verdadeiro potencial do Python para resolver problemas práticos, desenvolver aplicações, e mergulhar em áreas como análise de dados e automação.

Não subestime o poder dessa simples linha de código — ela é o primeiro degrau de uma escada que o levará a muitas oportunidades e desafios fascinantes.

Cada linha de código escrita será um passo em direção ao domínio de habilidades que podem transformar sua carreira e sua forma de pensar.

Agora que você deu o primeiro passo, é hora de se preparar para o próximo.

No próximo capítulo, vamos explorar como o Python lida com **variáveis e tipos de dados**, dois conceitos fundamentais para armazenar e manipular informações no mundo da programação.

Respire fundo, porque a aventura está só começando. Python é a sua chave para abrir as portas da programação, e cada capítulo será uma nova descoberta.



PARA SABER MAIS ...

Documentação oficial do Colab: Você pode acessar a documentação oficial do Google Colab, onde encontrarão informações detalhadas sobre todas as funcionalidades e recursos disponíveis. Link: <https://colab.google/resources/>

Documentação sobre o Google Colab elaborada pela comunidade Medium:
Link: <https://medium.com/google-colab>

Tutoriais em vídeo: Existem vários canais do YouTube e sites, tais como: <https://realpython.com/> e <https://www.youtube.com/@realpython> entre milhares de outros, que oferecem tutoriais completos sobre Python e o uso do Colab.

Livros e cursos: Acesse <https://pypro.com.br> para Livros e Cursos sobre a linguagem Python.

3

Variáveis e Tipos de Dados

Agora que você já deu seus primeiros passos no mundo da programação com Python, vamos aprofundar um pouco mais e falar sobre um dos conceitos fundamentais: **variáveis**.

Em Python, as variáveis são como **caixinhas** ou **contêineres** onde podemos armazenar diferentes tipos de informações.

Isso permite que reutilizemos esses dados ao longo do nosso código, o que torna a programação muito mais eficiente e organizada.

Vamos simplificar ainda mais. Pense em uma variável como um **rótulo** que você coloca em algo para identificá-lo facilmente mais tarde.

Imagine que você tem uma caixa e dentro dessa caixa coloca um bilhete com a frase "Hello World!".

Agora, se alguém lhe perguntar o que está dentro da caixa, você só precisa consultar o rótulo (a variável) para descobrir que contém "Hello World!".

Por exemplo, quando imprimimos "Hello World!" no nosso primeiro programa, essa frase é um tipo de dado que chamamos de **string** (não se preocupe, vamos falar mais sobre tipos de dados em breve).

Mas, e se quisermos usar essa frase várias vezes no código?

Não seria prático ter que digitá-la toda vez, certo?

A solução é armazená-la em uma variável. Veja como:

```
a = "Hello World!"  
print(a)
```

No exemplo acima, o valor "Hello World!" foi **atribuído** a uma variável chamada **a**.

Isso significa que agora **a** é um nome que podemos usar para se referir a "Hello World!".

Cada vez que usarmos **a**, o Python vai entender que estamos nos referindo à frase armazenada nela.

No entanto, há uma questão importante aqui: o nome da variável **a** não nos dá nenhuma pista sobre o que ela representa. Embora funcione, não é uma boa prática nomear variáveis com letras aleatórias ou nomes que não descrevem seu conteúdo.

Agora, imagine que a frase "Hello World!" é uma saudação. Nesse caso, um nome de variável mais apropriado poderia ser **saudacao** (note que utilizei a palavra saudação, sem acentos ou c-cedilha – isso é importante, pois as linguagens de programação foram construídas essencialmente na língua inglesa que não possui palavras acentuadas). Isso torna o código mais claro e fácil de entender, especialmente quando seu programa começa a crescer em complexidade.

Um Ponto Importante: Use Nomes Descritivos e em Inglês

Quando nomeamos variáveis, é importante seguir algumas boas práticas. Primeiro, evite usar **acentos** ou caracteres especiais como **ç**. Isso ocorre porque a maioria das linguagens de programação, incluindo Python, foi desenvolvida em um contexto internacional, onde o inglês é a língua predominante e os caracteres especiais são raramente usados.

Além disso, como o inglês é a língua padrão na programação, é altamente recomendável que você use **nomes de variáveis em inglês** sempre que possível. Isso não apenas facilita a leitura do seu código por outros programadores ao redor do mundo, mas também ajuda a manter um padrão internacional.

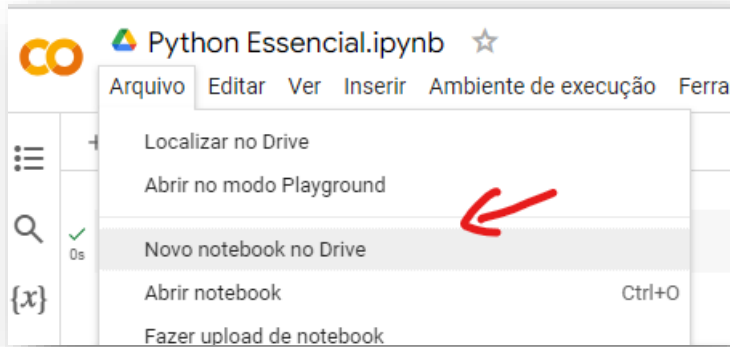
Portanto, em vez de usar **saudacao**, poderíamos nomear a variável como **greetings**, que significa "saudação" em inglês.

```
greetings = "Hello World!"  
print(greetings)
```

Agora, o valor "Hello World!" está armazenado na variável **greetings**, que faz muito mais sentido e segue as boas práticas de programação. Isso torna o código mais legível e fácil de ser entendido por qualquer pessoa que possa trabalhar com você no futuro.

Para executar o código acima no Google Colab, abra um novo notebook...

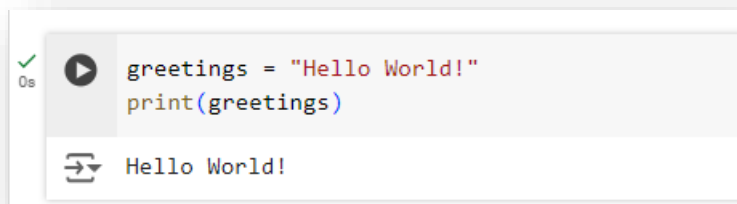
Se estiver em um notebook aberto, clique na opção “Arquivo” no canto superior esquerdo... e em seguida clique em “Novo Notebook no Drive”, como indica a imagem a seguir:



Altere o nome do notebook, de **Untitled0.ipynb** para **Python Essencial – Cap3.ipynb**.

Pronto! Agora podemos inserir o código acima em uma célula e executá-la...

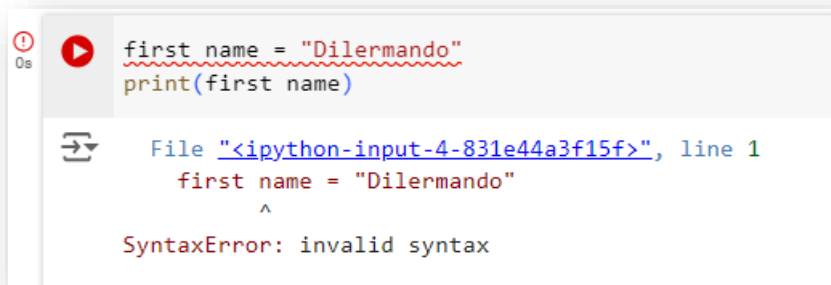
Executando o código acima agora ficaria assim:



Além disso, existem certas convenções de nomenclatura que você precisa seguir ao dar um nome à sua variável.

Digamos que você queira armazenar seu primeiro nome em uma variável.

```
first name = "Dilermando"
print(first_name) # OUTPUT: SyntaxError: invalid syntax
```

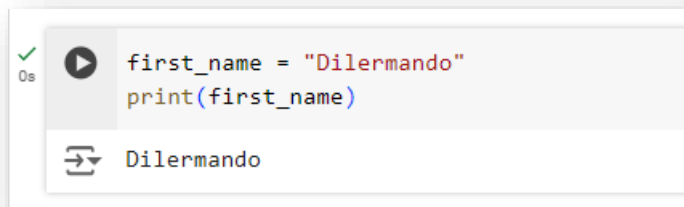


```
first name = "Dilermando"
print(first name)
```

File "<ipython-input-4-831e44a3f15f>", line 1
first name = "Dilermando"
^
SyntaxError: invalid syntax

O código acima exibirá um erro de sintaxe porque você não pode usar espaço entre as palavras ao criar variáveis; em vez disso, você pode usar sublinhado (_).

```
first_name = "Dilermando"
print(first_name) # OUTPUT: Dilermando
```



```
first_name = "Dilermando"
print(first_name)
```

Dilermando

Agora aqui está um exercício simples para você. Crie 5 variáveis diferentes e armazene sua idade, primeiro nome, sobrenome, nome do país e número de telefone. Abaixo está o meu.

Atenção! Sempre tente resolver antes de ver a resposta... Isso será importante para você fixar o conteúdo.

```
age = 53
first_name = "Dilermando"
last_name = "Piva Jr."
country = "Brazil"
phone_number = 999999999
```

```
✓ [6] age = 53
    first_name = "Dilermundo"
    last_name = "Piva Jr."
    country = "Brazil"
    phone_number = 999999999
```

Note que usamos diferentes tipos de valores para essas variáveis: um número inteiro para a idade e o número de telefone, e strings (textos) para os nomes e o país.

Mas o que exatamente são esses "tipos" de dados? É isso que vamos explorar a seguir.

Tipos de Dados

Em Python, assim como em outras linguagens de programação, os **tipos de dados** representam diferentes categorias de informações que você pode armazenar e manipular em variáveis.

Cada tipo de dado é adequado para uma tarefa específica, e saber qual tipo de dado usar é uma habilidade essencial na programação.

A escolha do tipo de dado correto ajuda o Python a entender como lidar com a informação de forma eficiente e precisa. Por exemplo:

- Quando temos dados de texto, chamamos isso de **string (str)**. Exemplo: "Hello World!", "abcd", "xyz". (Os dados de string (str) estão sempre entre aspas; pode ser uma aspa simples (' ') ou uma aspa dupla (" ")).
- Quando nossos dados são um número, chamamos isso de **inteiro (int)**. Exemplo: 34, 65, 0, 1.
- Quando nossos dados são um número decimal (ou pertencentes ao conjunto dos números reais), chamamos isso de **float**. Exemplo: 3.14, 0.343, 453.34324.
- Quando nossos dados estão na forma de Verdadeiro ou Falso, chamamos isso de **booleano (bool)**.

```
age = 53                #==> int
first_name = "Dilermundo" #==> str
last_name = "Piva Jr."   #==> str
country = "Brazil"       #==> str
```

```
phone_number = 999999999 #==> int
weight = 89.2             #==> float
married = True            #==> bool
```

```
age = 53                  #==> int
first_name = "Dilermando" #==> str
last_name = "Piva Jr."   #==> str
country = "Brazil"       #==> str
phone_number = 999999999 #==> int
weight = 89.2            #==> float
married = True           #==> bool
```

Vamos ver alguns exemplos para todos esses 4 tipos de dados... um por um.

Tipo Texto – String - str

```
text = "Olá! Como você está?"
```

Para saber o tipo de dado armazenado em uma variável, existe uma função chamada `type()`. Vamos usar a função `type` para ver que tipo de dado está presente na variável "text".

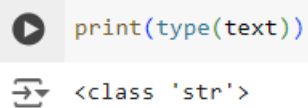
```
type(text)
```

```
text = "Olá! Como você está?"
type(text)
```

```
str
```

Quando você executa o código acima, obterá "str" como sua saída, indicando que a variável `text` possui dados do tipo **string**. Você pode simplesmente executar o código acima no Google Colab, mas se estiver usando outras plataformas para codificar, você precisa usar a função `print()`.

```
print(type(text))
```

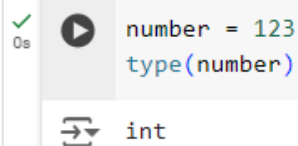


```
print(type(text))  
<class 'str'>
```

Tipo Inteiro - int

De forma similar:

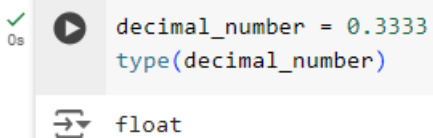
```
number = 123  
type(number)
```



```
✓ 0s ▶ number = 123  
      type(number)  
      int
```

Tipo Decimal / Real - float

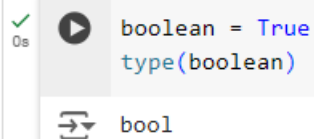
```
decimal_number = 0.3333  
type(decimal_number)
```



```
✓ 0s ▶ decimal_number = 0.3333  
      type(decimal_number)  
      float
```

Tipo Booleano - bool

```
boolean = True  
type(boolean)
```



```
✓ 0s ▶ boolean = True  
      type(boolean)  
      bool
```

Aqui criamos apenas um exemplo simples para cada tipo de dado, mas agora é a sua vez de experimentar.

Crie suas próprias variáveis, teste com diferentes valores, nomes e tipos de dados. Isso é fundamental para você ganhar familiaridade com a linguagem.

Não tenha medo de explorar: tente atribuir números, palavras, frases e até valores booleanos como True e False a variáveis. Apenas brinque com isso!

Quanto mais você praticar, mais confortável ficará com a ideia de manipular dados em Python.

A próxima etapa da nossa jornada envolve **manipular** esses dados. É aí que entram os **operadores** — ferramentas que nos permitem realizar ações sobre variáveis e valores.

Em Python, temos diferentes tipos de operadores que servem para diversas funções, como realizar cálculos, comparar valores, ou até mesmo verificar se certos elementos fazem parte de um grupo.

Esses operadores serão essenciais para que você possa **manipular e trabalhar** com os dados que já começamos a aprender aqui.

Então, continue explorando variáveis e tipos de dados, e prepare-se para entender como usar operadores para dar **vida e funcionalidade ao seu código**. No próximo capítulo, você verá como é fácil realizar operações e comparações em Python, o que será a base para criar programas dinâmicos e interativos.



PARA SABER MAIS ...

Python Documentation: Aprofundar na análise léxica, palavras reservadas e literais. https://docs.python.org/3/reference/lexical_analysis.html

Python Documentation: Valores e tipos de dados. <https://docs.python.org/3/reference/datamodel.html#objects-values-and-types>

Livros e cursos: Acesse <https://pypro.com.br> para Livros e Cursos sobre a linguagem Python. O curso “*Python Profissional: do Zero ao Mercado de Trabalho*” apresenta em detalhes a definição de variáveis e os diversos tipos de dados em detalhe.

4

Operadores

Até agora, aprendemos como armazenar e manipular diferentes tipos de dados em variáveis.

Mas, como podemos **interagir** com esses dados?

Como podemos **realizar cálculos**, **comparar valores** ou **tomar decisões lógicas** com base nas informações que nossas variáveis contêm?

As respostas estão nos **operadores**.

Em Python, os operadores são símbolos especiais ou palavras-chave que permitem realizar diversas operações com variáveis e valores.

Eles são as ferramentas que usamos para **processar dados**, realizar **cálculos matemáticos**, **comparar informações** e até mesmo **combinar condições lógicas**. Essencialmente, operadores são fundamentais para qualquer linguagem de programação, pois permitem transformar e analisar dados de forma prática e eficiente.

Você pode pensar nos operadores como as **ações** que seu código pode executar. Por exemplo, se você quer somar dois números, comparar a idade de duas pessoas, ou verificar se um item está presente em uma lista, são os operadores que fazem isso acontecer.

Existem diferentes tipos de operadores em Python, cada um com um propósito específico.

A seguir, vamos explorar os **6 principais tipos de operadores** que você usará frequentemente ao programar:

- **Operadores Aritméticos:** Permitem realizar operações matemáticas básicas, como soma, subtração, multiplicação e divisão, que são essenciais para manipular números em Python.
- **Operadores de Comparação:** Esses operadores são usados para comparar valores, verificando, por exemplo, se um número é maior, menor ou igual a

outro. Eles são fundamentais quando seu programa precisa tomar decisões com base em comparações.

- **Operadores de Atribuição:** Usamos operadores de atribuição para atribuir valores às variáveis. Já vimos o operador mais comum, o `=`, mas existem muitos outros que facilitam operações mais complexas.
- **Operadores Lógicos:** Muito úteis quando precisamos combinar múltiplas condições em uma única verificação. Operadores como `and`, `or` e `not` permitem criar expressões lógicas poderosas para seu código.
- **Operadores de Identidade:** Esses operadores verificam se duas variáveis referenciam o mesmo objeto na memória, sendo úteis em verificações de identidade entre objetos.
- **Operadores de Associação:** Nos permitem verificar se um elemento está presente em uma coleção, como uma lista ou string. Isso é especialmente útil quando trabalhamos com grandes conjuntos de dados.

Esses operadores são ferramentas essenciais para construir **lógica e funcionalidade** nos seus programas.

Eles permitem que você vá além da simples armazenagem de dados em variáveis e comece a **manipular, transformar e tomar decisões** com base nesses dados.

Vamos explorar cada um desses operadores detalhadamente, entendendo como e quando usá-los, e aplicando-os em exemplos práticos.

Operadores Aritméticos

Os Operadores Aritméticos são usados para realizar operações aritméticas, como Adição, Subtração, Multiplicação, Divisão, Módulo (resto da divisão), Exponenciação (potência) e Divisão de Piso (quociente sem resto). Abaixo estão os símbolos que usaremos para realizar essas operações.

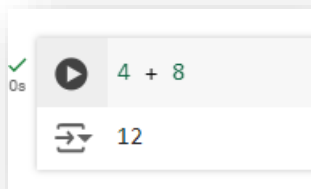
- `+` # Adição
- `-` # Subtração
- `*` # Multiplicação
- `/` # Divisão
- `//` # Divisão de Piso (quociente sem resto)
- `%` # Módulo (resto da divisão)

. ** # Exponenciação (potência)

Vamos usar os símbolos um por um e realizar diferentes operações aritméticas.

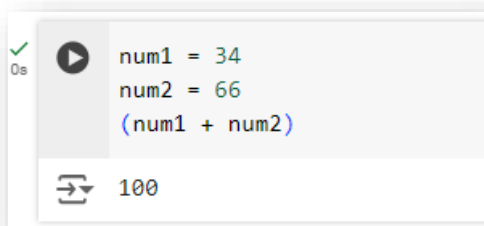
Adição:

```
4 + 8
```



Também podemos adicionar os valores armazenados dentro de variáveis.

```
num1 = 34  
num2 = 66  
(num1 + num2)
```



Experimente diferentes números.

Vamos, então, passar para a subtração.

Subtração:

```
num1 = 45  
num2 = 25  
56 - 34 # 22  
56 + 34 - 29 # 61
```

Vamos tentar com variáveis.

```
(num1 - num2) # 20
```

```
✓ [4] num1 = 45  
0s    num2 = 25  
      56 - 34  
      ↔ 22  
✓ [5] 56 + 34 - 29  
0s    ↔ 61  
✓ [6] (num1 - num2)  
0s    ↔ 20
```

Multiplicação:

Para multiplicar, usamos o símbolo asterisco (*).

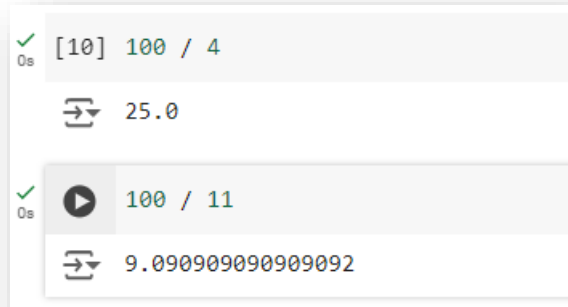
```
num1 = 467  
num2 = 345  
262 * 3  
45 * 3 + 35 - 23  
(num1 * num2)
```

```
✓ [7] num1 = 467  
0s    num2 = 345  
      262 * 3  
      ↔ 786  
✓ [8] 45 * 3 + 35 - 23  
0s    ↔ 147  
✓ [9] (num1 * num2)  
0s    ↔ 161115
```

Divisão:

Para Divisão, usamos a barra (/).

```
100 / 4
100 / 11
```

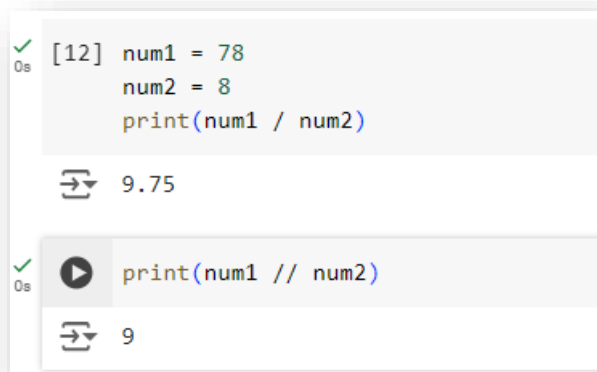


Quando dividimos dois números usando a divisão normal (/), obteremos um valor de saída decimal. Se você quiser que sua saída seja um inteiro, pode usar a divisão de piso (/).

Divisão de Piso:

Para Divisão de Piso, usaremos duas barras (/).

```
num1 = 78
num2 = 8
print(num1 / num2)
print(num1 // num2)
```



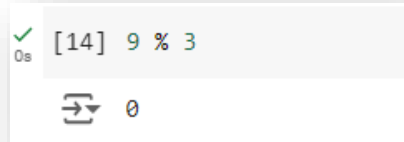
Ambos Divisão e Divisão de Piso nos darão o valor que obtemos como quociente após dividir dois números, mas e se quisermos o valor do restante?

Para obter o valor do restante, podemos usar o Operador Módulo (%).

Módulo ou Resto da Divisão

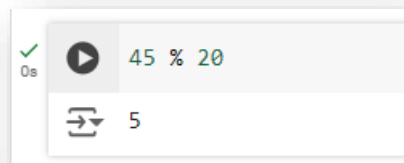
% é o símbolo para Operação Módulo.

9 % 3



Como você sabe, quando dividimos 9 por 3, o restante será 0.

45 % 20



Vamos tentar Exponenciação.

Exponenciação:

Usando o Operador Exponenciação (**), podemos encontrar o quadrado, cubo e mais de qualquer número. Na Exponenciação:

- $2^2 = 2^{**}2 = 2 \times 2$
- $2^3 = 2^{**}3 = 2 \times 2 \times 2$
- $2^4 = 2^{**}4 = 2 \times 2 \times 2 \times 2$ e assim por diante.

Vamos ver alguns exemplos de código.

```
3**2
65**12
num = 89
power = 6
print(num**power)
```

```
✓ [16] 3**2
0s
→ 9

✓ [17] 65**12
0s
→ 5688009063105712890625

✓ [18]
0s
▶ num = 89
  power = 6
  print(num**power)
→ 496981290961
```

Antes de passar para a próxima seção, experimente todos os operadores de maneiras diferentes.

Operadores de Comparação

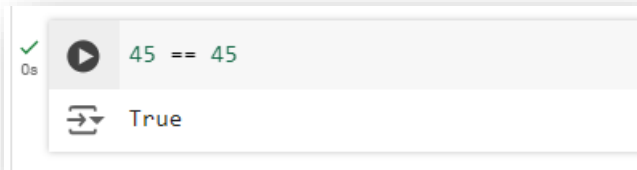
Os Operadores de Comparação são usados para comparar dois valores. É tão simples quanto isso. Abaixo estão todos os Operadores de Comparação:

- `==` # Igual a
- `!=` # Diferente de
- `<` # Menor que
- `>` # Maior que
- `<=` # Menor ou igual a
- `>=` # Maior ou igual a

Esses operadores retornarão valores booleanos como saída.

Igual a (==): Este operador compara se dois valores são iguais entre si.

```
45 == 45          # Vai Retornar True
```

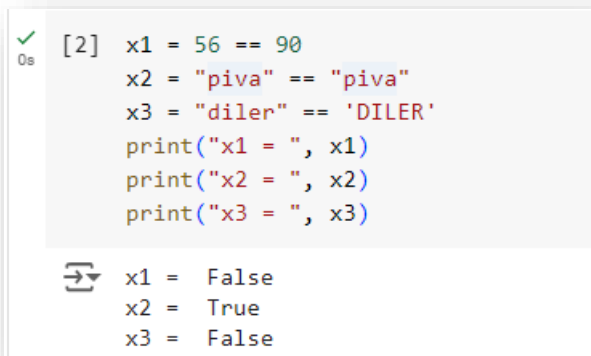
A screenshot of a Python REPL window. The top bar shows a green checkmark, a play button icon, and the text '45 == 45'. Below the bar, the output 'True' is displayed next to a copy icon.

O retorno desse comando é **True** porque 45 é igual a 45. Isso é óbvio.

Importante observar que: "=" é usado para atribuir um valor a uma variável, enquanto "==" é usado para comparar se dois valores são iguais ou não.

Sendo assim, eu consigo atribuir a uma variável (que será do tipo booleana – bool) o resultado de uma comparação. Veja os exemplos a seguir...

```
x1 = 56 == 90
x2 = "piva" == "piva"
x3 = "diler" == 'DILER'
print("x1 = ", x1)
print("x2 = ", x2)
print("x3 = ", x3)
```

A screenshot of a Python REPL window. The top bar shows a green checkmark, a play button icon, and the text '[2] x1 = 56 == 90'. Below the bar, the output of the code is displayed: 'x1 = False', 'x2 = True', and 'x3 = False'. A copy icon is visible next to the output.

Note que a última comparação (x3) retornou **False**. Isso acontece porque a linguagem Python diferencia maiúsculas de minúsculas, o que significa que para Python "a" e "A" são diferentes.

Diferente de (!=): Este operador é o oposto do operador que acabamos de ver. Ele retornará **True** se dois valores não forem iguais entre si e retornará **False** se forem iguais entre si.

```
x1 = 34 != 35
x2 = 765 != 765
x3 = 'PYTHON' != 'python'
print("x1 = ", x1)
print("x2 = ", x2)
print("x3 = ", x3)
```

```
✓ 0s ▶ x1 = 34 != 35
        x2 = 765 != 765
        x3 = 'PYTHON' != 'python'
        print("x1 = ", x1)
        print("x2 = ", x2)
        print("x3 = ", x3)

⇌ x1 = True
   x2 = False
   x3 = True
```

Menor que (<): "<" é usado para comparar se um valor é menor que o outro valor.

21 < 56

```
✓ 0s ▶ 21 < 56

⇌ True
```

```
num1 = 66
num2 = 99
print(num1 < num2)
```

```
✓ 0s ▶ num1 = 66
        num2 = 99
        print(num1 < num2)

⇌ True
```

Maior que (>): ">" é usado para comparar se um valor é maior que o outro valor.

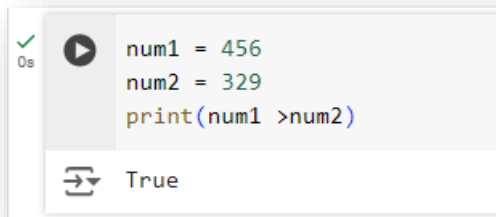
97 > 89

```
✓ 0s ▶ 97 > 89

⇌ True
```

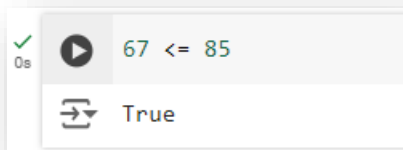
```
num1 = 456
num2 = 329
```

```
print(num1 > num2)
```



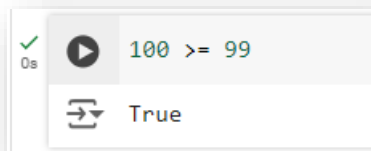
Menor ou Igual a (<=): "<=" retornará True se um número for menor ou igual ao outro número; caso contrário, retornará False.

```
67 <= 85
```



Maior ou Igual a (>=): ">=" retornará True se um número for maior ou igual ao outro número; caso contrário, retornará False.

```
100 >= 99
```



Terminamos com os Operadores de Comparação.

Operadores de Atribuição

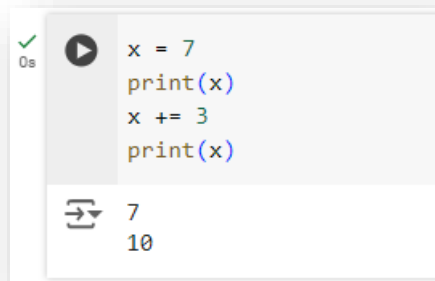
Os Operadores de Atribuição são usados para atribuir valores a uma variável. Já usamos "=" que é um Operador de Atribuição. Mas existem mais alguns Operadores de Atribuição que precisamos explorar. Eles são:

- `=` # Atribuir valor
- `+=` # Adicionar e atribuir
- `-=` # Subtrair e atribuir
- `*=` # Multiplicar e atribuir

- `/=` # Dividir e atribuir
- `//=` # Divisão de Piso e atribuir
- `%=` # Módulo e atribuir
- `**=` # Exponenciar e atribuir

Adicionar e atribuir (+=): Com este operador, podemos adicionar mais valor a uma variável existente que já possui algum valor nela. Não se preocupe se você ainda não conseguiu entender. Você entenderá uma vez que ver o exemplo.

```
x = 7
print(x)
x += 3
print(x)
```



No código acima, primeiro foi criada uma variável `x` e armazenado o valor inteiro 7 nela e, quando imprimimos `x`, ele me retornou 7. Na próxima linha, foi usado o operador `"+="` para adicionar 3 a `x`. `x` já tinha o valor 7 nele, quando adicionado 3, `x` se torna 10. Na verdade, `x += 3` é semelhante a `x = x + 3`. Simplificando, adicionando mais valor a uma variável já existente.

Da mesma forma, podemos fazer subtração, multiplicação e outras operações aritméticas que vimos anteriormente em uma variável já existente. Vamos passar por elas rapidamente. Nos exemplos a seguir, utilizamos o comentário em linha (`#`) para ilustrar o valor de saída em cada `print()`.

Subtrair e atribuir (-=):

```
x = 45
print(x)    # 45
x -= 35
print(x)    # 10
```

Multiplicar e atribuir (*=):

```
x = 4
print(x)  # 4
x *= 20
print(x)  # 80
```

Dividir e atribuir (/=):

```
x = 50
print(x)  # 50
x /= 20
print(x)  # 2.5
```

Divisão de Piso (ou quociente inteiro) e atribuir (//=):

```
x = 50
print(x)  # 50
x //= 20
print(x)  # 2
```

Módulo e atribuir (%=):

```
x = 50
print(x)  # 50
x %= 20
print(x)  # 10
```

Exponenciação e atribuir (|=):**

```
x = 20
print(x)  # 20
x **= 2
print(x)  # 400
```

Terminamos assim com os operadores de atribuição. Espero que agora você tenha uma ideia clara desses operadores. Todos os códigos acima são apenas exemplos. Você deve tentar coisas diferentes por conta própria usando os operadores.

Operadores Lógicos

Os Operadores Lógicos são sempre usados juntamente com os Operadores de Comparação. Operadores Lógicos são usados para combinar declarações

condicionais. Por exemplo, digamos que eu quero saber se um número é menor que 100, mas maior que 90; em tais casos, os Operadores Lógicos serão úteis.

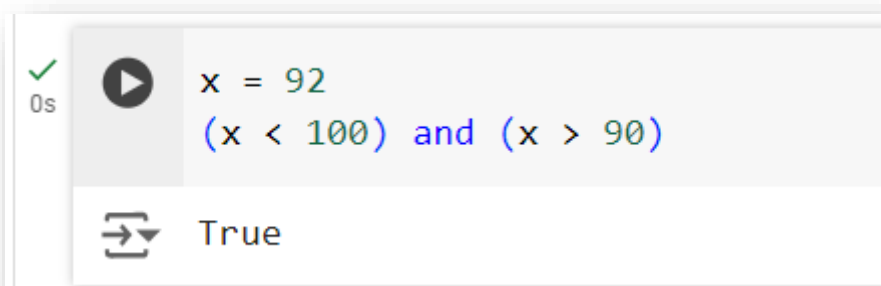
Não há problema se você ainda não entendeu. Uma vez que você veja o exemplo, você entenderá.

Os 3 principais Operadores Lógicos são:

- **and** # AND Lógico (ambas as condições devem ser True)
- **or** # OR Lógico (pelo menos uma condição deve ser True)
- **not** # NOT Lógico (inverte o resultado da condição)

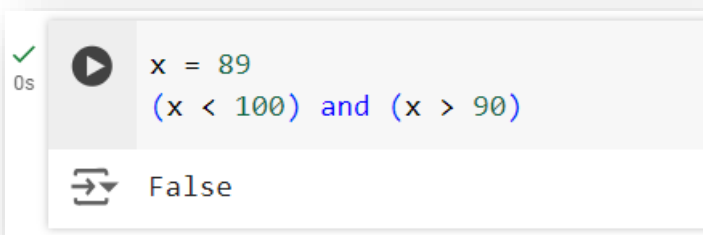
Operador and:

```
x = 92  
(x < 100) and (x > 90)
```



Como você pode ver no exemplo acima, ambas as condições `x < 100` e `x > 90` são **True**. Quando ambas as condições são **True**, a instrução **and** retornará **True**, mas se uma das condições for **False**, ela retornará **False**. Isso pode não fazer sentido para você agora, mas uma vez que você comece a aprender instruções de controle de fluxo como **if**, **if-else**, **for** loop e **while** loop, você entenderá o quão úteis esses operadores são.

```
x = 89  
(x < 100) and (x > 90)
```

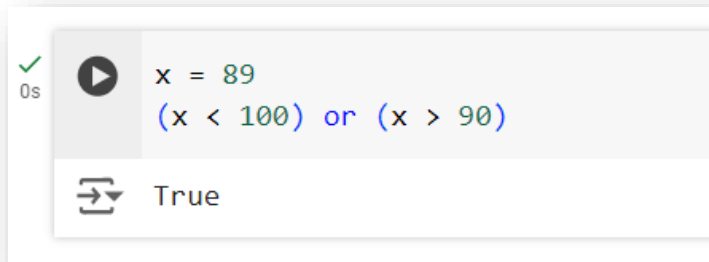


Agora uma condição no código acima é False, portanto, toda a instrução é False porque estamos usando o operador and.

Se você quiser retornar True mesmo que uma condição seja True, você precisa usar o operador or.

Operador or:

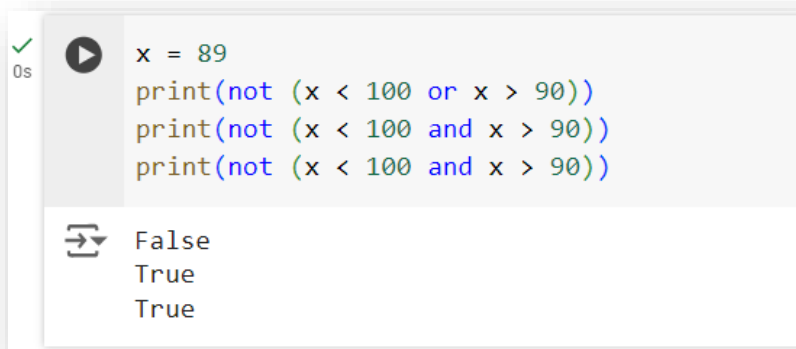
```
x = 89
(x < 100) or (x > 90)
```



Operador not:

O operador not é usado juntamente com and e or. O operador not retorna True quando é False e retorna False quando é True. Veja os códigos abaixo.

```
x = 89
print(not (x < 100 or x > 90))
print(not (x < 100 and x > 90))
print(not (x < 100 and x > 90))
```



Isso é tudo sobre os Operadores Lógicos. É hora dos Operadores de Identidade.

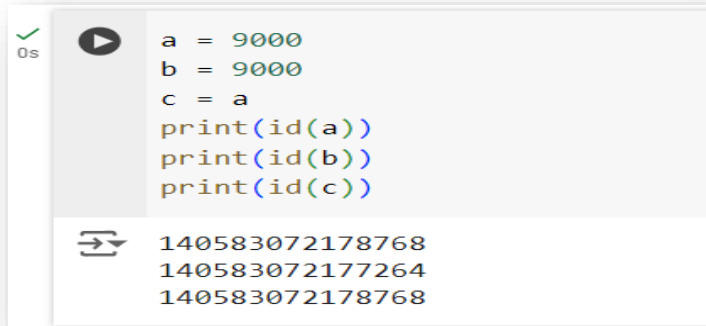
Operadores de Identidade

Os Operadores de Identidade são usados para saber se duas variáveis pertencem ao mesmo endereço de memória ou não. Isso pode ser muito bem explicado com um exemplo do que com palavras. Os Operadores de Identidade são:

- **is** # True se os operandos forem idênticos (mesmo objeto)
- **is not** # True se os operandos não forem idênticos

Vamos ver um exemplo...

```
a = 9000
b = 9000
c = a
print(id(a))
print(id(b))
print(id(c))
```



No código acima, a função `id()` é usada para imprimir o endereço de memória onde a variável está armazenada. A variável `a` tem 9000 nela, a variável `b` também tem 9000 e a variável `c` tem o valor 9000 que `a` tem. Quando imprimimos o endereço de memória, como você pode ver, as variáveis `a` e `c` têm o mesmo endereço de memória.

Vamos usar o Identidade para compará-las.

```
print(a is b)
print(a is c)
print(b is c)
```

```
✓ [13] print(a is b)
0s      print(a is c)
        print(b is c)

⇒ False
   True
   False
```

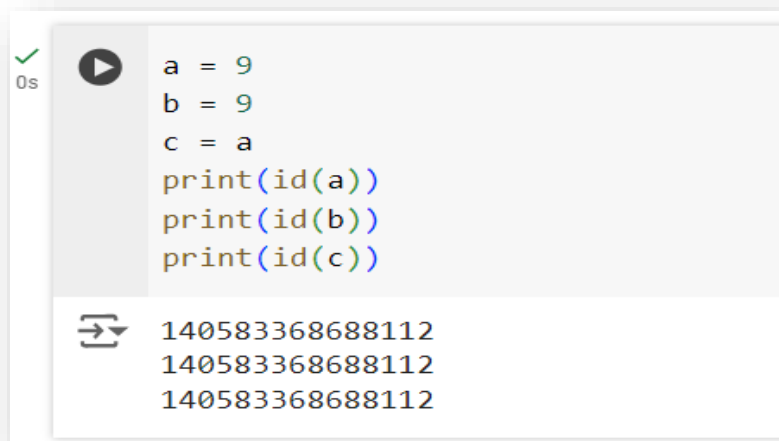
Embora a e b tenham o mesmo valor 9000, eles estão armazenados em diferentes endereços de memória, por isso obtemos **False** para "a is b". Mas "a is c" é **True** porque o 9000 dentro da variável c é o mesmo 9000 dentro da variável a e eles têm o mesmo endereço de memória. Se você quiser comparar os valores, pode usar o operador "==" , mas se quiser saber se duas variáveis compartilham o mesmo endereço de memória, você precisa usar o operador "is".

E inversamente:

```
a is not b # True
a is not c # False
```

ATENÇÃO: Se por acaso ao invés de 9000 você tivesse colocado 9 ou 90, atribuído às duas variáveis, poderia ter os endereços (ids) todos iguais. Isso poderia ocorrer, pois, no Python, quando você define variáveis com valores imutáveis como inteiros, o Python pode otimizar o uso da memória ao reutilizar o mesmo objeto em vez de criar um novo, principalmente para valores pequenos e comuns, como o número 9 ou 90, dado como exemplo acima.

Esse comportamento se dá porque o Python mantém uma cache de pequenos inteiros (normalmente no intervalo de -5 a 256), reutilizando o mesmo objeto para valores nesse intervalo. Isso economiza memória, já que múltiplas variáveis com o mesmo valor imutável podem apontar para o mesmo objeto na memória. Veja o exemplo a seguir:



```
0s a = 9
    b = 9
    c = a
    print(id(a))
    print(id(b))
    print(id(c))

140583368688112
140583368688112
140583368688112
```

Note que todos os ids são iguais. Portanto, muito cuidado ao utilizar esse recurso.

E finalmente chegamos ao último tópico desta seção: os operadores de associação.

Operadores de Associação

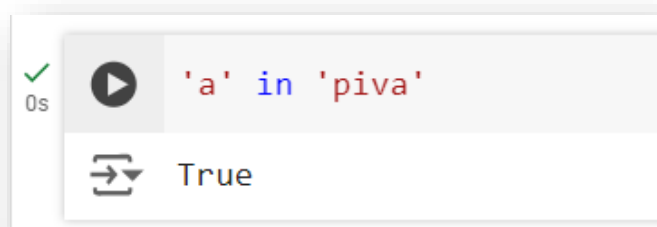
Este operador é usado quando queremos saber se um valor está presente em uma sequência. Uma sequência pode ser uma string, lista, tupla, conjunto ou dicionário. Até agora, cobrimos apenas strings; na próxima seção, você aprenderá sobre listas, tuplas, conjuntos e dicionários.

Os Operadores de Associação são:

- **in** # True se o valor for encontrado na sequência
- **not in** # True se o valor não for encontrado na sequência

Vamos ver um exemplo...

```
'a' in 'piva'
```

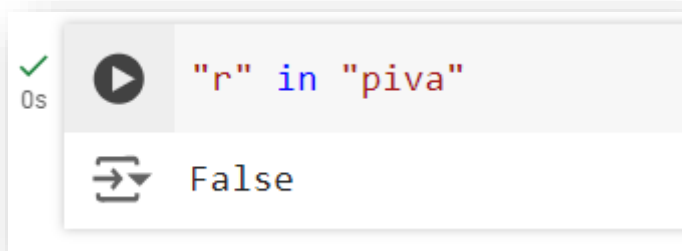


```
0s 'a' in 'piva'

True
```

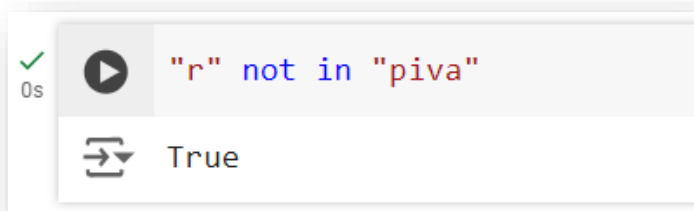
O código acima retornou True porque a letra "a" está presente na string "piva". Vamos tentar "r".

```
"r" in "piva"
```



Como "r" não está presente na sequência, diz False. Retornará True se usarmos o operador "not in".

```
"r" not in "piva"
```



Neste capítulo, exploramos como os operadores são as **ferramentas fundamentais** que permitem manipular e transformar dados em Python.

Desde realizar cálculos com operadores aritméticos até tomar decisões lógicas com operadores de comparação e associação, agora você tem uma sólida compreensão de como usar operadores para construir lógica e funcionalidade em seus programas.

Os operadores são a base para **tomar decisões, realizar operações e comparar informações**, mas isso é apenas o começo. À medida que seus programas se tornam mais complexos, você precisará de formas mais organizadas e eficientes de **armazenar e manipular conjuntos de dados**. É aí que entram as **estruturas de dados integradas** de Python.

No próximo capítulo, vamos explorar as principais **estruturas de dados** que Python oferece — listas, tuplas, conjuntos e dicionários. Essas estruturas são essenciais para trabalhar com **grandes volumes de informações** de maneira organizada e eficiente. Você aprenderá como armazenar, acessar e manipular coleções de dados com facilidade, elevando suas habilidades de programação a um novo nível.

Agora que você domina os operadores, está pronto para lidar com dados em larga escala. Prepare-se para dar o próximo passo na sua jornada de Python e mergulhar

nas poderosas estruturas de dados que farão seu código ainda mais eficiente e dinâmico.



PARA SABER MAIS ...

Python Documentation: Explicação do significado dos elementos das expressões em Python. <https://docs.python.org/3/reference/expressions.html>

Livros e cursos: Acesse <https://pypro.com.br> para Livros e Cursos sobre a linguagem Python. O curso “*Python Profissional: do Zero ao Mercado de Trabalho*” apresenta em detalhes o significado e utilização das expressões em Python.

5

Estruturas de Dados Integradas

Até agora, você aprendeu como armazenar dados em variáveis e manipular essas informações usando operadores.

Mas o que acontece quando você precisa trabalhar com **grandes volumes de dados** ou organizar múltiplos valores de maneira eficiente?

É aí que entram as **estruturas de dados integradas** de Python.

As estruturas de dados são como **caixas organizadoras**, onde você pode armazenar e acessar várias informações de maneira estruturada.

Em vez de criar múltiplas variáveis para guardar diferentes valores, você pode armazenar tudo em um único contêiner e manipular esses dados de forma muito mais **flexível e eficiente**.

Python nos oferece **quatro estruturas de dados principais** que são integradas à linguagem. Elas são projetadas para lidar com diferentes tipos de situações e exigências ao armazenar e organizar dados:

- **Listas:** Usadas para armazenar uma sequência ordenada de itens, como uma lista de compras ou uma coleção de números. As listas são flexíveis, permitindo que você adicione, remova e modifique elementos facilmente.
- **Tuplas:** Semelhantes às listas, mas imutáveis, ou seja, seus elementos não podem ser alterados depois de criados. Tuplas são ideais para armazenar dados que não devem ser modificados ao longo do tempo.
- **Conjuntos:** Utilizados para armazenar coleções de itens **únicos**, sem duplicatas, e permitem operações como interseção e união, sendo muito úteis quando você precisa lidar com elementos distintos e realizar comparações rápidas.
- **Dicionários:** Estruturas que armazenam **pares chave-valor**, permitindo uma forma rápida de acessar informações com base em uma chave. Pense

neles como um índice de um livro, onde você pode encontrar facilmente o valor associado a uma chave específica.

Cada uma dessas estruturas tem suas próprias características e propósitos, e aprender a usá-las é essencial para que você possa trabalhar com dados de maneira eficiente e organizada em seus programas Python.

Ao longo deste capítulo, vamos explorar essas quatro estruturas de dados detalhadamente, mostrando como e quando usá-las em diferentes situações.

Prepare-se para ver como essas ferramentas podem transformar a forma como você armazena e manipula informações, tornando seus programas mais **dinâmicos**, **eficientes**, e prontos para lidar com dados de qualquer tamanho.

Lista e seus Métodos

Lista é uma estrutura de dados integrada que pode ser usada para armazenar vários elementos de diferentes tipos de dados em uma variável.

A lista é mutável, o que significa que podemos modificar, inserir ou remover os elementos presentes na lista usando vários métodos.

Criamos listas usando colchetes ([]). Vamos criar algumas listas.

```
11 = [1, 2, 3, 4, 5]
12 = ['a', 'b', 'c', 'd', 'e']
13 = [1, 'b', 'apple', 4.5, True, 99]
```

Acima foram criadas listas chamadas **l1**, **l2** e **l3**. Em **l1**, foram armazenados 5 elementos ou valores apenas de inteiros; em **l2**, foram armazenados 5 elementos de string; e em **l3**, foram armazenados 6 elementos de diferentes tipos de dados.

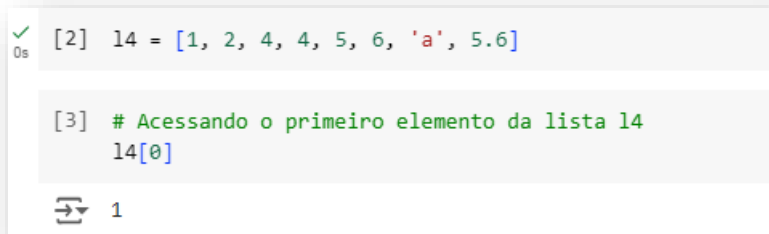
Você também pode armazenar o mesmo valor duas vezes na lista porque a lista permite valores duplicados nela.

```
14 = [1, 2, 4, 4, 5, 6, 'a', 5.6]
```

Cada elemento na lista tem um valor de índice; podemos acessá-los usando seu valor de índice.

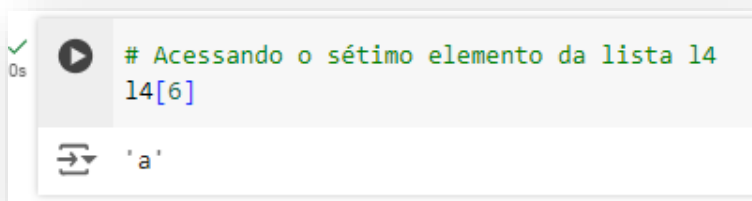
Por exemplo, o valor de índice do primeiro elemento da lista é **0** (zero), o valor de índice do segundo elemento é 1 (um), o terceiro elemento é 2 (dois) e assim por diante.

Lista e seus respectivos valores de índice: Se quiser acessar o primeiro elemento de uma lista, você pode simplesmente escrever: `l4[0]`



A screenshot of a Jupyter Notebook cell. The first line of code is `[2] l4 = [1, 2, 4, 4, 5, 6, 'a', 5.6]`. The second line is a comment `[3] # Acessando o primeiro elemento da lista l4` followed by the code `l4[0]`. Below the code, the output is shown as `1`.

Vamos acessar o 7º elemento da lista.



A screenshot of a Jupyter Notebook cell. The code is `# Acessando o sétimo elemento da lista l4` followed by `l4[6]`. Below the code, the output is shown as `'a'`.

Agora você sabe o que é uma lista e como criá-las. Vamos realizar algumas operações nelas usando seus **métodos**.

Métodos são como "superpoderes" que os objetos têm para fazer coisas específicas.

Pensando em listas no Python como caixas que guardam várias coisas dentro... Os métodos são comandos especiais que fazem essa caixa funcionar de diferentes maneiras, organizando, buscando, ou até mesmo removendo coisas de dentro dela.

Assim, os métodos são uma maneira de "falar" para a lista (e qualquer objeto / classe que tenha métodos) o que você quer que ela faça, como se fossem comandos especiais que já vêm programados para ajudar a organizar e manipular os itens dentro dela. É como se cada lista viesse com um manual de instruções para fazer várias coisas legais com os seus elementos.

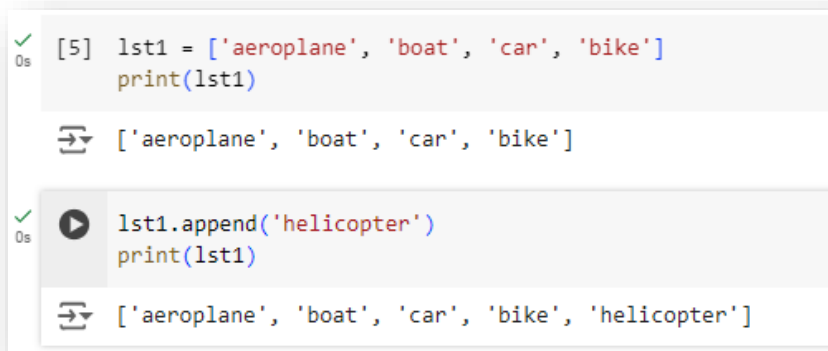
Métodos das Listas (ou do objeto 'list'): As listas em Python são extremamente poderosas e flexíveis, e para tirar o máximo proveito delas, Python oferece uma série de **métodos** que facilitam a manipulação de dados. Esses métodos permitem

adicionar, remover, ordenar, e realizar várias operações nas listas de maneira eficiente. Aqui estão os métodos mais importantes que você precisa conhecer:

- **append():** Adiciona um elemento ao final da lista.
- **pop():** Remove e retorna o último elemento da lista.
- **remove():** Remove a primeira ocorrência de um valor específico na lista.
- **insert():** Insere um elemento em uma posição específica.
- **sort():** Ordena os elementos da lista.
- **count():** Retorna o número de vezes que um valor específico aparece na lista.
- **index():** Retorna o índice da primeira ocorrência de um valor na lista.
- **copy():** Faz uma cópia superficial da lista.
- **clear():** Remove todos os elementos da lista.

Vamos agora explorar cada um desses métodos em detalhes, com exemplos para mostrar como funcionam na prática.

append() O método `append()` é usado para adicionar um novo elemento a uma lista já existente. Abaixo está o exemplo de código.



```
[5] lst1 = ['aeroplane', 'boat', 'car', 'bike']
print(lst1)

['aeroplane', 'boat', 'car', 'bike']

lst1.append('helicopter')
print(lst1)

['aeroplane', 'boat', 'car', 'bike', 'helicopter']
```

No código acima, adicionamos um elemento "helicopter" à lista `lst1` usando o método **append()**.

Agora você sabe como adicionar um elemento a uma lista. Vamos ver como remover um elemento da lista.

pop() Usando o método `pop()`, podemos remover um elemento da lista.

O método **pop()** remove o último elemento da lista (mais a esquerda).

Mas se você quiser remover algum outro elemento, você precisa especificar o valor de índice do elemento que deseja remover dentro do **pop()**.

Por exemplo, digamos que eu quero remover 'car' da lista e o valor de índice de 'car' é 2. Então:

```
[ 'aeroplane', 'boat', 'car', 'bike' ]  
  
lst1.pop(2)  
print(lst1)  
  
[ 'aeroplane', 'boat', 'bike' ]
```

The screenshot shows a list `['aeroplane', 'boat', 'car', 'bike']`. A code block executes `lst1.pop(2)` and `print(lst1)`. The output shows the list `['aeroplane', 'boat', 'bike']`, with 'car' removed.

Agora 'car' foi removido da lista. Você pode remover qualquer elemento que quiser da lista especificando seu valor de índice dentro do método **pop()**.

remove() Caso você não saiba o valor de índice de um elemento que deseja remover, mas saiba o valor do elemento, então você pode remover o elemento usando o método **remove()**. Veja um exemplo:

```
[10] cars = ['bmw', 'audi', 'toyota', 'honda']  
print(cars)  
  
[ 'bmw', 'audi', 'toyota', 'honda' ]  
  
cars.remove('toyota')  
print(cars)  
  
[ 'bmw', 'audi', 'honda' ]
```

The screenshot shows a list `cars = ['bmw', 'audi', 'toyota', 'honda']`. A code block executes `cars.remove('toyota')` and `print(cars)`. The output shows the list `['bmw', 'audi', 'honda']`, with 'toyota' removed.

insert() Quando você usa o método **append()**, pode apenas inserir um novo elemento na última posição do índice da lista, mas e se você quiser inserir um elemento na 2ª posição do índice ou na 4ª posição do índice ou em qualquer outra posição do índice?

É aí que o método **insert()** é útil. Usando o método **insert()**, podemos inserir qualquer valor em qualquer posição do índice que quisermos. Abaixo está o exemplo de código.

```
✓ [12] cars = ['bmw', 'audi', 'toyota', 'honda', 'mini', 'ford']  
0s print(cars)  
⇒ ['bmw', 'audi', 'toyota', 'honda', 'mini', 'ford']  
  
✓ [13] cars.insert(2, 'tesla')  
0s print(cars)  
⇒ ['bmw', 'audi', 'tesla', 'toyota', 'honda', 'mini', 'ford']
```

Como você pode ver, usando o método **insert()**, note que foi inserido **'tesla'** na 2ª posição do índice da lista. Note que todos os demais itens à direita, foram deslocados uma posição.

Agora você sabe como adicionar e remover elementos da lista em qualquer posição do índice. Mas você ainda não aprendeu como modificar ou alterar um elemento na lista. Para alterar um elemento na lista, não precisamos usar nenhum método. Abaixo está o código:

```
✓ [14] print(cars)  
0s ⇒ ['bmw', 'audi', 'tesla', 'toyota', 'honda', 'mini', 'ford']  
  
✓ [15] cars[3] = 'kia'  
0s print(cars)  
⇒ ['bmw', 'audi', 'tesla', 'kia', 'honda', 'mini', 'ford']
```

Anteriormente, na 3ª posição do índice, tínhamos **'toyota'**, mas agora o elemento foi mudado para **'kia'**.

sort() O método **sort()** é usado para classificar a lista (colocar em ordem, no caso da lista **cars**, em ordem alfabética).

```
✓ [16] print(cars)
0s
↳ ['bmw', 'audi', 'tesla', 'kia', 'honda', 'mini', 'ford']

✓ [17] cars.sort()
0s
print(cars)
↳ ['audi', 'bmw', 'ford', 'honda', 'kia', 'mini', 'tesla']
```

O código acima classificou os elementos presentes na lista de carros em ordem alfabética.

Vamos ver como o método `sort()` funciona para números.

```
✓ [18] numbers = [56, 34, 87, 12, 43, 23, 35, 90]
0s
numbers.sort()
print(numbers)
↳ [12, 23, 34, 35, 43, 56, 87, 90]

✓ [19] numbers.sort(reverse=True)
0s
print(numbers)
↳ [90, 87, 56, 43, 35, 34, 23, 12]
```

Por padrão, o método `sort()` classifica o elemento em ordem crescente. Mas se você quiser classificar sua lista em ordem decrescente, pode fazer isso passando um parâmetro chamado `'reverse'` e configurando seu valor para `True`.

count() O método `count()` é usado para contar quantas vezes um elemento está presente na lista.

```
✓ [21] ages = [12, 20, 18, 12, 23, 21, 20, 12, 19, 12]
0s      print(ages)

⇌ [12, 20, 18, 12, 23, 21, 20, 12, 19, 12]

✓ [22] ages.count(12)
0s

⇌ 4

✓ [23] ages.count(20)
0s

⇌ 2
```

O elemento '12' está presente 4 vezes na lista, já o elemento '20' está presente duas vezes na lista ages.

index() O método **index()** retornará o valor do índice de um elemento.

```
✓ [24] print(cars)
0s

⇌ ['audi', 'bmw', 'ford', 'honda', 'kia', 'mini', 'tesla']

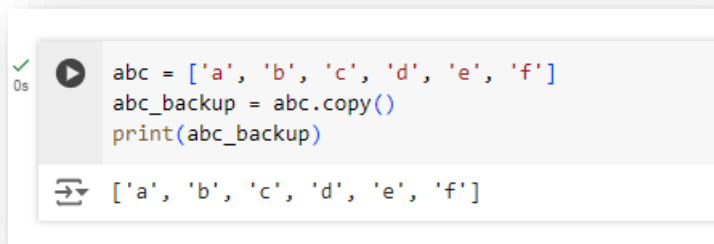
✓ [25] cars.index('honda')
0s

⇌ 3
```

O valor do índice do 'honda' é 3 e o valor do índice do 'tesla' é 6.

Atenção: se tiver mais de um elemento com o mesmo valor, o método **index()** retorna o primeiro que ele encontrar.

copy() Usando o método **copy()**, podemos copiar uma lista em uma variável para outra variável.

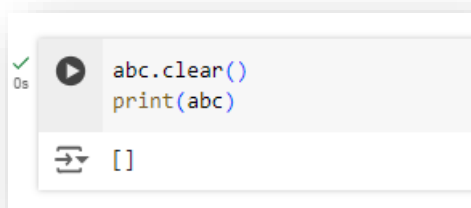


```
✓ 0s ▶ abc = ['a', 'b', 'c', 'd', 'e', 'f']
      abc_backup = abc.copy()
      print(abc_backup)

      [ 'a', 'b', 'c', 'd', 'e', 'f' ]
```

Antes de realizar qualquer tipo de modificação na sua lista, você pode fazer uma cópia dela usando o método **copy()** porque se cometer algum erro ao modificar a lista, você pode usar a cópia mais tarde.

clear() Se você quiser remover todos os elementos da sua lista e deixá-la vazia, pode usar o método **clear()**.



```
✓ 0s ▶ abc.clear()
      print(abc)

      []
```

Agora a lista **abc** está vazia.

É isso. Agora você sabe como criar uma lista, adicionar elementos a ela, remover elementos dela, classificar os elementos nela, fazer uma cópia dela e mais.

Vamos passar para a próxima estrutura de dados integrada: tuplas.

Tupla e seus Métodos

Uma Tupla é muito semelhante a uma lista. A única diferença é que, como uma lista, não podemos mudar ou modificar uma tupla usando vários métodos.

Criamos listas usando colchetes (**[]**) e, de maneira semelhante, para criar tuplas, usamos parênteses (**()**).

```
✓ 0s ▶ legumes = ('eggplant', 'onion', 'sweet potato', 'carrot', 'beetroot')  
      print(legumes)  
⇌ ('eggplant', 'onion', 'sweet potato', 'carrot', 'beetroot')
```

Para acessar o elemento dentro da tupla, podemos simplesmente usar seus índices. Vamos acessar a batata doce (Sweet potato).

```
✓ 0s ▶ print(legumes[2])  
⇌ sweet potato
```

Como mencionei antes, não podemos adicionar, atualizar ou remover um elemento de uma tupla, mas há uma maneira de fazer isso. Se você realmente quiser adicionar, atualizar ou remover um elemento da sua tupla, primeiro você precisa converter sua tupla em uma lista; depois de realizar as modificações, você pode converter a lista de volta para uma tupla. Abaixo está um exemplo de código de como fazer isso.

```
✓ 0s [31] print(legumes)  
⇌ ('eggplant', 'onion', 'sweet potato', 'carrot', 'beetroot')  
✓ 0s [32] list_legumes = list(legumes)  
      print(list_legumes)  
⇌ ['eggplant', 'onion', 'sweet potato', 'carrot', 'beetroot']  
✓ 0s [33] list_legumes.append('potato')  
      print(list_legumes)  
⇌ ['eggplant', 'onion', 'sweet potato', 'carrot', 'beetroot', 'potato']  
✓ 0s ▶ legumes = tuple(list_legumes)  
      print(legumes)  
⇌ ('eggplant', 'onion', 'sweet potato', 'carrot', 'beetroot', 'potato')
```

Primeiro, imprimimos a tupla para que você saiba quais são todos os elementos que já estão presentes na tupla.

Em seguida, convertemos a tupla em uma lista usando o método `list()`. Em seguida imprimimos essa nova lista... Note que os resultados das impressões de uma tupla e de uma lista são bem parecidos, divergindo apenas os seus limitadores (tupla = parênteses e listas = conchetes).

Depois, usando o método `append()`, adicionamos um elemento ("potato") à lista. E então mostramos novamente a nova lista.

Após adicionar o valor, converti novamente a lista em uma tupla usando o método `tuple()`. Finalmente, imprimimos a tupla alterada e o novo valor foi adicionado a ela.

Não podemos realizar nenhuma modificação, mas podemos verificar o número de vezes que um elemento está presente (`count()`) e o valor do índice de um elemento (`index()`), como fizemos para a lista.

```
0s ✓ letters = ('a', 'a', 'b', 'd', 'c', 'd', 'd', 'd', 'b', 'a')
letters.count('a')
⇒ 3
```

```
0s ✓ [37] print(letters)
⇒ ('a', 'a', 'b', 'd', 'c', 'd', 'd', 'd', 'b', 'a')

0s ✓ [38] letters.index('c')
⇒ 4

0s ✓ letters.index('b')
⇒ 2
```

O valor do índice do elemento 'c' é 4. E o valor do índice do elemento 'b' é 2, pois ele apresenta o índice da primeira ocorrência dentro da tupla.

Na tupla, há um conceito chamado desempacotamento que nos permite armazenar cada elemento da tupla em variáveis separadas. Abaixo está o código.

```
✓ 0s ▶ fruits = ('apple', 'banana', 'orange')
a, b, c = fruits
print(a)
print(b)
print(c)
```

```
↔ apple
banana
orange
```

Isso é tudo. Não há muito sobre tupla. Vamos passar para o próximo tópico: conjuntos

Conjunto e seus Métodos

Uma estrutura de dados de conjunto é muito diferente tanto da lista quanto da tupla. Como sabemos, listas e tuplas permitem valores duplicados, mas o conjunto não permite valores duplicados.

Para uma lista, armazenamos os elementos dentro de colchetes (`[]`); para uma tupla, armazenamos os elementos dentro de parênteses (`()`); de maneira semelhante, para um conjunto, armazenamos os elementos dentro de **chaves** (`{}`).

Vamos criar dois conjuntos: **set1** e **set2**.

```
✓ 0s ▶ set1 = {1, 2, 3, 4, 5}
set2 = {'a', 'b', 'c', 'd', 'e'}
print(set1)
print(set2)
```

```
↔ {1, 2, 3, 4, 5}
{'b', 'e', 'c', 'a', 'd'}
```

Diferentemente de uma lista ou tupla, um conjunto não tem valores de posição de índice para seu elemento. Além disso, o conjunto é desordenado, o que significa que ele organiza o elemento em qualquer ordem que quiser; por exemplo, veja "**set2**" no código acima.

Podemos apenas realizar operações de adição e remoção em conjunto. Vamos tentar.

add() Para adicionar um novo valor ao nosso conjunto, podemos usar um método chamado **add()**.

```
✓ 0s ▶ set2.add('f')
      print(set2)

⇌ {'b', 'e', 'c', 'a', 'd', 'f'}
```

remove() De maneira semelhante, para remover um elemento do nosso conjunto, podemos usar um método chamado **remove()**.

```
✓ 0s ▶ set2.remove('d')
      print(set2)

⇌ {'b', 'e', 'c', 'a', 'f'}
```

update() Há também outro método de conjunto chamado **update()**. Usando o método `update()`, podemos unir dois conjuntos. Vamos ver como funciona.

```
✓ 0s ▶ # Conjunto 1 de utensílios domésticos
      utensils_set1 = {"spoon", "fork", "knife", "plate", "bowl"}

      # Conjunto 2 de utensílios domésticos
      utensils_set2 = {"cup", "mug", "pan", "pot", "whisk"}

      # Unindo os conjuntos em set1
      utensils_set1.update(utensils_set2)
      print(utensils_set1)

⇌ {'pot', 'fork', 'spoon', 'mug', 'whisk', 'cup', 'plate', 'knife', 'pan', 'bowl'}
```

No código acima, atualizamos `set1` com os elementos em `set2` usando o método `update()`.

A relação dos métodos do tipo de dado conjunto que não foram apresentados anteriormente são apresentados na tabela a seguir:

Função	Sintaxe	Descrição
clear()	Set.clear()	Remove todos os itens do conjunto <i>Set</i> .
difference()	Set.difference(*outros)	Retorna um novo conjunto com itens no conjunto <i>Set</i> que não estão nos <i>outros</i> conjuntos.
discard()	Set.discard(item)	Remove um item do conjunto <i>Set</i> se estiver presente.
intersection()	Set.intersection(*outros)	Retorna um novo conjunto com itens comuns ao conjunto <i>Set</i> e todos os <i>outros</i> conjuntos.
isdisjoint()	Set.isdisjoint(outro)	Retorna True se o conjunto <i>Set</i> não tiver itens em comum com <i>outro</i> conjunto. Conjuntos são disjuntos se e somente se sua interseção for o conjunto vazio.
issubset()	Set.issubset(outro)	Retorna True se todos os itens do conjunto <i>Set</i> estiverem em <i>outro</i> conjunto.
superset()	Set.isuperset(outro)	Retorna True se cada elemento em <i>outro</i> conjunto estiver no conjunto <i>Set</i> .
pop()	Set.pop()	Remove e retorna um item arbitrário do conjunto <i>Set</i> . Ele gera <i>KeyError</i> se o conjunto for vazio.
symmetric_difference()	Set.symetric_difference(outro)	Retorna um novo conjunto com itens no Conjunto <i>Set</i> ou no conjunto <i>outro</i> , mas não em ambos. ((ou exclusivo))
union()	Set.union(*outros)	Retorna um novo conjunto com itens do conjunto <i>Set</i> e todos os <i>outros</i> conjuntos.

Isso é suficiente sobre a estrutura de dados integrada de conjunto.

Vamos começar com a última estrutura de dados integrada: os dicionários

Dicionário e seus Métodos

Um dicionário em Python é uma estrutura de dados que armazena elementos em pares de **chave-valor**.

Diferente de listas e tuplas, os elementos de um dicionário não são acessados por índices numéricos.

Em vez disso, cada valor no dicionário é associado a uma chave única, que serve como uma etiqueta para acessar o valor correspondente.

Para consultar ou modificar um elemento em um dicionário, utilizamos sua chave. Vamos ver um exemplo para entender melhor.

```
✓ 0s # Exemplo de dicionário
student_info = {
    "name": "Alice",
    "age": 20,
    "course": "Data Science",
    "grade": "A"
}
print(student_info)

{'name': 'Alice', 'age': 20, 'course': 'Data Science', 'grade': 'A'}
```

Neste exemplo, temos um dicionário chamado **student_info** que armazena informações sobre uma estudante. O dicionário é composto por pares de **chave-valor**, onde:

- **"name"** é a chave, e **"Alice"** é o valor.
- **"age"** é a chave, e **20** é o valor.
- **"course"** é a chave, e **"Data Science"** é o valor.
- **"grade"** é a chave, e **"A"** é o valor.

Para acessar um valor no dicionário, usamos a chave correspondente. Por exemplo:

```
✓ 0s # Acessando os valores usando as chaves
print(student_info["name"])
print(student_info["age"])

Alice
20
```

Quando usamos **student_info["name"]**, estamos pedindo ao Python para nos dar o valor associado à chave **"name"**, que é **"Alice"**.

Da mesma forma, **student_info["age"]** retorna o valor **20**, que é a idade da estudante.

Dessa forma, os dicionários são muito úteis para armazenar e acessar dados usando chaves, tornando a busca por informações mais rápida e intuitiva.

get() Há também um método chamado **get()** para fazer a mesma coisa que o código acima faz.

```
✓ 0s # Acessando os valores usando o método get()
print(student_info.get("name"))
print(student_info.get("age"))
```

⇒ Alice
20

O método `.get()` funciona de forma similar ao uso das chaves diretas (`student_info["name"]`), mas com uma diferença importante: se a chave não existir no dicionário, o método `.get()` não gera um erro. Em vez disso, ele retorna `None` (ou um valor padrão, se especificado).

Por exemplo, se tentarmos acessar uma chave que não existe usando o `.get()`:

```
✓ 0s # Tentando acessar uma chave inexistente
print(student_info.get("address"))
```

⇒ None

Isso é útil para evitar erros em situações onde não temos certeza se uma chave está presente no dicionário. Podemos também definir um valor padrão para ser retornado caso a chave não exista:

```
✓ 0s # Usando um valor padrão quando a chave não existe
print(student_info.get("address", "Address not available"))
```

⇒ Address not available

keys() Se você quiser acessar todas as chaves no dicionário, pode usar o método `keys()`.

```
✓ 0s print(student_info.keys())
```

⇒ dict_keys(['name', 'age', 'course', 'grade'])

values() De maneira semelhante, se você quiser acessar todos os valores no dicionário, pode usar o método **values()**.

```
✓ 0s ▶ print(student_info.values())  
⇌ dict_values(['Alice', 20, 'Data Science', 'A'])
```

items() Há também um método chamado **items()**. Se você quiser acessar tanto as chaves quanto os valores do dicionário.

```
✓ 0s ▶ print(student_info.items())  
⇌ dict_items([('name', 'Alice'), ('age', 20), ('course', 'Data Science'), ('grade', 'A')])
```

Podemos facilmente mudar um valor já existente no dicionário usando a **chave** do valor que você deseja alterar.

O código abaixo mudará o valor "Alice" para "Silva" na chave "name".

```
✓ 0s [11] print(student_info)  
⇌ {'name': 'Alice', 'age': 20, 'course': 'Data Science', 'grade': 'A'}  
  
✓ 0s ▶ student_info["name"] = "Silva"  
print(student_info)  
⇌ {'name': 'Silva', 'age': 20, 'course': 'Data Science', 'grade': 'A'}
```

Dessa forma, você pode alterar qualquer valor que quiser.

pop() Certamente você se lembra do método **pop()** que usamos quando estávamos aprendendo sobre listas. Podemos usar o mesmo método **pop()** no dicionário também para remover um par chave-valor dele. Vamos ver como é feito.

```
✓ [13] print(student_info)
0s
⇌ {'name': 'Silva', 'age': 20, 'course': 'Data Science', 'grade': 'A'}

✓ [14] student_info.pop("age")
0s
print(student_info)
⇌ {'name': 'Silva', 'course': 'Data Science', 'grade': 'A'}
```

clear() Se você quiser remover todos os pares chave-valor do dicionário, pode simplesmente usar o método **clear()**.

```
✓ [15] print(student_info)
0s
⇌ {'name': 'Silva', 'course': 'Data Science', 'grade': 'A'}

✓ [16] student_info.clear()
0s
print(student_info)
⇌ {}
```

A tabela a seguir apresenta os outros quatro métodos para o tipo de dado dicionário não citados ainda.

Função	Sintaxe	Descrição
fromkeys()	Dic.fromkeys(seq[,valor])	Cria um novo dicionário a partir da sequência dos elementos com um valor informado
popitem()	Dic.popitem()	Remove e retorna um par de tuplas arbitrário (chave, valor) do dicionário. Se o dicionário estiver vazio, chamar <code>popitem()</code> resultará em <code>KeyError</code> .
setdefault()	Dic.setdefault(chave[,padrão])	Retorna um valor para a chave presente no dicionário. Se a chave não estiver presente, insere a chave no dicionário com um valor padrão e retorna o valor padrão. Se a chave estiver presente, o padrão é <code>None</code> , para que esse método nunca gere um <code>KeyError</code> .
update()	Dic.update([outro])	Atualiza o dicionário com os pares chave:valor de outro objeto de dicionário e retorna <code>None</code> .

Isso é suficiente sobre dicionário. Agora você sabe o que é um dicionário e como adicionar, alterar e remover elementos dele.

Vamos agora complementar o assunto que já iniciamos, mas não falamos sobre como realizar manipulações, inclusive com métodos específicos: as strings!

Manipulação de Strings

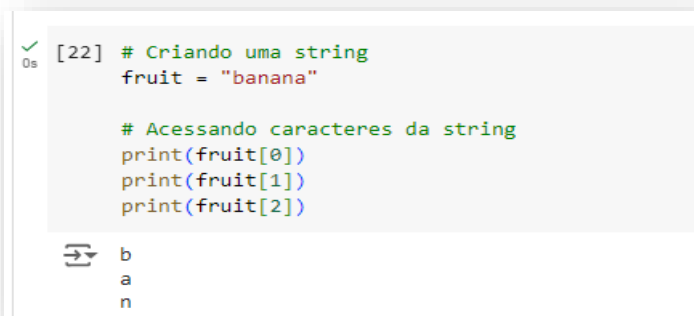
Até agora, aprendemos a trabalhar com as quatro principais estruturas de dados integradas em Python.

Agora, vamos falar novamente sobre **strings**. Você já viu, mas não custa enfatizar: diferente de outros tipos de dados, como **int**, **float** e **boolean**, uma **string** é uma **sequência de caracteres** — ou seja, é uma sequência de letras, números e símbolos.

Como as **strings** são sequências, elas compartilham algumas características com listas e tuplas, como a possibilidade de acessar cada caractere usando índices. Dizemos que essas estruturas são iteráveis.

Vamos explorar alguns dos principais métodos que nos ajudam a manipular **strings** de forma eficiente.

Acessando Caracteres em uma String: Cada caractere em uma string possui uma posição (ou índice), que começa em zero. Vamos usar o exemplo abaixo para entender melhor:



```
✓ [22] # Criando uma string
fruit = "banana"

# Acessando caracteres da string
print(fruit[0])
print(fruit[1])
print(fruit[2])
```

b
a
n

Aqui, o primeiro caractere da string "banana" tem índice 0, o segundo caractere tem índice 1, e assim por diante.

Esse conceito é importante para entender como manipulamos strings em Python.

Principais Métodos de Manipulação de Strings: Agora que sabemos como acessar os caracteres de uma string, vamos explorar alguns métodos úteis para trabalhar com elas.

upper() — Converte todos os caracteres da string para letras maiúsculas.

```
✓ [23] phrase = "I love programming"
0s      print(phrase.upper())

⇒ I LOVE PROGRAMMING
```

lower() — Converte todos os caracteres da string para letras minúsculas.

```
✓ 0s phrase = "I LOVE PROGRAMMING"
      print(phrase.lower())

⇒ i love programming
```

replace() — Substitui uma parte da string por outra.

```
✓ 0s message = "Hello World!"
      new_message = message.replace("World", "Python")
      print(new_message)

⇒ Hello Python!
```

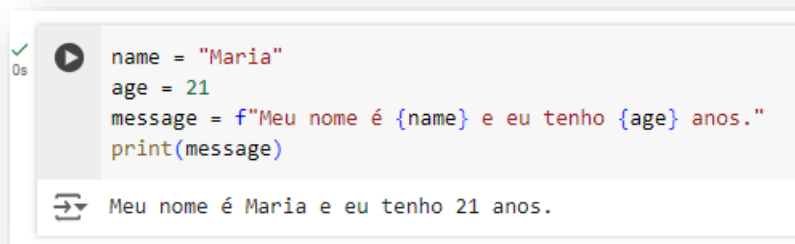
split() — Divide uma string em uma lista de palavras ou partes, com base em um separador.

```
✓ 0s text = "Python is Fun"
      words = text.split(" ")
      print(words)

⇒ ['Python', 'is', 'Fun']
```

Se não especificarmos um separador, o `split()` usa, por padrão, os espaços em branco para dividir a string.

Strings Formatadas: Uma maneira poderosa e simples de criar strings que incluam variáveis é usando **strings formatadas**. Elas nos permitem inserir valores diretamente dentro de uma string de forma clara e organizada.



```
✓ 0s ▶ name = "Maria"
      age = 21
      message = f"Meu nome é {name} e eu tenho {age} anos."
      print(message)

⇌ Meu nome é Maria e eu tenho 21 anos.
```

Ao adicionar um **f** antes das aspas da string, podemos incluir variáveis diretamente dentro das chaves {}, tornando o código mais legível e fácil de manter.

Com esses métodos, você agora tem as ferramentas necessárias para manipular e transformar tanto listas, tuplas, conjuntos, dicionários quanto strings de maneira eficiente em Python.

Ao dominar essas estruturas de dados e seus métodos, você deu um grande passo em direção à construção de programas mais dinâmicos e poderosos.

Agora que entendemos como trabalhar com dados, estamos prontos para um novo desafio: **controlar o fluxo dos nossos programas**.

No próximo capítulo, vamos aprender como usar **comandos condicionais** para tomar decisões e direcionar o caminho que nossos códigos seguirão, com base nas condições que definirmos.



PARA SABER MAIS ...

Python Documentation: Explicação dos diversos tipos de dados em Python.
<https://docs.python.org/3/reference/datamodel.html#objects-values-and-types>

Livros e cursos: Acesse <https://pypro.com.br> para Livros e Cursos sobre a linguagem Python. O curso “*Python Profissional: do Zero ao Mercado de Trabalho*” apresenta o que são e como utilizar as Estruturas de Dados Integradas, com exemplos reais e exercícios para fixação dos conteúdos.

6

Controle de Fluxo: Comandos Condicionais

Quando escrevemos programas, nem sempre queremos que as instruções sejam executadas de forma linear, uma após a outra.

Muitas vezes, precisamos que o programa tome decisões e siga diferentes **caminhos** dependendo de certas condições.

É aqui que entra o **controle de fluxo**, um dos aspectos mais fundamentais da programação. Ele permite que você determine **quando** e **como** partes do seu código serão executadas.

O **controle de fluxo** em Python define a ordem em que as instruções são processadas, permitindo que você ajuste a execução do programa com base em condições que você definir.

Isso significa que, dependendo de determinadas variáveis ou resultados, o programa pode seguir diferentes direções e realizar ações distintas.

Python nos oferece várias ferramentas para controlar o fluxo de execução, e neste capítulo, vamos focar nas **instruções condicionais**:

- **if**: Executa um bloco de código apenas se a condição for verdadeira.
- **if-else**: Permite escolher entre dois caminhos, executando um bloco de código se a condição for verdadeira e outro se for falsa.
- **if-elif-else**: Oferece ainda mais flexibilidade, permitindo que você verifique múltiplas condições e siga diferentes caminhos dependendo de qual delas for verdadeira.

Essas estruturas são a base para **tomar decisões** em um programa.

Elas permitem que o seu código se adapte a diferentes cenários e se comporte de maneira inteligente, respondendo às condições que surgem durante a execução.

No próximo capítulo, também exploraremos outras estruturas de controle de fluxo, como **laços de repetição** (**while**, **for**) e como interromper ou modificar a

execução desses laços usando as instruções **break** e **continue**. Mas, por enquanto, vamos focar nos **comandos condicionais**, que são o ponto de partida para construir uma lógica mais sofisticada nos seus programas.

Agora, prepare-se para entender como **decisões** são tomadas no código e como você pode usar isso para criar programas que respondem de forma dinâmica a diferentes condições!

Comandos Condicionais

Os comandos condicionais permitem que o programa tome decisões com base em condições.

O comando mais básico é o **if**, que executa um bloco de código apenas se uma condição for verdadeira (**True**). Vamos ver um exemplo simples para entender como funciona:



No exemplo acima, a instrução **if** verifica se a condição **5 > 3** é verdadeira. Como sabemos que 5 é realmente maior que 3, a condição é verdadeira, então o código dentro do bloco **if** é executado, e a mensagem "Sim" é exibida na tela.

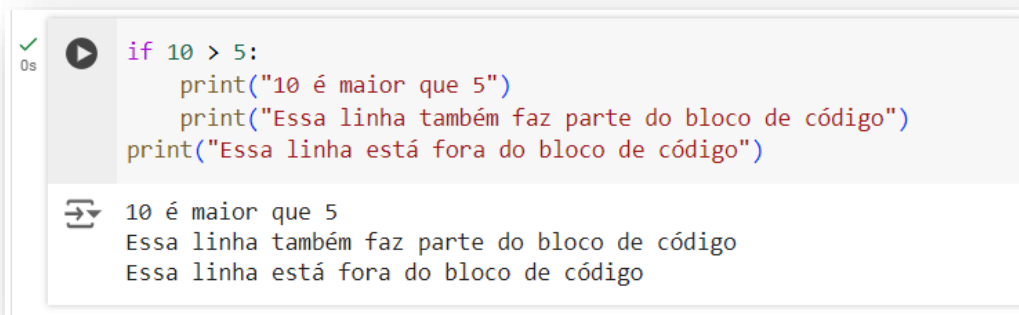
Mas o que são: “Blocos de Código”?

Um **bloco de código** é um conjunto de linhas de código que são agrupadas e executadas juntas. Em Python, um bloco de código é criado usando **indentação**, que é um espaço em branco no início da linha de código.

Na verdade, por convenção, são deixados 4 espaços em branco (aperta-se a barra de espaço 4 vezes). Todavia a maioria das IDEs e Editores de Código já incluem esses espaços, automaticamente, quando digitamos os comandos que exigem um bloco de código, como é o caso dos comandos de controle de fluxo, e teclamos <enter>. Esses editores já inserem esse espaço automaticamente. Isso também acontece quando teclamos <tab> nessas IDEs.

Quando você escreve uma instrução como `if`, o que vem depois dela precisa estar indentado para indicar que faz parte do bloco de código do `if`. Se a condição do `if` for verdadeira, então o Python vai executar todo o código que estiver dentro desse bloco.

Vamos ver um exemplo simples:



```
if 10 > 5:
    print("10 é maior que 5")
    print("Essa linha também faz parte do bloco de código")
print("Essa linha está fora do bloco de código")
```

10 é maior que 5
Essa linha também faz parte do bloco de código
Essa linha está fora do bloco de código

As duas primeiras linhas que começam com `print()` estão indentadas (com espaços em branco antes delas), indicando que fazem parte do **bloco de código** do `if`.

Como `10 > 5` é verdadeiro, as linhas dentro do bloco serão executadas, então veremos no console:

```
10 é maior que 5
Essa linha também faz parte do bloco de código
```

A última linha `print("Essa linha está fora do bloco de código")` não está indentada. Isso significa que ela **não** faz parte do bloco de código do `if`, então ela será executada independentemente da condição.

Por que a Indentação é Importante?

Em Python, a indentação é fundamental para definir onde começa e onde termina um bloco de código. Diferente de outras linguagens de programação que usam chaves `{}` para delimitar blocos, o Python usa a indentação para manter o código organizado e fácil de ler.

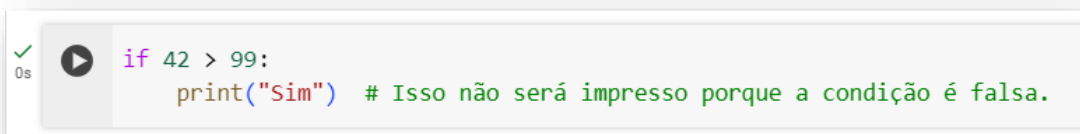
Se você não usar a indentação corretamente, o Python não vai entender que aquelas linhas pertencem ao bloco de código e pode até mostrar um erro.

Portanto, um **bloco de código** é um grupo de instruções que são executadas juntas. Em Python, esse bloco é definido pela **indentação**. Tudo que estiver indentado

após uma instrução como `if` faz parte do bloco de código e só será executado se a condição for verdadeira.

Voltando aos comandos condicionais, eles são como perguntas que fazemos ao programa, e ele responde com uma ação, dependendo da resposta. Se a resposta for "sim" (a condição é verdadeira), o programa faz uma coisa; se for "não" (a condição é falsa), ele pode fazer outra coisa ou simplesmente não fazer nada.

Vamos ver mais um exemplo em que a condição é falsa.



```
✓ 0s if 42 > 99:
    print("Sim") # Isso não será impresso porque a condição é falsa.
```

Neste exemplo, a condição **42 > 99** é claramente falsa, então o código dentro do bloco **if** não será executado. Como a condição não é satisfeita, o Python simplesmente ignora o que está dentro do bloco.

Comando if-else

Mas o que acontece se quisermos que algo aconteça quando a condição for falsa? Para isso, usamos a estrutura **if-else**. Vamos ver um exemplo:



```
✓ 0s if 42 > 99:
    print("Sim")
else:
    print("Não")
```

⇒ Não

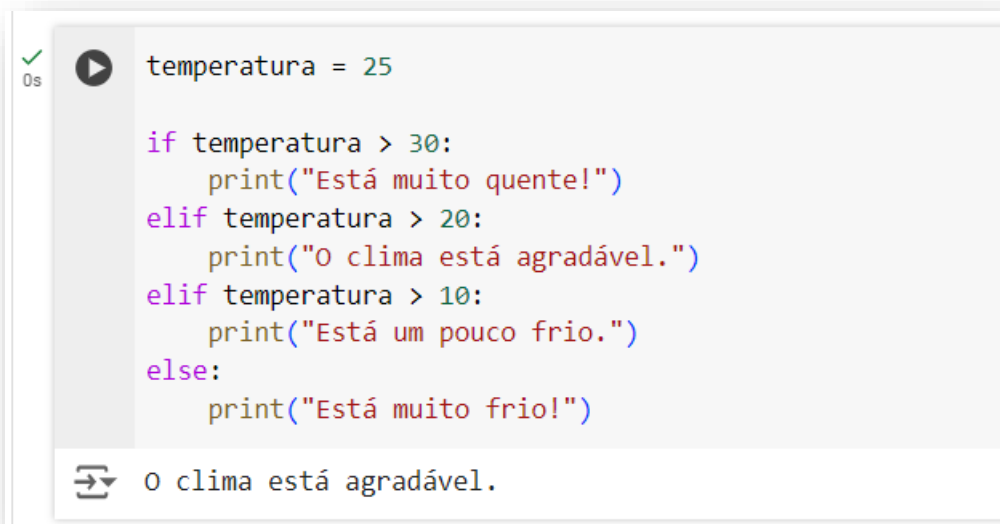
Aqui, se a condição **if** não for satisfeita (como é o caso), o código dentro do bloco **else** será executado. Então, veremos a mensagem "Não" na tela.

A estrutura **if-else** garante que uma das duas ações será realizada, dependendo do resultado da condição.

Comando if-elif-else

Às vezes, temos várias condições diferentes para verificar.

Nesse caso, usamos a estrutura **if-elif-else**, que nos permite testar múltiplas condições em sequência. Vamos explorar um exemplo:



```
✓ 0s ▶ temperatura = 25

if temperatura > 30:
    print("Está muito quente!")
elif temperatura > 20:
    print("O clima está agradável.")
elif temperatura > 10:
    print("Está um pouco frio.")
else:
    print("Está muito frio!")
```

⇒ O clima está agradável.

Primeiro, a condição **temperatura > 30** é verificada. Como é falsa, o Python pula para a próxima condição.

A condição **temperatura > 20** é verdadeira, então o código dentro desse bloco é executado, exibindo **"O clima está agradável."**

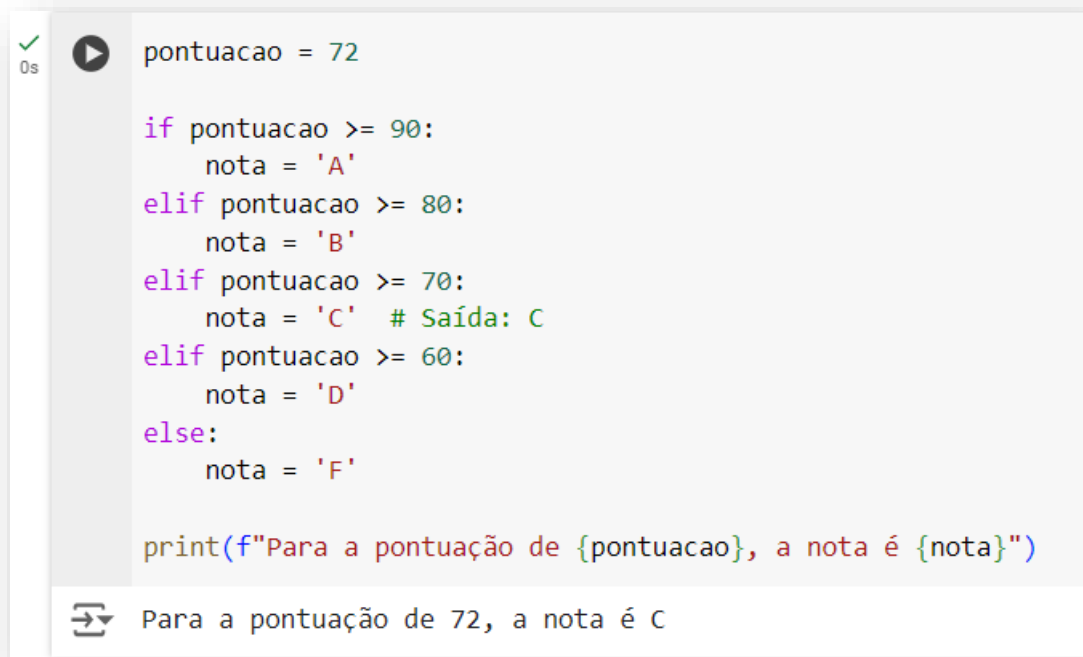
As condições seguintes são ignoradas porque o Python já encontrou uma condição verdadeira.

Você pode executar novamente esse código acima, alterando os valores da temperatura. Tente: 32, depois 15, e por último tente 5. Você verá que para cada nova temperatura, uma mensagem diferente aparecerá.

A estrutura **if-elif-else** é útil para lidar com várias condições de maneira organizada, permitindo que apenas um bloco de código seja executado.

Um exemplo final

Vamos ver um exemplo prático que classifica uma pontuação em uma nota escolar:



```
0s ▶ pontuacao = 72

if pontuacao >= 90:
    nota = 'A'
elif pontuacao >= 80:
    nota = 'B'
elif pontuacao >= 70:
    nota = 'C' # Saída: C
elif pontuacao >= 60:
    nota = 'D'
else:
    nota = 'F'

print(f"Para a pontuação de {pontuacao}, a nota é {nota}")
```

➡ Para a pontuação de 72, a nota é C

O código verifica se a pontuação é maior ou igual a 90, depois 80, 70 e assim por diante.

Como a pontuação é 72, a condição `pontuacao >= 70` é a primeira verdadeira, então o Python atribui a nota 'C' e ignora as outras condições.

Neste capítulo, você aprendeu como usar os comandos `if`, `if-else` e `if-elif-else` para tomar decisões e controlar o fluxo dos seus programas com base em diferentes condições.

Esses comandos são essenciais para criar programas que se comportam de maneira **dinâmica** e **inteligente**, adaptando-se a diferentes cenários de execução.

Agora que você já domina as estruturas condicionais, está preparado para o próximo grande passo: os **laços de repetição**.

Enquanto os comandos condicionais permitem que seu código escolha entre diferentes caminhos, os laços de repetição vão permitir que seu código **repita ações** automaticamente, o que é fundamental quando você precisa executar a mesma tarefa múltiplas vezes, seja para percorrer listas, processar dados ou implementar lógica repetitiva.

No próximo capítulo, vamos explorar os laços **while** e **for**, duas estruturas poderosas que permitem controlar o número de repetições no seu código com base em condições.

Além disso, veremos como as instruções **break** e **continue** podem ser usadas para interromper ou modificar o fluxo desses laços, tornando suas repetições mais eficientes e flexíveis.

Com as ferramentas de controle de fluxo e repetição, você estará pronto para construir programas mais robustos e sofisticados, capazes de lidar com uma grande variedade de situações e desafios.

Vamos em frente, pois agora o poder de repetir e otimizar tarefas em Python está ao seu alcance!



PARA SABER MAIS ...

Python Documentation: Explicação detalhada dos comandos condicionais em Python. https://docs.python.org/3/reference/compound_stmts.html#the-if-statement

Livros e cursos: Acesse <https://pypro.com.br> para Livros e Cursos sobre a linguagem Python. O curso “*Python Profissional: do Zero ao Mercado de Trabalho*” apresenta o que são e como utilizar os Comandos Condicionais, com exemplos reais e exercícios para fixação dos conteúdos.

7

Controle de Fluxo: Comandos de Repetição

Agora que já entendemos como usar os comandos condicionais para tomar decisões em nossos programas, chegou a hora de aprender sobre uma das ferramentas mais poderosas da programação: os **comandos de repetição**.

Quando você precisa executar um bloco de código várias vezes, seja para percorrer uma lista de itens ou repetir uma tarefa até que uma condição seja atendida, os comandos de repetição entram em cena.

Eles permitem que o seu programa faça o mesmo trabalho de maneira mais **eficiente e automatizada**, sem a necessidade de duplicar o código manualmente.

Repetir tarefas automaticamente não só torna o código mais compacto, como também mais fácil de manter e ajustar. Imagine, por exemplo, que você precise processar uma grande quantidade de dados ou realizar cálculos repetidos.

Em vez de escrever o mesmo código várias vezes, você pode usar **laços de repetição** para executar o mesmo conjunto de instruções com base em uma condição específica ou ao percorrer um conjunto de elementos.

Em Python, temos dois tipos principais de laços de repetição, cada um com sua função específica:

- **while:** Executa um bloco de código repetidamente enquanto uma condição é verdadeira.
- **for:** Itera sobre uma sequência (como listas, tuplas ou strings) e executa um bloco de código para cada item da sequência.

Além desses, há também comandos que nos permitem controlar melhor o comportamento dos laços:

- **break:** Interrompe o laço antes que ele tenha terminado de forma natural.
- **continue:** Faz com que o laço pule para a próxima iteração, ignorando o código restante na iteração atual.

No decorrer deste capítulo, vamos explorar cada um desses comandos em detalhes. Com exemplos práticos, você aprenderá a usar os **laços while e for** para resolver problemas de repetição, e verá como as instruções **break e continue** podem ajudá-lo a ajustar o controle do fluxo dentro desses laços.

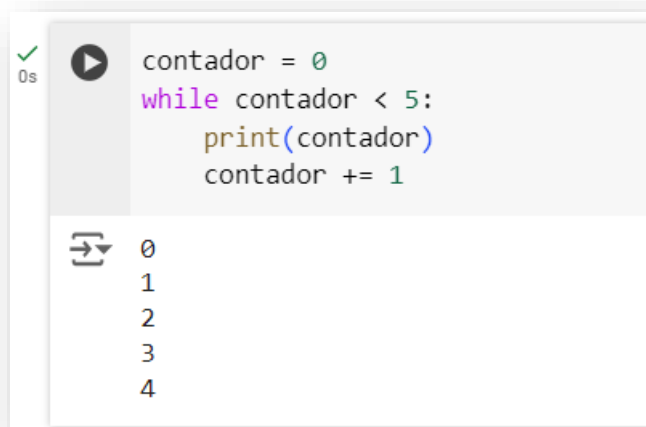
Esses comandos são essenciais para construir programas mais **dinâmicos e eficientes**, e entender como e quando usá-los abrirá uma nova dimensão de possibilidades para o seu código.

Vamos começar essa jornada pelos comandos de repetição e ver como eles podem automatizar tarefas e tornar seus programas muito mais poderosos!

Laço while

O laço **while** é usado quando queremos repetir um bloco de código **enquanto uma condição for verdadeira**. Ele continua executando o bloco até que a condição se torne falsa.

Vamos ver um exemplo bem simples para você possa entender o processo corretamente:

A screenshot of a code editor interface. At the top left, there is a green checkmark icon and the text '0s'. To the right of this is a play button icon. The code is written in Python and consists of four lines: 'contador = 0', 'while contador < 5:', ' print(contador)', and ' contador += 1'. Below the code, there is a list of numbers: 0, 1, 2, 3, 4, each preceded by a small icon that looks like a document with a checkmark.

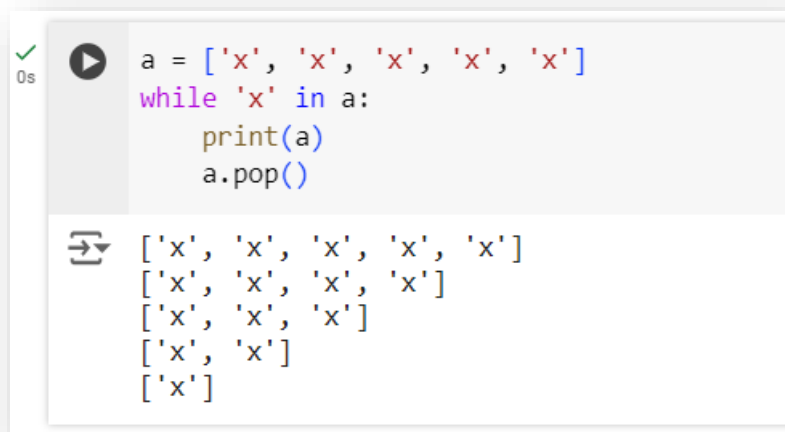
```
✓ 0s ▶ contador = 0
while contador < 5:
    print(contador)
    contador += 1
```

0
1
2
3
4

No exemplo acima, começamos definindo uma variável contador com o valor 0.

O laço **while** executa enquanto a condição **contador < 5** for verdadeira. Em cada repetição, imprimimos o valor de **contador** e depois aumentamos seu valor em 1. Quando contador atinge 5, a condição se torna falsa e o laço para.

Para que você possa observar esse consumo veja o exemplo a seguir:



```
0s a = ['x', 'x', 'x', 'x', 'x']
while 'x' in a:
    print(a)
    a.pop()
```

The output shows the list being printed at each iteration as it shrinks:

```
['x', 'x', 'x', 'x', 'x']
['x', 'x', 'x', 'x']
['x', 'x', 'x']
['x', 'x']
['x']
```

Note que a cada iteração, um elemento a menos é observado na lista. O método `pop()` vai retirando um a um a cada iteração, até que quando a lista se torna vazia, não existe mais 'x' em `a`. Portanto, o laço é interrompido.

Vamos ver um outro exemplo, um pouco diferente. Ao invés agora de termos uma condição, vamos fazer o consumo de elementos de um **iterável**, no caso uma lista.

Mas o que é um iterável?

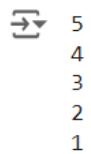
Um **iterável** é qualquer objeto em Python que pode ser percorrido, ou seja, que podemos passar por seus elementos um de cada vez. Objetos como **listas**, **tuplas**, **strings**, e **conjuntos** são todos iteráveis, pois podemos usar um laço (como `for` ou `while`) para acessar cada item que eles contêm.

Um jeito fácil de entender é pensar em um iterável como uma fila de coisas. Podemos pegar um item de cada vez até que todos os itens tenham sido usados.

Quando usamos um laço para percorrer uma lista ou uma string, estamos trabalhando com um iterável.

Agora, vamos utilizar o comando `while` com um iterável.

```
▶ numeros = [1, 2, 3, 4, 5]
while numeros:          # O laço continua enquanto a lista não estiver vazia
    print(numeros.pop()) # Remove e imprime o último elemento da lista
```



```
↔ 5
   4
   3
   2
   1
```

Iterável: A lista **numeros** é um iterável, o que significa que podemos percorrer cada um dos seus elementos, um de cada vez.

Enquanto a lista não estiver vazia: A condição **while numeros** verifica se a lista ainda tem elementos. Em Python, uma lista vazia é considerada **False**, então o laço para de executar quando a lista fica vazia.

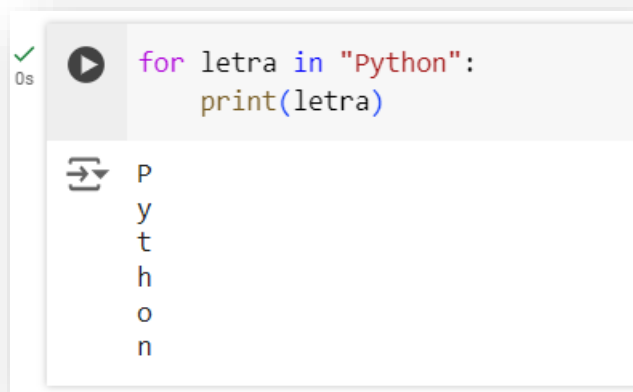
Método pop(): O método **pop()** remove e retorna o último elemento da lista. No primeiro passo, ele retira o número 5, depois 4, e assim por diante, até que todos os elementos tenham sido removidos.

Então, o que estamos fazendo aqui é consumir todos os elementos de um iterável (neste caso, uma lista) usando um laço **while**. O laço continua repetindo enquanto ainda houver elementos na lista. A cada repetição, o método **pop()** remove o último elemento, e nós o exibimos na tela. Assim que a lista se esvazia, o laço para.

Laço for

O laço **for** é mais adequado quando sabemos exatamente **quantas vezes queremos repetir uma ação**. Ele é frequentemente usado para percorrer listas, tuplas, conjuntos, strings ou até intervalos de números.

Vamos ver um exemplo do laço **for**, interagindo sobre uma String...



```
✓ 0s ▶ for letra in "Python":  
      print(letra)
```

↔ P
y
t
h
o
n

O laço **for** percorre cada letra da string "Python" e a imprime uma por uma. Isso é muito útil para acessar e manipular cada elemento de uma sequência. Da mesma forma que ele interagiu com uma string, esse comando pode interagir com outras estruturas iterativas. Vamos ver como ele se sai com uma lista!



```
✓ 0s ▶ frutas = ["maçã", "banana", "cereja", "laranja"]  
      for fruta in frutas:  
          print(fruta)
```

↔ maçã
banana
cereja
laranja

Veja que aqui ele (o comando **for**) a cada passada, fez o consumo de um elemento da lista, até que ela estivesse vazia. A cada iteração, ele colocava o valor que consumiu na variável **fruta** e então exibia o elemento correspondente com o comando **print**.

A função **range()**

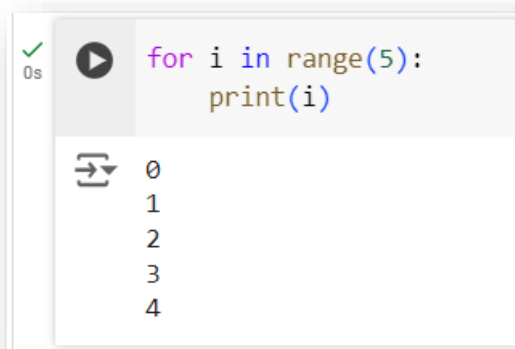
Range() é uma função poderosa em Python que é frequentemente usada em conjunto com laços **for** para gerar uma sequência de números.

É muito útil quando sabemos exatamente quantas vezes queremos repetir uma ação ou quando precisamos percorrer uma série de números de maneira ordenada.

A função **range()** pode aceitar até três parâmetros: **start** (início), **stop** (parada) e **step** (passo). Vamos entender o que cada um faz:

1. **start (início, opcional):** Indica o número onde a sequência começa. Se não for especificado, o valor padrão é 0.
2. **stop (parada, obrigatório):** Indica onde a sequência termina. **Importante:** o valor de stop **não é incluído** na sequência, ou seja, a sequência vai até stop - 1. Se o valor de parada é 8, a sequência vai até 7.
3. **step (passo, opcional):** Indica o intervalo entre os números da sequência. O valor padrão é 1, mas você pode especificar outro valor para pular números.

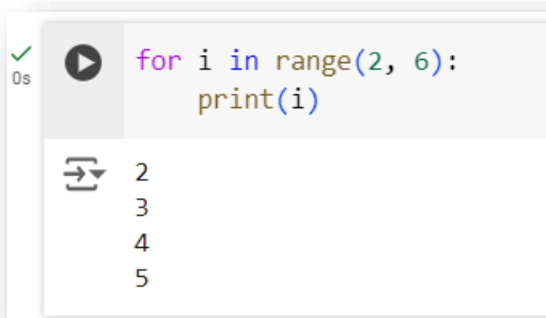
Para fixar, vamos explorar diferentes maneiras de usar a função **range()** com alguns exemplos:



```
✓ 0s ▶ for i in range(5):  
      print(i)  
↔ 0  
   1  
   2  
   3  
   4
```

Quando usamos apenas um valor, como **range(5)**, o Python assume que este é o valor de parada e começa a sequência em 0, indo até 4 (um número antes do valor de parada).

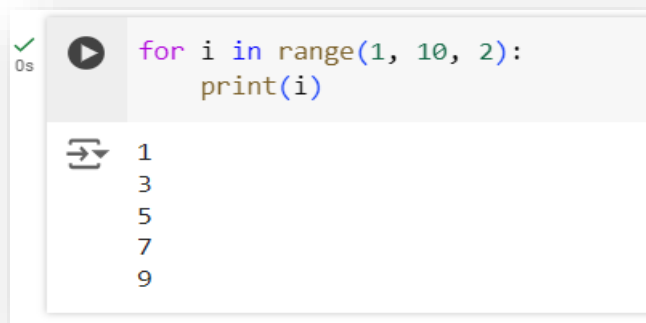
Vamos ver agora um exemplo com Início e Parada (start, stop)



```
✓ 0s ▶ for i in range(2, 6):  
      print(i)  
↔ 2  
   3  
   4  
   5
```

Neste exemplo, a sequência começa em 2 e vai até 5, pois o valor de parada 6 não é incluído.

Vamos incluir agora todos os parâmetros da função **range()**.

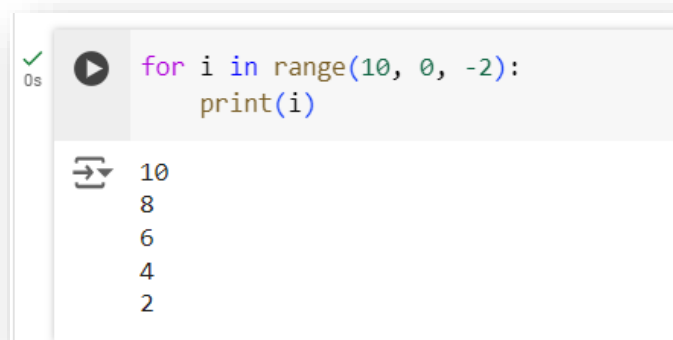


```
✓ 0s ▶ for i in range(1, 10, 2):  
      print(i)
```

↔ 1
3
5
7
9

Aqui, a sequência começa em 1, termina em 9, e aumenta de 2 em 2 (devido ao passo especificado). Esse é um uso comum quando precisamos pular números em uma sequência.

Para finalizar os exemplos com a função **range()**, também podemos usá-la para contar de trás para frente, utilizando um passo negativo:



```
✓ 0s ▶ for i in range(10, 0, -2):  
      print(i)
```

↔ 10
8
6
4
2

Neste exemplo, a sequência começa em 10 e diminui até 2, contando de 2 em 2 devido ao passo negativo -2.

Como você pode perceber, a função **range()** é muito eficiente e fácil de usar quando precisamos gerar sequências de números para iterar em laços for.

Ela nos permite controlar exatamente de onde começamos, onde terminamos e de quanto em quanto queremos avançar, tornando os nossos laços mais precisos e poderosos.

Comandos break e continue

break: Interrompendo um Laço

O comando break é usado para **sair do laço completamente**, mesmo que a condição ainda seja verdadeira.

A screenshot of a code editor showing a Python script. The script defines a list 'animais' with elements 'gato', 'cachorro', 'papagaio', 'tigre', and 'elefante'. It then enters a 'for' loop that iterates over each animal. Inside the loop, there is an 'if' statement that checks if the current animal is 'tigre'. If it is, the 'break' statement is executed, which immediately exits the loop. After the loop, the 'print(animal)' statement is executed. The output of the program is shown below the code: 'gato', 'cachorro', and 'papagaio'.

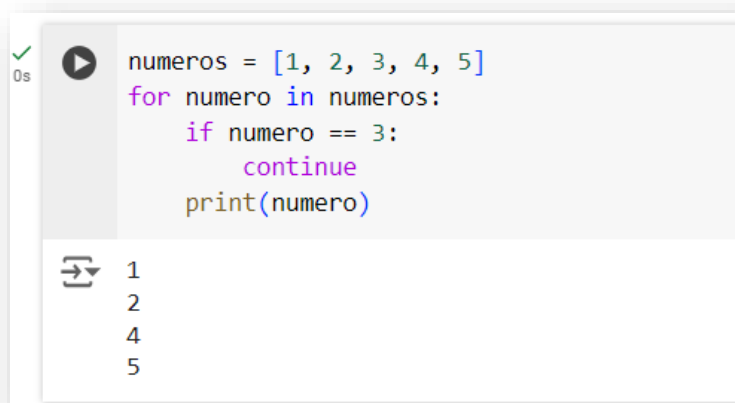
```
animais = ["gato", "cachorro", "papagaio", "tigre", "elefante"]
for animal in animais:
    if animal == "tigre":
        break
    print(animal)
```

gato
cachorro
papagaio

O laço para de executar assim que encontra "**tigre**", graças ao comando **break**, que interrompe o laço assim que a condição é satisfeita.

continue: Pulando para a Próxima Iteração

O comando continue faz com que o laço pule a iteração atual e passe diretamente para a próxima.

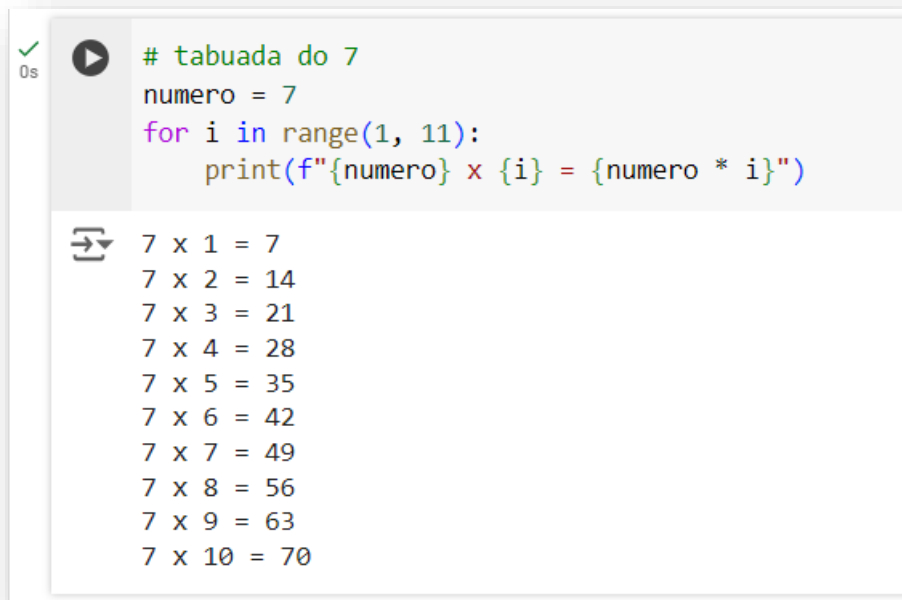
A screenshot of a code editor showing a Python script. The script defines a list 'numeros' with elements 1, 2, 3, 4, and 5. It then enters a 'for' loop that iterates over each number. Inside the loop, there is an 'if' statement that checks if the current number is 3. If it is, the 'continue' statement is executed, which skips the rest of the current iteration and jumps to the start of the next iteration. After the 'if' statement, the 'print(numero)' statement is executed. The output of the program is shown below the code: 1, 2, 4, and 5.

```
numeros = [1, 2, 3, 4, 5]
for numero in numeros:
    if numero == 3:
        continue
    print(numero)
```

1
2
4
5

Quando o valor 3 é encontrado, o comando **continue** faz com que o laço pule essa iteração e passe para o próximo número, ignorando o valor 3.

E para finalizar, vamos criar uma **Tabela de Multiplicação** (também conhecida como **Tabuada**) com o comando **for**.



```
# tabuada do 7
numero = 7
for i in range(1, 11):
    print(f'{numero} x {i} = {numero * i}')
```

7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70

Este exemplo gera a tabela de multiplicação do número 7 de forma clara e organizada, mostrando como usar laços para automatizar cálculos.

Neste capítulo, exploramos como os **comandos de repetição** são fundamentais para automatizar tarefas e manipular grandes volumes de dados em Python.

Ao entender e praticar o uso dos laços **while** e **for**, juntamente com as instruções **break** e **continue**, você adquiriu ferramentas essenciais para criar programas mais **eficientes** e **versáteis**, capazes de lidar com cenários repetitivos e complexos de forma simplificada.

Com os comandos de repetição, você pode reduzir o esforço manual, escrever códigos mais compactos e aprimorar a performance dos seus programas, tornando-os mais **dinâmicos** e **flexíveis**.

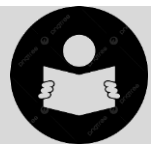
Agora que você dominou a repetição e o controle de fluxo, estamos prontos para avançar ainda mais na organização e estruturação de códigos.

No próximo capítulo, vamos descobrir o poder das **funções em Python**. As funções são uma maneira poderosa de encapsular blocos de código, permitindo que você **reutilize** e **organize** seu programa de forma mais eficiente e modular.

Com elas, você pode quebrar grandes problemas em partes menores, simplificando o processo de desenvolvimento e manutenção dos seus programas.

Vamos aprender como **criar nossas próprias funções** e ver como elas podem ajudar a resolver problemas de maneira mais **estruturada, elegante, e escalável**.

Este próximo passo será fundamental para tornar seu código mais limpo e organizado, permitindo que você desenvolva programas mais sofisticados e fáceis de manter.



PARA SABER MAIS ...

Python Documentation: Explicação detalhada do comando while em Python.
https://docs.python.org/3/reference/compound_stmts.html#the-while-statement

Python Documentation: Explicação detalhada do comando for em Python.
https://docs.python.org/3/reference/compound_stmts.html#the-for-statement

Python Documentation: Explicação detalhada do comando break em Python.
https://docs.python.org/3/reference/simple_stmts.html#the-break-statement

Python Documentation: Explicação detalhada do comando continue em Python.
https://docs.python.org/3/reference/simple_stmts.html#the-continue-statement

Livros e cursos: Acesse <https://pypro.com.br> para Livros e Cursos sobre a linguagem Python. O curso “*Python Profissional: do Zero ao Mercado de Trabalho*” apresenta o que são e como utilizar os Comandos de Repetição, com exemplos reais e exercícios para fixação dos conteúdos.

8

Funções

Vamos começar pensando em uma função como algo que fazemos todos os dias. Imagine que você está na cozinha preparando um café. Para fazer isso, você segue uma série de passos bem definidos: ferver a água, preparar o filtro, adicionar o pó de café, despejar a água quente e, finalmente, servir o café na xícara.

Essa série de passos é uma "função" que você repete sempre que precisa fazer café.

Agora, suponha que você precise fazer café todos os dias para sua família.

Em vez de lembrar e executar cada passo do processo manualmente, seria muito mais fácil e rápido se você tivesse uma **máquina de café** que faz tudo para você com o apertar de um botão, certo?

Essa máquina é um exemplo perfeito de como uma função pode automatizar uma tarefa, poupando seu tempo e esforço.

No **mundo da programação**, as funções funcionam de forma muito parecida. Elas são blocos de código que executam uma tarefa específica sempre que você as "aperta", ou melhor, sempre que você as chama.

Em vez de escrever o mesmo conjunto de instruções várias vezes, você cria uma função uma vez e a reutiliza sempre que precisar.

As funções ajudam a tornar o seu código mais **organizado, eficiente e fácil de entender**, assim como uma máquina de café facilita o processo de fazer café. Vamos ver como isso funciona na prática com um exemplo simples em Python.

Criando Funções em Python

Para definir uma função em Python, usamos a palavra-chave **def**, seguida pelo nome da função, parênteses **()** e dois pontos **:**. Vamos começar com uma função simples que faz uma tarefa básica.

Vamos criar uma função bem simples... Imagine que queremos criar uma função que simule a ação de dar boas-vindas a alguém:

```
✓ [1] def boas_vindas():  
0s      print("Bem-vindo! Estamos felizes em ter você aqui.")
```

def: indica ao Python que estamos definindo uma nova função.

boas_vindas: é o nome da função, escolhido para representar claramente o que a função faz.

O bloco de código indentado que segue é o que a função executará quando for chamada.

Agora, para usar essa função, basta chamá-la como se estivesse apertando o botão da máquina de café:

```
✓ 0s ▶ boas_vindas()  
⇄ Bem-vindo! Estamos felizes em ter você aqui.
```

Por que usar funções?

Imagine que você precisa calcular a mesma coisa em diferentes partes do seu programa.

Em vez de escrever o mesmo bloco de código repetidamente, você pode criar uma função que faz esse cálculo e simplesmente chamá-la sempre que precisar.

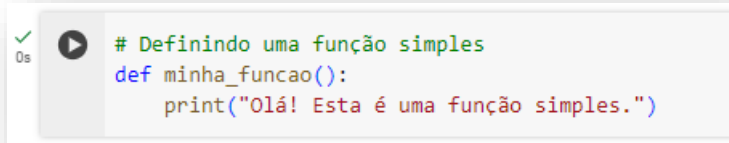
Isso deixa o código mais limpo, organizado e mais fácil de manter.

Criando Funções em Python

Nós já criamos uma função acima de **boas_vindas()**. Mas vamos ver como isso aconteceu, detalhadamente.

Para definir uma função em Python, usamos a palavra-chave **def**, seguida pelo nome da função, parênteses **()** e dois pontos **:**. O bloco de código que faz parte da função deve ser indentado (ter um deslocamento de quatro espaços em branco no início de cada linha). Vamos ver como isso funciona na prática.

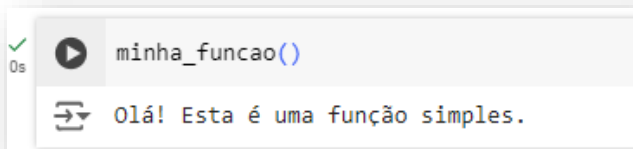
Sintaxe Básica de uma Função

A code editor snippet with a green checkmark and '0s' in the top left corner. It contains a single line of Python code:

```
# Definindo uma função simples
def minha_funcao():
    print("Olá! Esta é uma função simples.")
```

- **def:** é uma palavra-chave usada para dizer ao Python que estamos definindo uma nova função.
- **minha_funcao:** é o nome da função. Você pode escolher um nome que descreva bem o que a função faz.
- **():** são parênteses usados para passar informações para a função (veremos isso em detalhes mais tarde).
- O bloco de código que a função executará é indentado para indicar que faz parte da função. Aqui foi colocada apenas uma linha (um comando). Aqui se pode colocar quantos comandos / linhas for necessários.

Depois que a função é definida, podemos **chamá-la** (em inglês: **call** the function) sempre que quisermos que ela execute seu código. No caso da função acima, colocamos o nome que definimos dessa função, seguido de **()** (parênteses). Assim, teríamos:

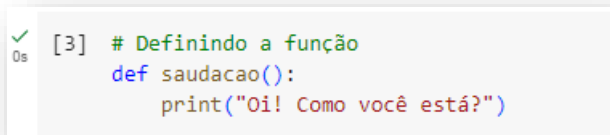
A code editor snippet with a green checkmark and '0s' in the top left corner. It shows the function call

```
minha_funcao()
```

 on the first line. Below it, the output of the function is displayed:

```
Olá! Esta é uma função simples.
```

Vamos ver um exemplo completo:

A code editor snippet with a green checkmark and '0s' in the top left corner. It contains two lines of Python code:

```
[3] # Definindo a função
def saudacao():
    print("Oi! Como você está?")
```

```
✓ 0s # Chamando a função
    saudacao()

    ➞ Oi! Como você está?
```

Explicação:

- Primeiro, definimos a função chamada `saudacao` que imprime a mensagem "Oi! Como você está?".
- Em seguida, chamamos a função usando seu nome, e ela executa o bloco de código dentro dela.

Funções com Parâmetros

Podemos tornar as funções mais poderosas passando **parâmetros** para elas. Parâmetros são como variáveis que recebem valores quando a função é chamada.

Eles permitem que a função trabalhe com dados diferentes a cada vez que é executada.

Vamos ver um exemplo desse tipo de função:

```
✓ 0s def saudacao_personalizada(nome):
    print(f"Olá, {nome}! Seja bem-vindo!")
```

Vamos ver agora como podemos fazer a chamada dessa função:

```
✓ 0s [6] saudacao_personalizada("João")

    ➞ Olá, João! Seja bem-vindo!
```

```
✓ 0s saudacao_personalizada("Maria")

    ➞ Olá, Maria! Seja bem-vindo!
```

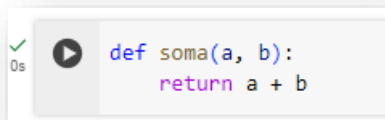
Definimos uma função chamada `saudacao_personalizada` que recebe um parâmetro chamado `nome`. Quando chamamos a função e passamos o valor

"Maria", a função usa esse valor para exibir a mensagem "Olá, Maria! Seja bem-vindo!".

Podemos reutilizar essa mesma função para saudar qualquer pessoa, apenas passando nomes diferentes.

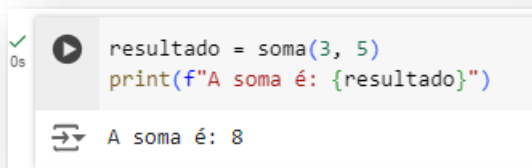
Funções com Retorno de Valores

Às vezes, precisamos que uma função nos dê de volta um resultado. Para isso, usamos a palavra-chave **return**. Vamos ver um exemplo:

A screenshot of a code editor interface. On the left, there is a green checkmark and a play button icon, with '0s' below them. The code area contains the following Python code:

```
def soma(a, b):  
    return a + b
```

Vamos então chamar a função e armazenar o resultado em uma variável. Em seguida, vamos exibir o valor dessa variável...

A screenshot of a code editor interface. On the left, there is a green checkmark and a play button icon, with '0s' below them. The code area contains the following Python code:

```
resultado = soma(3, 5)  
print(f"A soma é: {resultado}")
```

Below the code area, there is a terminal window showing the output:

```
A soma é: 8
```

A função **soma** recebe dois parâmetros **a** e **b**, e retorna o valor da soma desses números.

Usamos **return** para devolver o resultado para onde a função foi chamada.

Armazenamos o resultado em uma variável **resultado** e o imprimimos na tela.

Funções como Automatizadores de Tarefas

Vamos usar outro exemplo para entender como as funções podem ser úteis para automatizar processos repetitivos.

Imagine que você está em uma loja e precisa calcular o total a pagar aplicando um desconto em vários produtos.

Em vez de fazer o cálculo manualmente para cada produto, podemos criar uma função para isso.

```
0s  def calcular_desconto(preco, desconto):  
    preco_com_desconto = preco - (preco * desconto / 100)  
    return preco_com_desconto
```

Aqui, **preco** e **desconto** são como os ingredientes que colocamos na máquina de café — são os valores que a função precisa para funcionar. A função então faz o cálculo e retorna o **preço final** com o desconto aplicado.

Vamos ver como usar essa função em diferentes situações:

```
0s  produto1 = calcular_desconto(100, 10) # Calcula 10% de desconto em R$ 100  
produto2 = calcular_desconto(200, 15) # Calcula 15% de desconto em R$ 200  
  
print(f"Preço do produto 1 com desconto: R$ {produto1}")  
print(f"Preço do produto 2 com desconto: R$ {produto2}")  
  
 Preço do produto 1 com desconto: R$ 90.0  
Preço do produto 2 com desconto: R$ 170.0
```

Neste exemplo, usamos a função **calcular_desconto** para calcular automaticamente o preço final de dois produtos diferentes.

Em vez de repetir o cálculo manualmente, podemos usar a função para realizar essa tarefa em uma única linha.

Isso é muito mais eficiente, especialmente quando estamos lidando com um grande número de produtos.

Antes de concluirmos nosso capítulo sobre funções, é importante explorar uma função especial que permite a interação direta entre o programa e a pessoa que está usando: a função **input()**.

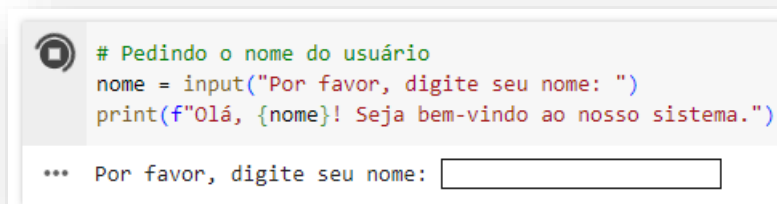
Interação com o Usuário: Função **input()**

Esta função nos permite criar programas mais interativos, onde o usuário pode fornecer informações que o programa utilizará para realizar cálculos ou tomar decisões. Vamos ver como essa função funciona e como ela pode tornar nossos programas muito mais dinâmicos.

A função `input()` é como uma maneira de fazer perguntas ao usuário e obter respostas que o programa pode usar para realizar cálculos ou tomar decisões.

Se a função `print()` é usada para mostrar mensagens na tela, a função `input()` faz o oposto: ela espera que o usuário digite algo e então retorna essa entrada como um valor que podemos usar no programa.

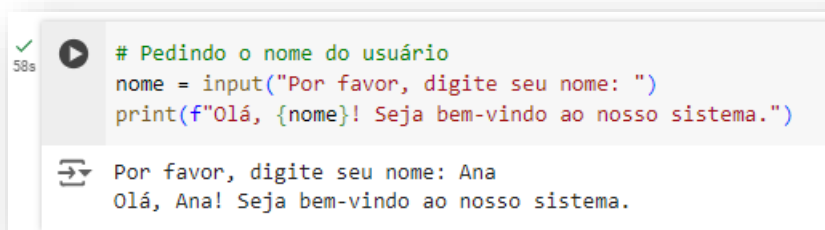
Vamos ver um exemplo simples de como isso funciona:



```
# Pedindo o nome do usuário
nome = input("Por favor, digite seu nome: ")
print(f"Olá, {nome}! Seja bem-vindo ao nosso sistema.")
```

*** Por favor, digite seu nome:

`input("Por favor, digite seu nome: ")` — Essa linha exibe uma mensagem pedindo que o usuário digite seu nome.



```
# Pedindo o nome do usuário
nome = input("Por favor, digite seu nome: ")
print(f"Olá, {nome}! Seja bem-vindo ao nosso sistema.")
```

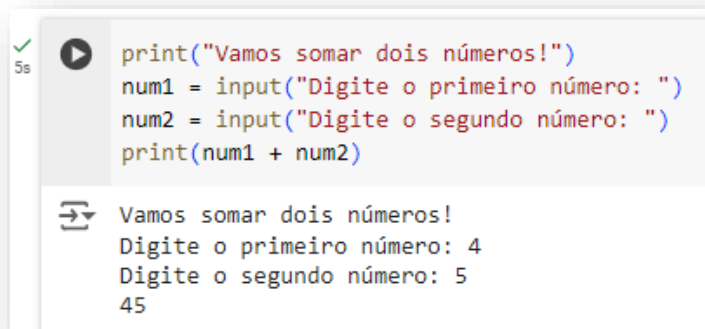
Por favor, digite seu nome: Ana
Olá, Ana! Seja bem-vindo ao nosso sistema.

Assim que o usuário digita a resposta (“Ana”) e pressiona Enter, o programa armazena o valor na variável `nome`.

Usamos a função `print()` para mostrar uma mensagem personalizada com o nome que o usuário digitou.

Um Detalhe Importante: Entrada como String

Por padrão, a função `input()` sempre recebe a entrada do usuário como uma **string** (texto). Isso significa que, mesmo que o usuário digite números, eles serão tratados como texto. Vamos ver um exemplo para entender melhor:



```
5s ▶ print("Vamos somar dois números!")
    num1 = input("Digite o primeiro número: ")
    num2 = input("Digite o segundo número: ")
    print(num1 + num2)
```

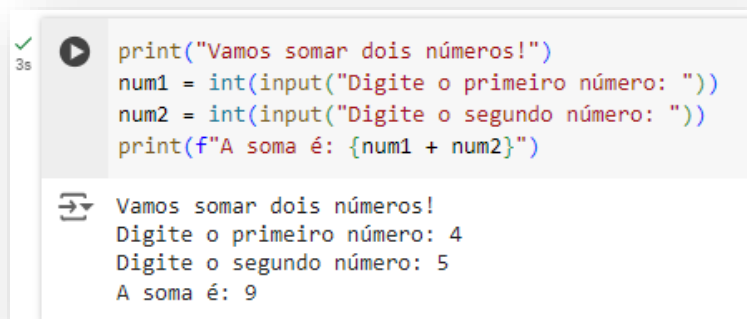
↳ Vamos somar dois números!
Digite o primeiro número: 4
Digite o segundo número: 5
45

Você esperaria que a resposta fosse **9**, mas o programa mostra **45**.

Isso acontece porque **num1** e **num2** são tratados como **strings**, e quando somamos duas strings em Python, elas são **concatenadas** (juntadas) em vez de serem somadas como números.

Como podemos resolver isso? Podemos fazer a conversão da entrada para números...

Para que a entrada do usuário seja tratada como números, precisamos converter as strings em **inteiros** usando a função **int()**. Vamos ajustar o exemplo anterior para fazer isso:



```
3s ▶ print("Vamos somar dois números!")
    num1 = int(input("Digite o primeiro número: "))
    num2 = int(input("Digite o segundo número: "))
    print(f"A soma é: {num1 + num2}")
```

↳ Vamos somar dois números!
Digite o primeiro número: 4
Digite o segundo número: 5
A soma é: 9

Agora sim... temos o resultado esperado: **9**.

Usamos a função **int()** para converter a entrada do usuário em números inteiros, permitindo que a operação de soma seja realizada corretamente.

Agora, quando somamos **num1** e **num2**, obtemos a resposta esperada.

Portanto, assim como uma máquina de café que nos poupa tempo, as funções em programação ajudam a automatizar tarefas repetitivas, tornando o código mais

rápido e fácil de manter. Elas também tornam o código mais modular — ou seja, você pode dividir grandes tarefas em partes menores e mais gerenciáveis.

As funções são ferramentas poderosas que nos permitem organizar, reutilizar e otimizar nosso código. Ao compreender como criar e usar funções, você dá um passo importante para se tornar um programador mais eficiente e capaz de resolver problemas de forma estruturada.

No próximo capítulo, vamos explorar algo ainda mais empolgante: o uso de **módulos ou pacotes**. Esses recursos são como caixas de ferramentas prontas, cheias de funções já desenvolvidas que você pode usar para simplificar e acelerar seus projetos.

Ao aprender como utilizar esses pacotes, você vai descobrir que muitas soluções para os seus problemas já estão à sua disposição, prontas para serem aproveitadas.

Continue conosco nessa jornada e veja como é fácil aproveitar todo o potencial que o Python oferece!



PARA SABER MAIS ...

Python Documentation: Explicação detalhada do processo de definição de Funções em Python.
https://docs.python.org/3/reference/compound_stmts.html#function-definitions

Livros e cursos: Acesse <https://pypro.com.br> para Livros e Cursos sobre a linguagem Python. O curso “*Python Profissional: do Zero ao Mercado de Trabalho*” apresenta o que são e como criar e utilizar as Funções em Python, com exemplos reais e exercícios para fixação dos conteúdos.

9

Módulos ou Pacotes em Python

À medida que aprendemos mais sobre Python, percebemos que ele tem uma enorme quantidade de recursos e funcionalidades.

Muitos desses recursos são organizados em **módulos** e **pacotes**, que são coleções de funções, variáveis e classes que podemos usar para facilitar o nosso trabalho.

Em vez de escrever todo o código do zero, podemos simplesmente importar esses módulos ou pacotes e aproveitar as soluções que já foram desenvolvidas para resolver problemas específicos.

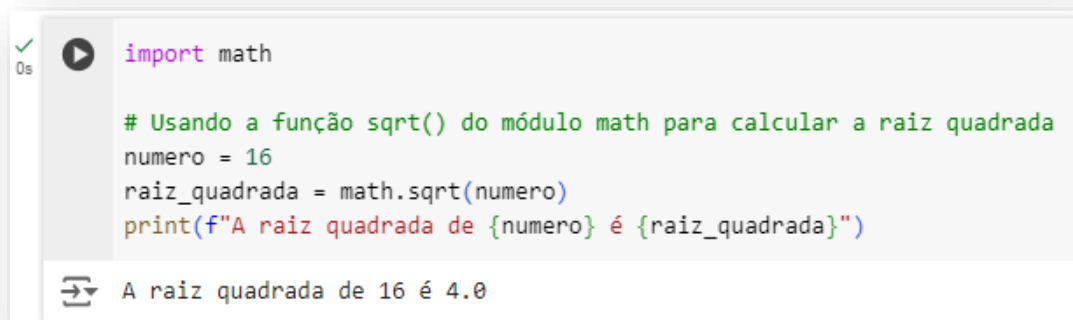
Mas, o que são Módulos?

Um **módulo** em Python é um arquivo que contém definições e implementações de funções, variáveis e classes que podemos reutilizar em outros programas.

Esses módulos ajudam a dividir grandes programas em partes menores e mais gerenciáveis, tornando o código mais organizado e fácil de manter.

Para tanto, vamos ver um exemplo de utilização de módulos... Você já usou um módulo sem perceber! Quando usamos a função `print()` ou `input()`, estamos usando funções que fazem parte de módulos integrados ao Python (dizemos que se trata de funções **builtin**). Mas também existem módulos que precisamos importar para utilizar, como o módulo `math`, que nos dá acesso a várias funções matemáticas.

Vamos ver um exemplo simples de como usar o módulo `math`:

A screenshot of a Python code editor showing a code cell with a green checkmark and a play button icon. The code imports the math module, defines a variable 'numero' with the value 16, calculates the square root using 'math.sqrt(numero)', and prints the result. The output below the code shows 'A raiz quadrada de 16 é 4.0'.

```
import math

# Usando a função sqrt() do módulo math para calcular a raiz quadrada
numero = 16
raiz_quadrada = math.sqrt(numero)
print(f"A raiz quadrada de {numero} é {raiz_quadrada}")
```

A raiz quadrada de 16 é 4.0

import math: Esta linha diz ao Python que queremos usar o módulo **math**.

Depois, usamos a função **sqrt()** do módulo **math** para calcular a raiz quadrada de um número.

Muitas vezes utilizamos os nomes “módulos” e “pacotes” como sinônimos. Todavia, existe uma sutil diferença...

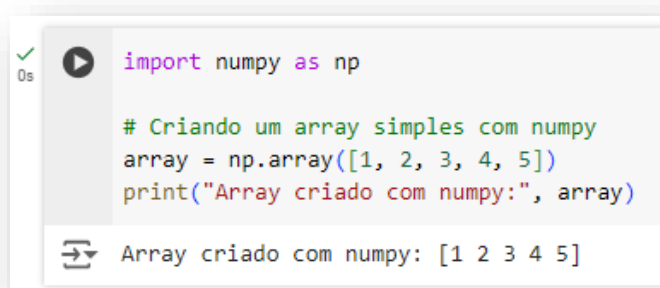
O que são “pacotes”?

Um **pacote** é uma coleção de módulos organizados em uma estrutura de diretórios. É como uma pasta que contém vários módulos, cada um com suas próprias funções e classes. Pacotes nos ajudam a organizar melhor nosso código, especialmente quando estamos trabalhando com projetos grandes.

Vamos ver um exemplo de Pacote em Python ...

Um dos **pacotes** mais populares no Python, é o **numpy**. Ele é amplamente utilizado para operações matemáticas e manipulação de arrays, e é muito eficiente para trabalhar com grandes volumes de dados.

Vamos ver como importar e usar o **numpy**:

A screenshot of a Python code editor showing a code cell with a green checkmark and a play button icon. The code imports numpy as 'np', creates a numpy array with values [1, 2, 3, 4, 5], and prints it. The output below the code shows 'Array criado com numpy: [1 2 3 4 5]'.

```
import numpy as np

# Criando um array simples com numpy
array = np.array([1, 2, 3, 4, 5])
print("Array criado com numpy:", array)
```

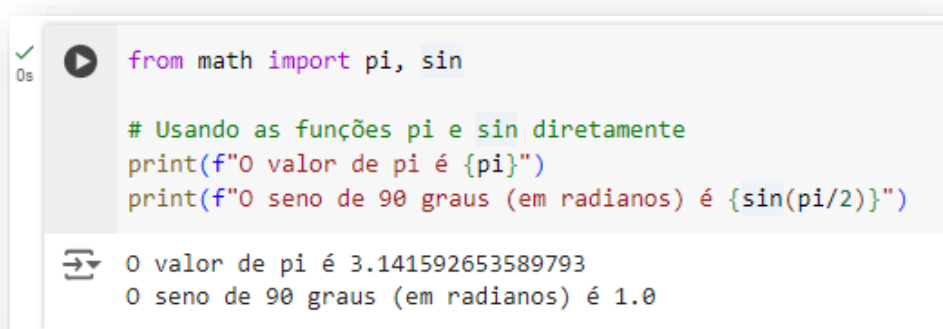
Array criado com numpy: [1 2 3 4 5]

import numpy as np: Essa linha importa o pacote **numpy** e permite que o utilizemos com a abreviação **np**, facilitando a escrita do código.

Usamos a função **array()** do **numpy** para criar um array a partir de uma lista de números.

Importando Funções Específicas

Às vezes, não precisamos importar um módulo ou pacote inteiro; podemos importar apenas uma função específica para economizar memória e tornar o código mais claro.



```
0s  from math import pi, sin

# Usando as funções pi e sin diretamente
print(f"O valor de pi é {pi}")
print(f"O seno de 90 graus (em radianos) é {sin(pi/2)}")

 O valor de pi é 3.141592653589793
O seno de 90 graus (em radianos) é 1.0
```

from math import pi, sin: Nessa linha, importamos apenas as funções **pi** e **sin** do módulo **math**, evitando a necessidade de importar o módulo inteiro.

Podemos usar essas funções diretamente, sem precisar digitar **math.** antes delas. Isso torna o código mais legível e reduz a ocupação de memória.

Por que usar Módulos e Pacotes?

Um dos maiores fatores que fizeram o Python se tornar a linguagem de programação mais popular do mundo é a enorme quantidade de **módulos** e **pacotes** disponíveis.

Esses pacotes prontos para uso permitem que desenvolvedores em todo o mundo criem soluções poderosas de maneira rápida e eficiente.

Em vez de reinventar a roda, você pode aproveitar uma vasta biblioteca de ferramentas que já foram criadas, testadas e otimizadas pela comunidade global.

A principal razão para utilizar módulos e pacotes é que eles tornam o desenvolvimento de software muito mais eficiente, organizado e sustentável. Em vez de escrever todas as funções e estruturas do zero, podemos aproveitar

soluções prontas e testadas que outras pessoas já desenvolveram e colocaram à nossa disposição.

Vamos entender em detalhes por que isso é tão importante.

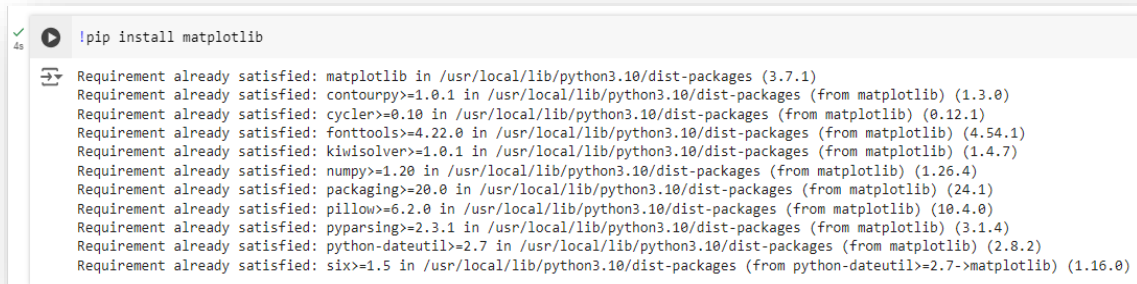
1. **Reutilização de Código** Imagine que você está construindo uma casa e precisa de uma escada. Em vez de fabricar cada degrau manualmente, seria muito mais rápido e eficiente comprar uma escada pronta. Da mesma forma, os módulos e pacotes em Python são como peças prontas de código que podemos usar nos nossos projetos, evitando a necessidade de reinventar a roda.
2. **Organização e Modularidade** Ao dividir o código em módulos e pacotes, conseguimos manter nosso programa organizado, especialmente em projetos maiores e mais complexos. Assim como em uma biblioteca, onde os livros são organizados em seções para facilitar a localização, os módulos e pacotes ajudam a manter as diferentes partes do código separadas e fáceis de encontrar.
3. **Eficácia e Otimização** Muitos pacotes em Python, como numpy, pandas, e scikit-learn, são otimizados para realizar operações complexas de maneira muito eficiente. Isso significa que eles utilizam algoritmos avançados e implementações em baixo nível que são muito mais rápidas do que se tentássemos escrever o código manualmente.
4. **Comunidade e Suporte** Quando usamos pacotes populares, como matplotlib para gráficos ou scikit-learn para aprendizado de máquina, estamos aproveitando a força da comunidade Python. Esses pacotes são amplamente utilizados e têm um grande suporte da comunidade, o que significa que existe muita documentação, exemplos e ajuda disponível online para resolver problemas ou aprender novas funcionalidades.

Assim, o uso de módulos e pacotes não apenas acelerou a adoção do Python em todo o mundo, mas também permitiu que ele se tornasse uma linguagem versátil, adequada para uma ampla gama de aplicações, desde análise de dados e inteligência artificial até desenvolvimento web e automação. Saber como utilizar esses recursos é fundamental para se tornar um programador mais produtivo e eficaz.

Como Instalar Pacotes no Google Colab

No Google Colab, muitos pacotes populares já vêm instalados por padrão. No entanto, se você precisar de um pacote que não está instalado, pode usar o comando `!pip install nome_do_pacote` para instalá-lo diretamente no notebook.

Por exemplo, para instalar o pacote `matplotlib` para visualizações gráficas, você pode usar:



```
!pip install matplotlib

Requirement already satisfied: matplotlib in /usr/local/lib/python3.10/dist-packages (3.7.1)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib) (1.3.0)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.10/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib) (4.54.1)
Requirement already satisfied: kiwisolver>=1.0.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib) (1.4.7)
Requirement already satisfied: numpy>=1.20 in /usr/local/lib/python3.10/dist-packages (from matplotlib) (1.26.4)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib) (24.1)
Requirement already satisfied: pillow>=6.2.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib) (10.4.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib) (3.1.4)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.10/dist-packages (from matplotlib) (2.8.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.10/dist-packages (from python-dateutil>=2.7->matplotlib) (1.16.0)
```

Note que após a execução do comando `!pip install matplotlib`, a resposta foi que o pacote `matplotlib` e seus pacotes dependentes já estavam instalados.

A mensagem "**Requirement already satisfied**" significa que o `matplotlib` e todos os pacotes que ele precisa para funcionar (como `numpy`, `cycler`, `pillow`, entre outros) já estão instalados no ambiente do Google Colab. Portanto, não houve necessidade de reinstalar nada. O pacote `matplotlib` está pronto para uso, e você pode utilizá-lo imediatamente no seu notebook. Essas mensagens são normais e indicam que o ambiente já possui todos os recursos necessários para trabalhar com esse pacote.

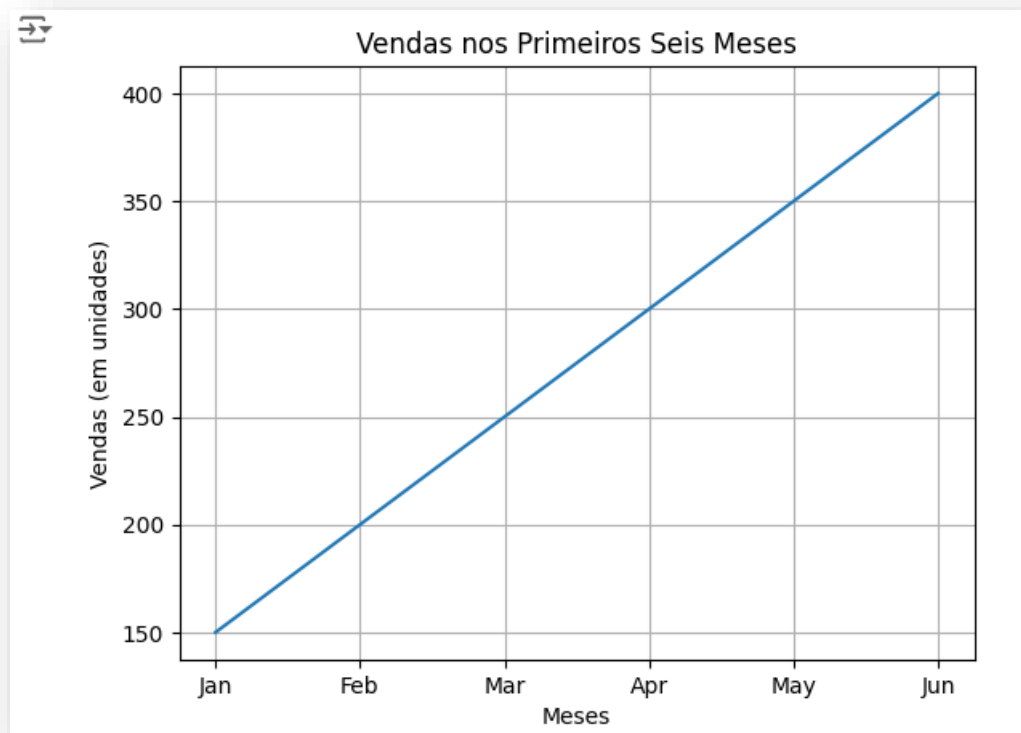
Assim, depois de instalar o pacote `matplotlib`, vamos usá-lo para criar uma visualização gráfica simples. O `matplotlib` é uma biblioteca poderosa para criar gráficos e visualizações que ajudam a entender melhor os dados.

Vamos criar um gráfico de linha simples para mostrar como funciona:

```
0s  import matplotlib.pyplot as plt

# Dados para o gráfico
meses = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun']
vendas = [150, 200, 250, 300, 350, 400]

# Criando o gráfico de linha
plt.plot(meses, vendas)
plt.title('Vendas nos Primeiros Seis Meses')
plt.xlabel('Meses')
plt.ylabel('Vendas (em unidades)')
plt.grid(True)
plt.show()
```



E o gráfico acima é então exibido ao executar o código indicado. Cada uma das linhas tem um significado... a seguir descrevemos cada uma delas de forma bem resumida:

plt.plot(): Cria o gráfico de linha usando os dados fornecidos.

plt.title(), plt.xlabel() e plt.ylabel(): Adicionam um título ao gráfico e rótulos aos eixos.

plt.grid(True): Adiciona uma grade ao fundo do gráfico para facilitar a leitura dos dados.

plt.show(): Exibe o gráfico na tela.

Este é um exemplo básico, mas o **matplotlib** é uma ferramenta muito poderosa que pode criar visualizações complexas para nos ajudar a analisar e entender os dados de maneira mais clara.

Os módulos e pacotes são como caixas de ferramentas prontas que podemos usar para resolver uma ampla variedade de problemas sem precisar reinventar a roda.

Saber como utilizar esses recursos faz com que seu código seja mais eficiente e poderoso, além de tornar o processo de desenvolvimento mais rápido e organizado.

No próximo capítulo, vamos dar um passo emocionante no nosso aprendizado: uma introdução ao **Machine Learning**.

Vamos explorar como os computadores podem aprender com os dados para fazer previsões e tomar decisões inteligentes.

Prepare-se para mergulhar nesse mundo fascinante e descobrir como o Python e suas bibliotecas desempenham um papel crucial nesse processo!



PARA SABER MAIS ...

Python Documentation: Explicação detalhada do comando do processo de importação de módulos, pacotes e bibliotecas em Python. https://docs.python.org/3/reference/simple_stmts.html#the-import-statement

Python Package Index: Site oficial de Índice completo de pacotes disponíveis para utilização na linguagem Python. <https://pypi.org/>

Livros e cursos: Acesse <https://pypro.com.br> para Livros e Cursos sobre a linguagem Python. O curso “*Python Profissional: do Zero ao Mercado de Trabalho*” apresenta o que são e como criar e utilizar as Módulos e Pacotes em Python, com exemplos reais e exercícios para fixação dos conteúdos.

10

Introdução ao Machine Learning

A linguagem **Python** se tornou uma das principais escolhas para a área de **Machine Learning** devido à sua simplicidade, facilidade de leitura e, principalmente, pela vasta quantidade de **módulos e pacotes** específicos para aprendizado de máquina.

O crescimento do uso de Python está diretamente relacionado ao desenvolvimento dessas bibliotecas que facilitam a criação, treinamento e avaliação de modelos complexos de machine learning com poucas linhas de código.

Pacotes como **scikit-learn**, **TensorFlow**, **Keras** e **PyTorch** são amplamente utilizados por desenvolvedores e pesquisadores para implementar algoritmos de aprendizado de máquina de maneira eficiente. Esses pacotes oferecem uma série de funcionalidades prontas que permitem desde a manipulação de dados até a construção de redes neurais profundas.

Além disso, Python é uma linguagem altamente versátil, sendo capaz de integrar soluções de machine learning com outras áreas como ciência de dados, visualização de dados e inteligência artificial.

Com o suporte de uma vasta comunidade e uma ampla gama de pacotes especializados, Python se consolidou como a linguagem de referência para machine learning, permitindo a criação de sistemas robustos e escaláveis para diversas aplicações, desde previsões de mercado até a automação de tarefas industriais.

Agora, vamos explorar o que é machine learning, os tipos de aprendizado de máquina e os algoritmos mais comumente usados nessa área e como as máquinas aprendem. Finalizaremos esse capítulo com um exemplo de aplicação de machine learning em Python.

O que é Machine Learning?

De forma simplificada, Machine Learning é a capacidade de um computador aprender com os dados e fazer previsões ou tomar decisões sem que cada comportamento seja explicitamente programado.

A máquina "**aprende**" encontrando padrões ocultos nos dados (esses padrões na verdade vão se transformar em equações matemáticas). Em vez de definir regras rígidas, o computador generaliza a partir dos dados históricos e aplica esse conhecimento para resolver novos problemas.

Existem três componentes principais em Machine Learning:

1. **Dados:** A base para o aprendizado, contendo padrões e informações ocultas.
2. **Características:** As propriedades ou variáveis de interesse nos dados.
3. **Algoritmos:** Métodos matemáticos usados para construir os modelos de previsão.

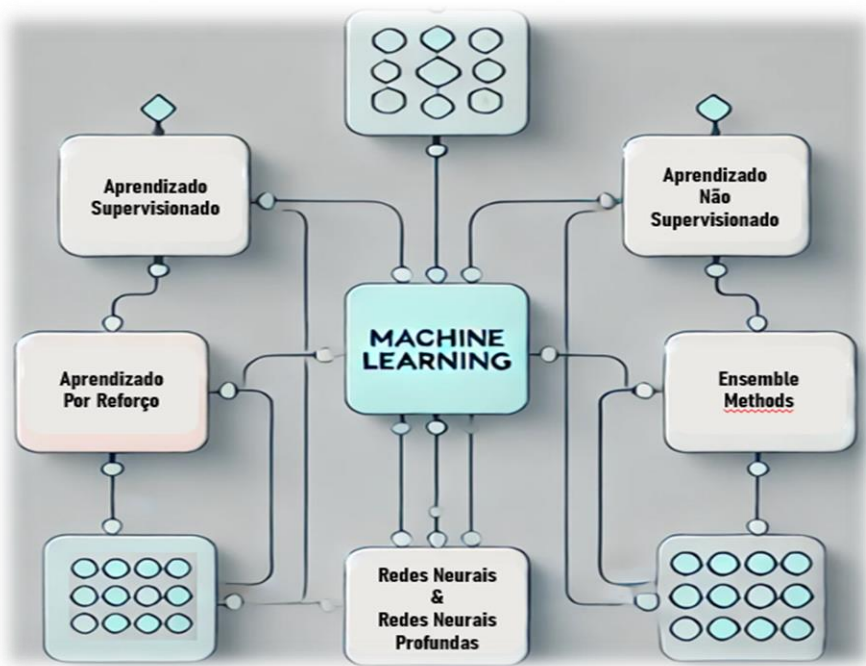
O processo de aprendizado começa com a análise de grandes quantidades de dados, onde o computador reconhece padrões e regularidades.

Com base nisso, o sistema gera um **modelo preditivo**, que é essencialmente, como já dissemos anteriormente, uma função matemática que pode prever o comportamento de novos dados com base no aprendizado anterior.

Como funciona e quais os principais tipo de Machine Learning?

A ideia principal do Machine Learning é trabalhar com dados históricos (chamados de "dataset"). Esses dados podem ser rotulados (com respostas conhecidas) ou não rotulados, e o objetivo do aprendizado é criar um modelo que seja capaz de prever resultados com base em novos dados.

Existem muitos tipos de Machine Learning. Os cinco principais são destacados na figura abaixo.



Vamos ver os detalhes de cada um desses tipos de machine learning.

1. Aprendizado Supervisionado

O aprendizado supervisionado envolve treinar o modelo com dados rotulados, ou seja, dados que já possuem uma resposta correta associada.

O objetivo é que o modelo aprenda a associar as entradas (características) às saídas (resultados) corretas, para que, quando novos dados sejam fornecidos, ele possa prever o resultado.

Exemplo: Prever se um cliente fará uma compra com base em seu histórico de comportamento.

Algoritmos Comuns:

- Regressão Linear
- Árvore de Decisão
- K-Nearest Neighbors (KNN)
- Random Forest
- Support Vector Machine (SVM)

2. Aprendizado Não Supervisionado

No aprendizado não supervisionado, o modelo trabalha com dados não rotulados, ou seja, sem uma resposta definida.

O objetivo é que o modelo encontre padrões ocultos ou estruturas inerentes aos dados, como agrupamentos ou correlações.

Exemplo: Segmentação de clientes com base em comportamentos semelhantes sem um rótulo pré-definido.

Algoritmos Comuns:

- K-Means (algoritmo de clusterização)
- PCA (Análise de Componentes Principais)
- DBSCAN (Density-Based Spatial Clustering)

3. Aprendizado por Reforço

O aprendizado por reforço envolve um agente que interage com um ambiente e aprende a tomar decisões com base em recompensas e penalidades. O objetivo é maximizar a recompensa ao longo do tempo, e o agente ajusta suas ações para obter melhores resultados a cada interação.

Exemplo: Um robô aprendendo a navegar por um ambiente sem bater em obstáculos, ajustando suas ações conforme as recompensas que recebe.

Algoritmos Comuns:

- Q-Learning
- Deep Q-Networks (DQN)

4. Ensemble Methods

Os *Ensemble Methods* combinam múltiplos modelos para criar um modelo mais robusto e preciso. A ideia é que a combinação de diferentes modelos pode melhorar a precisão e reduzir o risco de overfitting, já que erros de um modelo podem ser compensados por outros.

Exemplo: Classificar imagens utilizando múltiplos modelos de classificação e combinando os resultados.

Algoritmos Comuns:

- Random Forest (que combina várias árvores de decisão)
- Gradient Boosting
- AdaBoost
- Bagging

5. Redes Neurais

As **Redes Neurais Artificiais (ANNs)** são uma classe de algoritmos inspirados no funcionamento do cérebro humano. Elas são compostas de "neurônios" organizados em camadas e são particularmente eficazes para problemas

complexos, como reconhecimento de imagem, processamento de linguagem natural e tradução automática.

Nos últimos anos, com o avanço do **Deep Learning**, redes neurais profundas (ou Deep Neural Networks) se tornaram uma das abordagens mais populares em Machine Learning.

Exemplo: Reconhecimento de voz em assistentes virtuais, onde a rede neural aprende a identificar padrões de fala e responder adequadamente.

Algoritmos Comuns:

- Redes Neurais Artificiais (ANN)
- Redes Neurais Convolucionais (CNN) — muito usadas para análise de imagens.
- Redes Neurais Recorrentes (RNN) — aplicadas em dados sequenciais, como séries temporais e textos.

Cada um desses tipos de aprendizado tem suas aplicações e vantagens. Escolher o método correto depende da natureza do problema e do tipo de dados disponível.

Muitas vezes, uma combinação de diferentes abordagens (como os **Ensemble Methods**) pode ser a melhor solução.

Como as Máquinas Aprendem?

O conceito central de **Machine Learning** é a capacidade das máquinas de aprenderem a partir de dados, sem que todas as regras precisem ser explicitamente programadas. Mas como isso realmente acontece?

O processo de aprendizado das máquinas pode ser explicado em algumas etapas fundamentais, que ocorrem dentro de um modelo de machine learning:

1. Entrada de Dados

O aprendizado de uma máquina começa com os **dados**. Esses dados podem ser números, imagens, textos ou qualquer tipo de informação que o modelo precisa processar. No **aprendizado supervisionado**, os dados vêm acompanhados de "rótulos" que indicam as respostas corretas. Por exemplo, para um modelo que classifica imagens de animais, cada imagem seria associada a um rótulo como "gato" ou "cachorro". Já no **aprendizado não supervisionado**, os dados são fornecidos sem rótulos, e a máquina deve encontrar padrões por conta própria.

2. Processamento de Dados (Extração de Características)

Para que uma máquina entenda os dados, eles precisam ser transformados em algo que o modelo possa interpretar. Isso envolve a **extração de características**

(features). As características são propriedades ou atributos dos dados que ajudam a máquina a encontrar padrões. Por exemplo, em um conjunto de dados de casas, características poderiam ser o tamanho da casa, o número de quartos, ou o bairro em que ela está localizada. Em dados de imagem, as características podem ser padrões de cor ou formas dentro da imagem.

3. Treinamento do Modelo

Durante o treinamento, a máquina utiliza um **algoritmo de aprendizado** para ajustar seu comportamento de acordo com os dados que recebe. O algoritmo compara as previsões que faz com os valores corretos (no caso de aprendizado supervisionado), ajustando os parâmetros internos do modelo para melhorar sua precisão ao longo do tempo. Esse ajuste é feito por meio de um processo chamado **otimização**, no qual o algoritmo tenta minimizar o erro de suas previsões.

Uma das formas mais comuns de ajustar o modelo é através do **gradiente descendente**, um método que ajusta os parâmetros de forma iterativa, reduzindo o erro a cada passo. As **redes neurais**, por exemplo, utilizam esse processo para ajustar os pesos das conexões entre os neurônios a fim de aprender padrões complexos.

4. Generalização

A máquina deve aprender a **generalizar** o conhecimento adquirido nos dados de treinamento para novos dados que nunca viu antes. Isso é crucial, pois o objetivo final é que a máquina seja capaz de realizar previsões ou classificações corretas em cenários reais e não apenas no conjunto de dados com o qual foi treinada.

Se um modelo aprende muito bem os dados de treinamento, mas não consegue fazer previsões precisas em novos dados, dizemos que ele **superajustou** (overfitting) os dados, ou seja, ele memorizou o conjunto de treino em vez de aprender padrões generalizáveis. Para evitar isso, técnicas como a validação cruzada são usadas durante o treinamento.

5. Ajustes e Avaliação

Depois de treinar o modelo, ele é testado com novos dados para verificar o quão bem ele aprendeu. A máquina faz previsões sobre esses novos dados e os compara com as respostas corretas (se disponíveis), gerando métricas de desempenho, como **acurácia**, **precisão** ou **erro médio**. Se o desempenho não for satisfatório, os parâmetros do modelo podem ser ajustados ou o algoritmo de aprendizado pode ser alterado.

6. Previsões Finais

Depois que o modelo é treinado e testado, ele está pronto para fazer previsões em dados reais. Neste ponto, a máquina "aprendeu" a partir do conjunto de dados fornecido e agora é capaz de aplicar esse conhecimento para tomar decisões ou fazer previsões sobre novos dados que ela nunca viu antes.

De forma simplificada, as máquinas aprendem por meio de um ciclo de processamento de dados, ajuste de parâmetros e avaliação, que permite a elas melhorar continuamente com o tempo e fazer previsões mais precisas.

É um processo que imita o aprendizado humano, em que experimentamos, ajustamos nossas estratégias e, com o tempo, melhoramos nosso desempenho em diversas tarefas. Para fazer isso, o computador utiliza, além dos dados, algoritmos específicos. A seguir, vamos falar dos principais algoritmos de machine learning.

Algoritmos Comuns no Machine Learning

Além da categorização dos tipos de aprendizado, existem muitos algoritmos de machine learning usados para resolver diferentes problemas. Cada algoritmo tem suas vantagens dependendo da natureza dos dados e da tarefa. Abaixo, estão alguns dos algoritmos mais comuns que você encontrará ao começar a trabalhar com machine learning em Python:

1. **Regressão Linear:** Utilizada para prever valores contínuos.
2. **KNN (K-Nearest Neighbors):** Algoritmo simples de classificação baseado na proximidade dos vizinhos.
3. **Random Forest:** Conjunto de árvores de decisão, muito eficiente em tarefas de classificação e regressão.
4. **SVM (Support Vector Machine):** Utiliza margens de separação para classificar dados.
5. **K-Means:** Algoritmo de clustering para agrupamento de dados.
6. **Redes Neurais Artificiais:** Inspiradas no cérebro humano, são usadas em problemas complexos como reconhecimento de imagens e linguagem natural.
7. **PCA (Principal Component Analysis):** Algoritmo para redução de dimensionalidade e visualização de dados.

Cada algoritmo tem suas vantagens e desvantagens, e é importante entender suas características antes de aplicá-los. A seguir, vamos implementar um exemplo prático para aplicar esses conceitos.

Exemplo Prático: Utilizando Machine Learning em Python

Agora que entendemos os tipos e algoritmos de machine learning, vamos ver um exemplo prático usando Python e a biblioteca **scikit-learn**.

Vamos construir um modelo de classificação para prever a espécie de uma flor, utilizando o famoso *dataset Iris*.

```
✓ 11s [1] # Importando os pacotes e funções específicas de ML
      from sklearn.datasets import load_iris
      from sklearn.model_selection import train_test_split
      from sklearn.neighbors import KNeighborsClassifier
```

Importação dos pacotes e funções de ML: Aqui fizemos as importações das funções e métodos que vamos utilizar, de cada um dos pacotes/módulos específicos de ML.

```
✓ 0s [2] # Carregar o dataset Iris
      iris = load_iris()
      X = iris.data # Variáveis independentes (características das flores)
      y = iris.target # Variável dependente (espécie da flor)
```

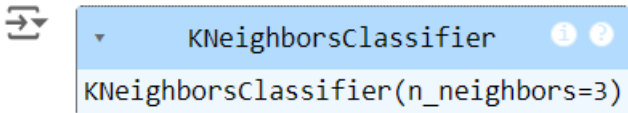
Carregar o dataset Iris: Usamos o *dataset Iris*, um conjunto de dados clássico de machine learning.

```
✓ 0s [3] # Dividir os dados em treino e teste
      X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

Divisão em treino e teste: Separamos os dados em 70% para treinamento e 30% para teste.

```
✓ [4] # Criar o modelo KNN
0s model = KNeighborsClassifier(n_neighbors=3)

✓ [5] # Treinar o modelo
0s model.fit(X_train, y_train)
```

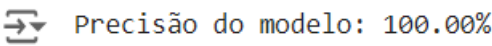


The variable explorer shows a dropdown menu for the variable 'model'. The selected item is 'KNeighborsClassifier', and the expanded view shows 'KNeighborsClassifier(n_neighbors=3)'.

Treinamento do modelo KNN: Criamos um classificador KNN e o treinamos nos dados.

```
✓ [6] # Fazer previsões
0s predictions = model.predict(X_test)

✓ [7] # Avaliar o modelo
0s accuracy = model.score(X_test, y_test)
print(f"Precisão do modelo: {accuracy * 100:.2f}%")
```



The output of the code is 'Precisão do modelo: 100.00%'.

Avaliação: Em seguida, após criarmos o modelo, realizamos as previsões, utilizando a base de teste. Por fim, medimos a precisão do modelo ao realizar as previsões com os dados de teste.

No caso desse treinamento, tivemos uma precisão incrível: 100% das previsões na base de teste foram corretas!

Com poucas linhas de código, conseguimos construir e treinar um modelo de machine learning para resolver um problema de classificação: qual é o tipo de flor Iris.

Assim, o que vimos ao longo deste capítulo sobre **Machine Learning** é apenas a **ponta do iceberg**. O campo de aprendizado de máquina é vasto e está em constante evolução, com novas técnicas, algoritmos e aplicações surgindo a todo momento.

Embora tenhamos abordado os conceitos básicos, como os tipos de aprendizado e alguns algoritmos comuns, há muito mais para se explorar, desde arquiteturas mais complexas, como redes neurais profundas, até técnicas avançadas de otimização e ajuste de modelos.

À medida que você se aprofunda no estudo de Machine Learning, vai descobrir que ele pode ser aplicado em diversas áreas, desde a **classificação de dados**, **agrupamento**, até o **reconhecimento de padrões em imagens e linguagem natural**.

No entanto, o que vimos aqui é suficiente para você começar a entender o poder dessa tecnologia e como ela pode ser usada para resolver problemas reais.

No próximo capítulo, vamos explorar uma área mais ampla que utiliza o Machine Learning como parte de seu processo: a **Ciência de Dados**.

Vamos aprender como manipular, analisar e visualizar dados, utilizando bibliotecas essenciais do Python, como pandas, numpy e matplotlib, preparando o caminho para integrar técnicas de Machine Learning em projetos reais de ciência de dados.

Vamos então para a próxima etapa da nossa jornada!!



PARA SABER MAIS ...

Google Machine Learning: Curso Intensivo de Machine Learning, com 12 módulos, feito pelo Google. <https://developers.google.com/machine-learning/crash-course?hl=pt-br>

Microsoft Machine Learning: Ambiente de aprendizagem da Microsoft sobre machine learning, focado no Azure. <https://learn.microsoft.com/pt-br/azure/machine-learning/?view=azureml-api-2>

Amazon Machine Learning: Página de documentação de Machine Learning da Amazon. <https://docs.aws.amazon.com/machine-learning/>

Python Package Index: Site oficial de Índice completo de pacotes disponíveis para utilização na linguagem Python: Busque por “machine learning” e veja a quantidade de pacotes disponíveis nessa área. <https://pypi.org/>

Livros e cursos: Acesse <https://pypro.com.br> para Livros e Cursos sobre a linguagem Python. O curso “*Ciência de Dados Profissional: do Zero ao Mercado de Trabalho*” apresenta um módulo completo e específico explicando detalhadamente Machine Learning e seus diversos tipos e algoritmos.

11

Introdução a Ciência de Dados

Nos últimos anos, a **Ciência de Dados** revolucionou a forma como empresas e organizações tomam decisões. Em vez de depender apenas da intuição ou do "feeling" de seus gestores, as empresas começaram a **basear suas decisões em dados concretos**. Isso foi impulsionado tanto pelo crescimento do volume de dados, o chamado **Big Data**, quanto pelo surgimento de ferramentas e tecnologias que facilitam o acesso, análise e visualização desses dados.

Além de linguagens como Python, que simplificam a criação de algoritmos e pipelines de ciência de dados, surgiram também diversas ferramentas de **Business Intelligence (BI)**, como **PowerBI**, **Tableau** e **Qlik**, que tornaram a análise e visualização de dados acessíveis para profissionais de diversas áreas, sem a necessidade de conhecimentos avançados em programação.

Essas ferramentas, juntamente com bibliotecas como **pandas**, **numpy** e **matplotlib** no Python, permitiram que a **Ciência de Dados** se expandisse para todos os setores, facilitando a coleta, o tratamento e a análise de grandes volumes de dados, com o objetivo de **extrair insights valiosos** e criar previsões baseadas em fatos, em vez de suposições.

Agora, vamos explorar o que é Ciência de Dados, entender como funciona um pipeline típico e conhecer as principais ferramentas e bibliotecas utilizadas na área.

O que é Ciência de Dados?

Ciência de Dados é o campo interdisciplinar que combina matemática e estatística, programação e o conhecimento específico do domínio (conhecimento do negócio / business) para **extrair, analisar e interpretar grandes volumes de dados**.

O principal objetivo da ciência de dados é transformar dados brutos em **informações valiosas**, ajudando na tomada de decisões, na previsão de eventos futuros e na criação de novos produtos ou serviços.

A ciência de dados não se resume a uma única tarefa ou técnica. Ela envolve um conjunto de práticas que incluem a **coleta, limpeza, análise e visualização** de dados, além de **modelagem preditiva** e o uso de **algoritmos de aprendizado de máquina** para encontrar padrões ou prever resultados futuros.

O diferencial da ciência de dados é a capacidade de **automatizar processos** e gerar insights em larga escala, permitindo que empresas e organizações possam basear suas decisões em dados reais, ao invés de intuição.

Além disso, a ciência de dados permite a criação de soluções que podem se adaptar e aprender ao longo do tempo, como no caso de sistemas de recomendação (exemplo: recomendações da Netflix ou do Spotify) ou em ferramentas de detecção de fraudes em transações financeiras.

Com a crescente popularidade da ciência de dados, surgiram diversas profissões especializadas que desempenham papéis complementares dentro desse ecossistema. Cada uma dessas profissões tem um foco específico, mas todas trabalham em conjunto para extrair o máximo valor dos dados. Vamos conhecer algumas dessas profissões:

1. Cientista de Dados: é o profissional central na área de ciência de dados. Ele é responsável por **explorar, analisar e interpretar dados** para resolver problemas complexos de negócios. Utilizando técnicas de aprendizado de máquina, modelagem estatística e programação, o cientista de dados identifica padrões, desenvolve previsões e gera insights que orientam a tomada de decisões estratégicas.

- **Principais Habilidades:** Programação (Python, R), estatística, machine learning, manipulação de dados, storytelling com dados.
- **Ferramentas Comuns:** pandas, numpy, scikit-learn, TensorFlow, matplotlib.

2. Analista de Dados: é responsável por **coletar, processar e realizar análises descritivas** de conjuntos de dados. O analista de dados foca em gerar relatórios, criar dashboards e fornecer respostas diretas sobre as operações da empresa. Eles frequentemente trabalham com **ferramentas de Business Intelligence** para ajudar a visualizar os dados de forma clara e acessível.

- **Principais Habilidades:** Ferramentas de visualização de dados (PowerBI, Tableau), SQL, análise de dados, geração de relatórios.
- **Ferramentas Comuns:** Excel, SQL, PowerBI, Tableau.

3. Engenheiro de Dados: trabalha nos bastidores, projetando e mantendo a **infraestrutura** que permite a coleta, o armazenamento e o processamento de grandes volumes de dados. Ele constrói **pipelines de dados**, integra diferentes fontes de dados e garante que esses dados estejam organizados e acessíveis para os cientistas de dados e analistas.

- **Principais Habilidades:** Engenharia de software, bancos de dados, pipelines de dados, processamento de grandes volumes de dados (Big Data).
- **Ferramentas Comuns:** Hadoop, Spark, AWS, Kafka, SQL.

4. Engenheiro de IA (Inteligência Artificial): cria e implementa **modelos avançados de machine learning e deep learning**. Este profissional é responsável por desenvolver algoritmos que automatizam tarefas cognitivas complexas, como reconhecimento de voz e imagens, processamento de linguagem natural e sistemas autônomos. Eles estão na linha de frente de projetos que envolvem inteligência artificial e aprendizado profundo.

- **Principais Habilidades:** Machine learning, deep learning, redes neurais, programação em Python, desenvolvimento de algoritmos complexos.
- **Ferramentas Comuns:** TensorFlow, Keras, PyTorch, OpenCV.

5. Engenheiro de Machine Learning: é o profissional responsável por implementar e escalonar os modelos desenvolvidos por cientistas de dados em **ambientes de produção**. Eles também trabalham no aprimoramento contínuo desses modelos, garantindo que os sistemas sejam ágeis e eficientes.

- **Principais Habilidades:** Machine learning, otimização de modelos, desenvolvimento de software, integração de modelos em sistemas.
- **Ferramentas Comuns:** scikit-learn, TensorFlow, Kubernetes, Docker.

6. Arquiteto de Dados: é responsável por **projetar a estrutura dos dados** em uma organização. Ele define como os dados serão armazenados, acessados e integrados em toda a empresa. Seu trabalho é garantir que os dados fluam de maneira eficiente e sejam utilizados da melhor maneira possível.

- **Principais Habilidades:** Design de sistemas de dados, bancos de dados relacionais e não relacionais, governança de dados.
- **Ferramentas Comuns:** SQL, NoSQL, AWS, Azure, Google Cloud.

Essas diferentes funções mostram que o campo da ciência de dados é vasto e multifacetado. Cada uma dessas profissões desempenha um papel fundamental no ciclo de vida dos dados, desde a coleta e organização até a análise e implementação de modelos inteligentes.

Embora o **Cientista de Dados** tenha um papel central, é o trabalho colaborativo entre essas diferentes funções que permite a extração do valor completo dos dados.

O Pipeline de Ciência de Dados

O processo de ciência de dados é frequentemente descrito como um **pipeline**, composto por várias etapas, desde a coleta de dados até a obtenção de insights.

Cada uma dessas etapas desempenha um papel crucial no sucesso de um projeto de ciência de dados. Vamos examinar cada uma delas:

1. Coleta de Dados: A primeira etapa do pipeline é a coleta de dados. Esses dados podem vir de várias fontes, como bancos de dados, arquivos CSV, APIs, ou sensores. Em alguns casos, os dados podem ser armazenados em formatos não estruturados, como texto ou imagens.

- **Exemplo:** Coletar dados de clientes de uma loja online, incluindo informações de transações, cliques no site e histórico de compras.

2. Limpeza e Pré-processamento de Dados: Após a coleta, os dados muitas vezes estão desorganizados, incompletos ou contêm erros. Essa etapa envolve a limpeza dos dados, removendo ou corrigindo valores ausentes, duplicados ou inconsistentes. Além disso, os dados podem precisar ser transformados para facilitar sua análise, como normalizar números ou transformar variáveis categóricas em numéricas.

- **Exemplo:** Remover valores ausentes de uma coluna que registra o número de compras feitas por clientes em um determinado mês.

3. Análise Exploratória de Dados (EDA): A análise exploratória de dados é uma etapa essencial que permite aos cientistas de dados **entender** melhor os dados que possuem. Nesta fase, utilizam-se gráficos e estatísticas descritivas para identificar padrões, correlações e outliers nos dados.

- **Exemplo:** Usar um gráfico de dispersão para visualizar a relação entre o preço de produtos e as vendas mensais.

4. Modelagem e Algoritmos: Após a exploração inicial, a próxima etapa é criar **modelos preditivos** ou algoritmos que podem aprender com os dados e fazer previsões. Essa etapa pode envolver o uso de técnicas como regressão, classificação, ou clustering, dependendo do problema que está sendo resolvido.

- **Exemplo:** Usar um modelo de regressão linear para prever a receita de vendas com base em investimentos em marketing.

5. Avaliação de Modelos: Após construir o modelo, ele é testado para verificar se suas previsões são precisas e generalizáveis para novos dados. Métricas como **acurácia**, **precisão**, **recall** e **erro médio absoluto** são comumente usadas para avaliar o desempenho do modelo.

- **Exemplo:** Avaliar a precisão de um modelo que prevê se um cliente irá cancelar seu serviço com base em seu comportamento recente.

6. Implantação e Monitoramento: Uma vez que o modelo está pronto e aprovado, ele pode ser implantado em um sistema de produção, onde será usado para prever resultados ou automatizar decisões em tempo real. Após a implantação, é importante monitorar o modelo continuamente para garantir que ele continue performando bem à medida que novos dados chegam.

- **Exemplo:** Implantar um modelo que recomenda produtos para clientes em um site de e-commerce.

Principais Pacotes de Ciência de Dados em Python

A linguagem Python tornou-se uma escolha natural para a ciência de dados devido à vasta coleção de pacotes especializados. Vamos explorar alguns dos pacotes mais importantes:

1. Pandas



O **pandas** é a principal biblioteca para manipulação e análise de dados em Python. Ele permite que os cientistas de dados trabalhem com **DataFrames**, que são estruturas de dados organizadas em tabelas (linhas e colunas). Com pandas, é fácil carregar, limpar e transformar dados.

- **Exemplo:** Carregar um arquivo CSV e filtrar todas as linhas onde o valor da coluna "idade" é maior que 30.

2. Numpy



O **numpy** é uma biblioteca essencial para **operações numéricas** em Python. Ele oferece suporte a arrays multidimensionais e uma vasta coleção de funções

matemáticas para operações como álgebra linear, estatística e transformações de dados.

- **Exemplo:** Realizar cálculos rápidos em grandes volumes de dados numéricos, como somar ou multiplicar matrizes.

3. Matplotlib



O **matplotlib** é uma biblioteca de visualização de dados que permite criar gráficos de alta qualidade em Python. Ele é amplamente utilizado para criar gráficos de linhas, barras, dispersão e muitos outros.

- **Exemplo:** Criar um gráfico de linha para visualizar a variação de temperatura ao longo de uma semana.

4. Scikit-Learn



O **scikit-learn** é uma biblioteca para **machine learning** que fornece uma série de algoritmos pré-construídos para classificação, regressão, clustering, e redução de dimensionalidade. Além disso, ele oferece ferramentas para particionar dados em conjuntos de treino e teste, além de diversas métricas para avaliar modelos.

- **Exemplo:** Construir um modelo de regressão logística para prever se um cliente comprará um produto.

5. Seaborn



O **seaborn** é uma biblioteca baseada no matplotlib que oferece gráficos estatísticos mais atraentes e fáceis de criar. Ele é particularmente útil para criar gráficos mais complexos, como mapas de calor ou gráficos de dispersão com diferentes cores para diferentes categorias.

- **Exemplo:** Visualizar a correlação entre variáveis em um mapa de calor.

Esses são apenas algumas das muitas e muitas bibliotecas, pacotes e módulos existentes na área de ciência de dados. Existe um “mundo” de possibilidades a serem exploradas nessa área.

Para finalizar, vamos ver um exemplo real, envolvendo todo o pipeline descrito anteriormente de Ciência de Dados. Vamos utilizar um conjunto de dados muito conhecido e utilizado de forma didática, para exemplificar um processo de análise de dados: o dataset do Titanic.

Pipeline Completo de Ciência de Dados com Python

Agora, vamos ver um **pipeline completo de ciência de dados** utilizando Python e um dataset público. Usaremos o **dataset Titanic**, disponível publicamente, que contém informações sobre os passageiros do famoso navio, e nosso objetivo será **prever quais passageiros sobreviveram com base em suas características**. Para tanto, vamos passar por cada uma das etapas... vamos começar!

1. Carregar os Dados

A primeira etapa em qualquer pipeline de ciência de dados é a **coleta e carregamento dos dados**.

Para isso, usamos o **pandas**, uma das bibliotecas mais populares para manipulação de dados em Python.

```
✓ [1] # Importando o pacote Pandas
0s  import pandas as pd

# Carregar os dados
df = pd.read_csv("https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv")
```

Aqui, utilizamos a função **read_csv()** do pandas para carregar os dados do Titanic, que estão em formato CSV.

O dataset contém informações sobre os passageiros do navio Titanic, incluindo suas idades, gêneros, classes de viagem e se sobreviveram ou não ao desastre.

2. Limpeza e Tratamento de Dados

Após carregar os dados, o próximo passo é realizar a **limpeza e tratamento dos dados**. Nesta etapa, removemos valores faltantes, corrigimos dados inconsistentes e preparamos o dataset para a análise e modelagem.

```
✓ [2] # Verificar as colunas disponíveis no DataFrame
0s print(df.columns)

➡ Index(['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age', 'SibSp',
        'Parch', 'Ticket', 'Fare', 'Cabin', 'Embarked'],
        dtype='object')

✓ [3] # Remover colunas irrelevantes
0s df = df.drop(['PassengerId', 'Name', 'Ticket', 'Cabin'], axis=1)

# Preencher valores faltantes, reatribuindo diretamente as colunas
df['Age'] = df['Age'].fillna(df['Age'].mean())
df['Embarked'] = df['Embarked'].fillna(df['Embarked'].mode()[0])
```

`print(df.columns)`: Esse comando lista todas as colunas do DataFrame. É importante verificar as colunas disponíveis para entender quais informações o conjunto de dados contém. Isso também ajuda a garantir que as colunas que desejamos remover existam no DataFrame.

`df = df.drop(['PassengerId', 'Name', 'Ticket', 'Cabin'], axis=1, errors='ignore')`: Aqui, estamos removendo as colunas que consideramos irrelevantes para a análise e a construção do modelo preditivo. No caso do dataset Titanic:

- **PassengerId**: Um identificador único para cada passageiro, que não fornece informações úteis para a análise.
- **Name**: O nome do passageiro não afeta diretamente a análise ou predição.
- **Ticket**: O número do bilhete é um dado irrelevante para prever a sobrevivência.
- **Cabin**: Contém muitos valores nulos e, portanto, pode ser removida.

O argumento `axis=1` indica que estamos removendo colunas, e `errors='ignore'` garante que, se alguma das colunas não existir no DataFrame, o código não resultará em erro.

`df['Age'] = df['Age'].fillna(df['Age'].mean())` : A coluna Age (idade) pode conter valores nulos (faltantes). Para lidar com isso, preenchemos os valores faltantes com a **média** de todas as idades disponíveis no dataset. A escolha

da média é uma forma simples de substituir valores ausentes sem introduzir vieses muito fortes.

```
df['Embarked'] =  
df['Embarked'].fillna(df['Embarked'].mode()[0])
```

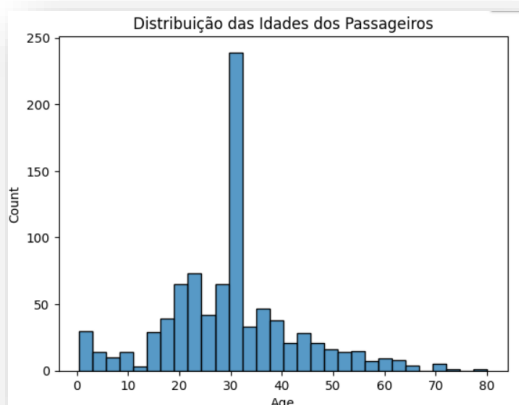
 : A coluna **Embarked** (porto de embarque) também pode ter valores nulos. Neste caso, preenchemos os valores ausentes com o valor mais frequente (ou **moda**) da coluna, ou seja, o porto onde a maioria dos passageiros embarcou.

3. Análise Exploratória de Dados (EDA)

Na **Análise Exploratória de Dados (EDA)**, investigamos os dados para entender melhor suas características, distribuições e relações entre variáveis. Para isso, utilizamos técnicas gráficas e estatísticas.

```
✓ 2s [4] # Importando os pacotes e bibliotecas para exibição de gráficos  
import seaborn as sns  
import matplotlib.pyplot as plt  
  
# Distribuição das idades dos passageiros  
sns.histplot(df['Age'], kde=False, bins=30)  
plt.title("Distribuição das Idades dos Passageiros")  
plt.show()
```

Nesse exemplo, usamos o **seaborn** e o **matplotlib** para visualizar a distribuição das idades dos passageiros. Esse gráfico ajuda a identificar padrões, como a faixa etária dos passageiros, além de fornecer insights iniciais sobre o dataset. O resultado pode ser visto na imagem a seguir:



Outros gráficos que podem ser utilizados incluem: Gráficos de barras (para variáveis categóricas) e Gráficos de dispersão (para explorar a relação entre duas variáveis)

4. Modelagem de Dados

Nesta etapa, construímos um **modelo preditivo**. Isso envolve a escolha do algoritmo apropriado (neste caso, a **Random Forest**), a preparação dos dados para o treinamento (convertendo variáveis categóricas em numéricas, por exemplo), e a divisão dos dados em conjunto de treino e teste.

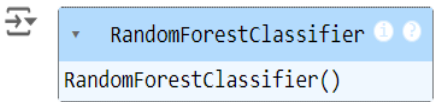
```
✓ [5] # Importando os pacotes e bibliotecas para criação do modelo
1s from sklearn.model_selection import train_test_split
    from sklearn.ensemble import RandomForestClassifier

    # Converter variáveis categóricas em numéricas
    df = pd.get_dummies(df, drop_first=True)

    # Separar variáveis independentes e dependentes
    X = df.drop('Survived', axis=1)
    y = df['Survived']

    # Dividir em conjunto de treino e teste
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

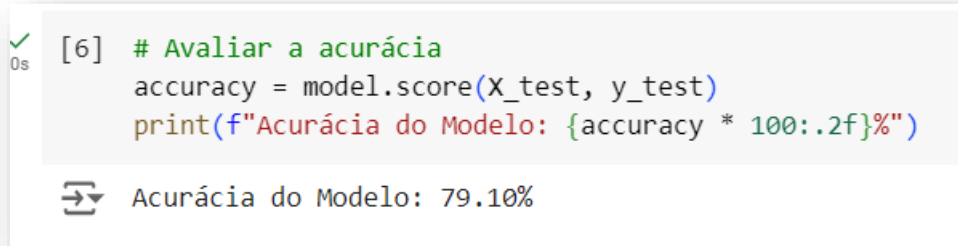
    # Treinar o modelo
    model = RandomForestClassifier()
    model.fit(X_train, y_train)
```



Em síntese, fizemos o seguinte: **Conversão de variáveis categóricas**: O método `get_dummies()` transforma variáveis categóricas, como 'Sex' ou 'Embarked', em variáveis numéricas que o modelo pode interpretar. Em seguida, fizemos a **Divisão dos dados**: Dividimos os dados em um conjunto de treino (70%) e um conjunto de teste (30%) usando `train_test_split()`. O conjunto de treino é usado para ajustar o modelo, e o conjunto de teste é usado para avaliar seu desempenho. E por fim utilizamos o algoritmo **Random Forest**, é um método de aprendizado supervisionado baseado em árvores de decisão, útil para tarefas de classificação e regressão, para criamos o modelo.

5. Avaliação do Modelo

Depois de treinar o modelo, precisamos verificar seu desempenho, utilizando o conjunto de dados de teste para medir a **acurácia** (ou outra métrica de desempenho).



```
✓ [6] # Avaliar a acurácia
0s accuracy = model.score(X_test, y_test)
print(f"Acurácia do Modelo: {accuracy * 100:.2f}%")

⇒ Acurácia do Modelo: 79.10%
```

O método **score()** calcula a acurácia do modelo com base nos dados de teste, ou seja, quantas previsões corretas o modelo fez. Esse valor é expresso como uma porcentagem, indicando a proporção de previsões corretas. No caso, o modelo criado tem uma acurácia de 79,10%.

Este pipeline exemplifica um fluxo completo de ciência de dados, desde o carregamento dos dados até a avaliação do modelo. Ele segue as etapas clássicas de **coleta de dados, limpeza, análise, modelagem e avaliação**, que são essenciais em qualquer projeto de ciência de dados.

E assim... a **Ciência de Dados** oferece uma maneira poderosa de transformar dados brutos em insights valiosos, e Python, com suas bibliotecas específicas, facilita muito esse processo. O pipeline de ciência de dados abordado aqui serve como um guia básico, mas pode ser expandido para atender a casos mais complexos, com mais dados ou técnicas avançadas de modelagem.

No próximo capítulo, vamos mergulhar em **projetos reais**, onde exploraremos uma série de exemplos, desde os mais simples até os mais avançados, para consolidar o aprendizado e aplicar essas técnicas em situações práticas.



PARA SABER MAIS ...

Google Data Science: Um pacote completo de ferramentas de gerenciamento de dados, análise e machine learning para gerar insights e agregar valor aos dados. <https://cloud.google.com/data-science?hl=pt-BR>

Microsoft Fabric: Saiba mais sobre a experiência de IA e Ciência de Dados no Microsoft Fabric. <https://learn.microsoft.com/pt-br/fabric/data-science/>

Amazon SageMaker: Amazon SageMaker é um serviço totalmente gerenciado de machine learning. Com o Amazon SageMaker, os cientistas de dados e os desenvolvedores podem criar e treinar modelos de machine learning com rapidez e implantá-los em um ambiente hospedado pronto para produção. https://docs.aws.amazon.com/pt_br/sagemaker/?id=docs_gateway

Python Package Index: Site oficial de Índice completo de pacotes disponíveis para utilização na linguagem Python: Busque por “data science” e veja a quantidade de pacotes disponíveis nessa área. <https://pypi.org/>

Livros e cursos: Acesse <https://pypro.com.br> para Livros e Cursos sobre a linguagem Python. O curso “*Ciência de Dados Profissional: do Zero ao Mercado de Trabalho*” apresenta detalhadamente todo o pipeline de ciência de dados, de forma didática e com mais de 25 projetos reais desenvolvidos passo a passo.

12

Desenvolvimento de Projetos Reais

A prática é uma parte fundamental no processo de aprendizagem de qualquer habilidade, e com a programação não é diferente.

Neste capítulo, você terá a oportunidade de aplicar os conceitos que aprendeu até agora, enfrentando projetos reais que simulam problemas e desafios do mundo prático.

Desenvolver projetos reais é uma das melhores formas de solidificar o conhecimento e desenvolver as habilidades necessárias para se tornar um programador competente.

Uma dica importante: antes de olhar a solução de cada projeto, tente desenvolver o código por conta própria.

Tentar resolver problemas por conta própria é fundamental para que você ganhe confiança, entenda as nuances da linguagem e desenvolva o raciocínio lógico necessário para superar desafios no dia a dia de um profissional de tecnologia. Mesmo que você cometa erros, cada tentativa falha é uma oportunidade de aprendizado.

Cada projeto seguirá o seguinte formato:

- Um **nome**, que descreve o objetivo do projeto.
- Um **descritivo**, explicando todos os detalhes necessários para sua implementação.
- E por fim, o **código de solução**, acompanhado por uma explicação linha a linha, para garantir que você entenda como cada parte do código contribui para a resolução do problema.

Agora, vamos ao nosso primeiro projeto!

Projeto 1: Pedra, Papel e Tesoura

Descritivo:

Neste projeto, vamos criar o famoso jogo **Pedra, Papel e Tesoura**. O jogo envolve dois jogadores, onde cada um escolhe uma das opções: pedra, papel ou tesoura. As regras são simples:

- **Pedra** vence **Tesoura** (Pedra quebra Tesoura)
- **Tesoura** vence **Papel** (Tesoura corta Papel)
- **Papel** vence **Pedra** (Papel envolve Pedra)

Neste caso, você será o jogador 1, e o computador será o jogador 2, que fará uma escolha aleatória entre as três opções. Após as escolhas, o programa determinará o vencedor com base nas regras do jogo.

Regras do Jogo:

1. O jogador (você) escolhe entre pedra, papel ou tesoura.
2. O computador escolhe aleatoriamente uma das três opções.
3. O programa compara as duas escolhas e decide quem é o vencedor.
4. O resultado é exibido na tela.

Resolução:

Aqui está a implementação do jogo:

```
# código
import random

# Lista de opções disponíveis
opcoes = ['Pedra', 'Papel', 'Tesoura']

# Jogador escolhe sua jogada
jogada_jogador = input("Escolha Pedra, Papel ou Tesoura: ").capitalize()

# Computador escolhe sua jogada aleatoriamente
jogada_computador = random.choice(opcoes)

# Exibir as escolhas
print(f"Você escolheu: {jogada_jogador}")
print(f"O computador escolheu: {jogada_computador}")

# Determinar o vencedor
if jogada_jogador == jogada_computador:
    print("Empate!")
elif (jogada_jogador == 'Pedra' and jogada_computador == 'Tesoura') or \
     (jogada_jogador == 'Tesoura' and jogada_computador == 'Papel') or \
     (jogada_jogador == 'Papel' and jogada_computador == 'Pedra'):
    print("Você venceu!")
else:
    print("Computador venceu!")
```

Exemplo de execução:



```
Escolha Pedra, Papel ou Tesoura: Pedra
Você escolheu: Pedra
O computador escolheu: Papel
Computador venceu!
```

Explicação linha a linha:

- **import random:** Importa o módulo random, que será usado para gerar a escolha aleatória do computador.
- **opcoes = ['Pedra', 'Papel', 'Tesoura']:** Define uma lista com as três opções disponíveis no jogo.
- **jogada_jogador = input("Escolha Pedra, Papel ou Tesoura: ").capitalize():** O jogador insere sua escolha através do teclado, e o método capitalize() garante que a primeira letra da escolha seja maiúscula, facilitando a comparação posterior.
- **jogada_computador = random.choice(opcoes):** O computador escolhe uma das opções de forma aleatória, usando a função random.choice().
- **print(f"Você escolheu: {jogada_jogador}") e print(f"O computador escolheu: {jogada_computador}"): Exibe na tela as escolhas feitas pelo jogador e pelo computador.**
- **Bloco if-elif-else para determinar o vencedor:**
 - Primeiro, verificamos se as escolhas são iguais, o que resulta em um empate.
 - Em seguida, verificamos se a jogada do jogador vence a do computador com base nas regras do jogo.
 - Caso contrário, o computador vence.
- **print("Você venceu!") ou print("Computador venceu!"): Exibe o resultado final com base na lógica das comparações.**

E é isso... Vamos para o segundo projeto!

Projeto 2: Conversor de Moedas (Real para Dólar e Dólar para Real)

Descritivo:

Neste projeto, vamos criar um **Conversor de Moedas** que permite ao usuário converter valores entre **Real (BRL)** e **Dólar (USD)**.

O usuário poderá escolher se deseja converter de **Real para Dólar** ou de **Dólar para Real**, e o programa calculará o valor correspondente com base em uma taxa de câmbio fixa.

A taxa de câmbio será definida no início do programa, mas pode ser ajustada conforme necessário.

Regras do Conversor:

1. O usuário escolhe a direção da conversão: de Real para Dólar ou de Dólar para Real.
2. O usuário insere o valor que deseja converter.
3. O programa calcula o valor convertido com base na taxa de câmbio.
4. O resultado da conversão é exibido na tela.

Código:

Aqui está a implementação do conversor de moedas:

```
# Taxa de câmbio fixa
taxa_cambio = 5.10 # 1 USD = 5.10 BRL

# Função para converter de Real para Dólar
def real_para_dolar(valor_real):
    return valor_real / taxa_cambio

# Função para converter de Dólar para Real
def dolar_para_real(valor_dolar):
    return valor_dolar * taxa_cambio

# Escolha do usuário sobre o tipo de conversão
print("Conversor de Moedas")
print("1: Converter de Real para Dólar")
print("2: Converter de Dólar para Real")
opcao = input("Escolha uma opção (1 ou 2): ")

# Realizando a conversão com base na escolha
if opcao == "1":
    valor_real = float(input("Digite o valor em Reais: R$ "))
    valor_convertido = real_para_dolar(valor_real)
    print(f"R$ {valor_real:.2f} equivalem a US$ {valor_convertido:.2f}")
elif opcao == "2":
    valor_dolar = float(input("Digite o valor em Dólares: US$ "))
    valor_convertido = dolar_para_real(valor_dolar)
    print(f"US$ {valor_dolar:.2f} equivalem a R$ {valor_convertido:.2f}")
else:
    print("Opção inválida! Escolha 1 ou 2.")
```

Exemplo de Execução:

```
➡ Conversor de Moedas
1: Converter de Real para Dólar
2: Converter de Dólar para Real
Escolha uma opção (1 ou 2): 1
Digite o valor em Reais: R$ 230
R$ 230.00 equivalem a US$ 45.10
```

Explicação linha a linha:

- **taxa_cambio = 5.10:** Definimos uma taxa de câmbio fixa, onde 1 dólar equivale a 5,10 reais. Essa taxa pode ser ajustada para refletir a taxa de câmbio atual.

- **Função real_para_dolar(valor_real):**

```
def real_para_dolar(valor_real):
    return valor_real / taxa_cambio
```

Esta função recebe como argumento o valor em reais que o usuário deseja converter e o divide pela taxa de câmbio para obter o valor equivalente em dólares.

- **Função dolar_para_real(valor_dolar):**

```
def dolar_para_real(valor_dolar):
    return valor_dolar * taxa_cambio
```

Esta função faz o processo inverso: multiplica o valor em dólares pela taxa de câmbio para obter o valor em reais.

- **Menu de Opções:**

```
print("Conversor de Moedas")
print("1: Converter de Real para Dólar")
print("2: Converter de Dólar para Real")
opcao = input("Escolha uma opção (1 ou 2): ")
```

Exibimos um menu simples que permite ao usuário escolher entre as duas opções de conversão (Real para Dólar ou Dólar para Real). O valor inserido pelo usuário é armazenado na variável opcao.

- **Bloco condicional if-elif-else:**

- Se a opção for "1" (Real para Dólar), o programa solicita ao usuário que insira o valor em reais e calcula a conversão chamando a função `real_para_dolar()`. O resultado é exibido em dólares.
- Se a opção for "2" (Dólar para Real), o programa solicita o valor em dólares e usa a função `dolar_para_real()` para calcular o equivalente em reais, exibindo o resultado.
- Se o usuário inserir uma opção inválida (algo que não seja 1 ou 2), o programa exibe uma mensagem de erro.

- **Conversão e exibição dos resultados:**

As conversões utilizam as funções criadas e formatam os valores com **duas casas decimais** para garantir uma apresentação clara e precisa:

```
print(f"R$ {valor_real:.2f} equivalem a US$ {valor_convertido:.2f}")  
OU  
print(f"US$ {valor_dolar:.2f} equivalem a R$ {valor_convertido:.2f}")
```

Esse conversor de moedas é um projeto simples, mas muito útil para aprender como trabalhar com **funções** e **operações matemáticas** básicas em Python.

Além disso, ele mostra como interagir com o usuário através do teclado e como estruturar um programa com base em condições (if-elif-else).

No próximo projeto, vamos aumentar a complexidade e explorar novos conceitos de programação. Preparados(as)? Vamos ao projeto 3!!

Projeto 3: Calculadora Simples

Descritivo:

Neste projeto, vamos criar uma **calculadora simples** que realiza as operações matemáticas básicas: adição, subtração, multiplicação e divisão. O usuário pode escolher qual operação deseja realizar, inserir dois números, e o programa retorna o resultado.

Detalhes:

- O programa apresenta um menu de opções para as quatro operações.
- O usuário escolhe a operação e insere dois números.
- O programa realiza a operação escolhida e exibe o resultado.
- Se o usuário inserir uma operação inválida, o programa exibe uma mensagem de erro.

Código:

Aqui está a implementação do projeto:

```
 # Função para exibir o menu de operações
def menu():
    print("Calculadora Simples")
    print("Selecione uma operação:")
    print("1. Adição")
    print("2. Subtração")
    print("3. Multiplicação")
    print("4. Divisão")

# Função para realizar a operação de adição
def adicionar(x, y):
    return x + y

# Função para realizar a operação de subtração
def subtrair(x, y):
    return x - y

# Função para realizar a operação de multiplicação
def multiplicar(x, y):
    return x * y

# Função para realizar a operação de divisão
def dividir(x, y):
    if y == 0:
        return "Erro! Divisão por zero não é permitida."
    else:
        return x / y
```

```

# Loop principal do programa
while True:
    # Exibir o menu
    menu()

    # Solicitar ao usuário que escolha uma operação
    escolha = input("Escolha uma operação (1/2/3/4) ou 'sair' para encerrar: ")

    if escolha == 'sair':
        print("Encerrando a calculadora.")
        break

    # Verificar se a escolha é válida
    if escolha in ['1', '2', '3', '4']:
        # Solicitar ao usuário que insira dois números
        num1 = float(input("Digite o primeiro número: "))
        num2 = float(input("Digite o segundo número: "))

        # Executar a operação escolhida
        if escolha == '1':
            print(f"{num1} + {num2} = {adicionar(num1, num2)}")
        elif escolha == '2':
            print(f"{num1} - {num2} = {subtrair(num1, num2)}")
        elif escolha == '3':
            print(f"{num1} * {num2} = {multiplicar(num1, num2)}")
        elif escolha == '4':
            resultado = dividir(num1, num2)
            print(f"{num1} / {num2} = {resultado}")
    else:
        print("Opção inválida! Tente novamente.")

```

Exemplo de Execução:

```

⇒ Calculadora Simples
Selecione uma operação:
1. Adição
2. Subtração
3. Multiplicação
4. Divisão
Escolha uma operação (1/2/3/4) ou 'sair' para encerrar: 1
Digite o primeiro número: 45
Digite o segundo número: 23
45.0 + 23.0 = 68.0
Calculadora Simples
Selecione uma operação:
1. Adição
2. Subtração
3. Multiplicação
4. Divisão
Escolha uma operação (1/2/3/4) ou 'sair' para encerrar: sair
Encerrando a calculadora.

```

Explicação detalhada linha a linha:

- **Função menu():**

Esta função exibe o menu de operações disponíveis. Cada operação é numerada de 1 a 4, representando adição, subtração, multiplicação e divisão.

```
def menu():  
    print("Calculadora Simples")  
    print("Selecione uma operação:")  
    print("1. Adição")  
    print("2. Subtração")  
    print("3. Multiplicação")  
    print("4. Divisão")
```

- **Funções de Operações:**

Cada função realiza uma das quatro operações básicas. Elas recebem dois números (x e y) como parâmetros e retornam o resultado da operação.

```
def adicionar(x, y):  
    return x + y  
  
def subtrair(x, y):  
    return x - y  
  
def multiplicar(x, y):  
    return x * y  
  
def dividir(x, y):  
    if y == 0:  
        return "Erro! Divisão por zero não é permitida."  
    else:  
        return x / y
```

A função dividir() inclui uma verificação para evitar a **divisão por zero**, retornando uma mensagem de erro quando isso ocorre.

- **Loop Principal:**

O programa usa um loop while para continuar executando até que o usuário escolha encerrar digitando "sair". Dentro do loop, o programa exibe o menu, recebe a escolha da operação e solicita dois números ao usuário.

```
while True:  
    menu()  
    escolha = input("Escolha uma operação (1/2/3/4) ou 'sair' para encerrar: ")
```

- **Condição de Saída:**

Se o usuário digitar "sair", o programa encerra o loop e exibe a mensagem "Encerrando a calculadora".

```
if escolha == 'sair':  
    print("Encerrando a calculadora.")  
    break
```

- **Validação da Escolha:**

O código verifica se a escolha do usuário é válida (se é uma das opções 1, 2, 3 ou 4). Se for uma escolha válida, o programa solicita dois números e executa a operação correspondente.

```
if escolha in ['1', '2', '3', '4']:  
    num1 = float(input("Digite o primeiro número: "))  
    num2 = float(input("Digite o segundo número: "))
```

- **Executando a Operação Escolhida:**

Dependendo da escolha do usuário, o programa chama a função correspondente (adicionar, subtrair, multiplicar, dividir) e exibe o resultado da operação.

```
if escolha == '1':  
    print(f"{num1} + {num2} = {adicionar(num1, num2)}")  
elif escolha == '2':  
    print(f"{num1} - {num2} = {subtrair(num1, num2)}")  
elif escolha == '3':  
    print(f"{num1} * {num2} = {multiplicar(num1, num2)}")  
elif escolha == '4':  
    resultado = dividir(num1, num2)  
    print(f"{num1} / {num2} = {resultado}")
```

- **Mensagem de Erro para Escolhas Inválidas:**

Se o usuário inserir uma opção que não está no menu (algo diferente de 1, 2, 3 ou 4), o programa exibe uma mensagem de erro e continua.

```
else:  
    print("Opção inválida! Tente novamente.")
```

Essa **Calculadora Simples** é um ótimo exemplo para praticar **funções**, **condições (if-else)**, **loops** e **entrada de dados do usuário** em Python. Além disso, ele mostra como lidar com erros comuns, como a divisão por zero. Tente expandir o projeto adicionando mais operações ou funcionalidades, como o cálculo de potência ou raiz quadrada! Vamos então para o quarto projeto!!

Projeto 4: Gerador de Senhas Aleatórias

Descritivo:

Neste projeto, vamos criar um **gerador de senhas aleatórias**. O programa permitirá ao usuário definir o tamanho da senha e se deseja incluir letras maiúsculas, minúsculas, números e símbolos. O programa, então, gerará uma senha aleatória com base nessas escolhas.

Detalhes:

- O usuário escolhe o comprimento da senha.
- O usuário decide se deseja incluir números, letras maiúsculas, minúsculas e símbolos.
- O programa gera uma senha aleatória conforme as preferências do usuário.
- A senha é exibida no final.

Código:

Aqui está a implementação do projeto:

```
import random
import string

# Função para gerar a senha aleatória
def gerar_senha(tamanho, incluir_maiusculas, incluir_minusculas, incluir_numeros, incluir_simbolos):
    # Conjunto de caracteres para a senha
    caracteres = ''

    if incluir_maiusculas:
        caracteres += string.ascii_uppercase
    if incluir_minusculas:
        caracteres += string.ascii_lowercase
    if incluir_numeros:
        caracteres += string.digits
    if incluir_simbolos:
        caracteres += string.punctuation

    # Garante que ao menos um tipo de caractere seja escolhido
    if caracteres == '':
        return "Erro: Nenhuma opção válida foi selecionada para gerar a senha."

    # Gerar senha aleatória
    senha = ''.join(random.choice(caracteres) for _ in range(tamanho))

    return senha
```

```

# Função principal
def main():
    print("Gerador de Senhas Aleatórias")

    # Solicitar ao usuário o comprimento da senha
    tamanho = int(input("Digite o comprimento desejado para a senha: "))

    # Solicitar as opções de inclusão de caracteres
    incluir_maiusculas = input("Incluir letras maiúsculas? (s/n): ").lower() == 's'
    incluir_minusculas = input("Incluir letras minúsculas? (s/n): ").lower() == 's'
    incluir_numeros = input("Incluir números? (s/n): ").lower() == 's'
    incluir_simbolos = input("Incluir símbolos? (s/n): ").lower() == 's'

    # Gerar a senha com base nas escolhas do usuário
    senha = gerar_senha(tamanho, incluir_maiusculas, incluir_minusculas, incluir_numeros, incluir_simbolos)

    # Exibir a senha gerada
    print(f"Sua senha gerada é: {senha}")

# Executar o programa principal
main()

```

Exemplo de Execução:

```

➡ Gerador de Senhas Aleatórias
  Digite o comprimento desejado para a senha: 9
  Incluir letras maiúsculas? (s/n): s
  Incluir letras minúsculas? (s/n): s
  Incluir números? (s/n): s
  Incluir símbolos? (s/n): s
  Sua senha gerada é: d/"?3[[7\

```

Explicação detalhada linha a linha:

- **Importações:**

```

import random
import string

```

O módulo random será usado para gerar caracteres aleatórios.

O módulo string contém conjuntos de caracteres úteis, como letras maiúsculas (ascii_uppercase), minúsculas (ascii_lowercase), números (digits) e símbolos (punctuation).

- **Função gerar_senha():**

Esta função é responsável por gerar a senha aleatória com base nas escolhas do usuário.

```
def gerar_senha(tamanho, incluir_maiusculas, incluir_minusculas,
incluir_numeros, incluir_simbolos):
    caracteres = ''
```

A variável caracteres será usada para armazenar os conjuntos de caracteres a serem utilizados na geração da senha. Dependendo das opções do usuário, essa variável incluirá letras maiúsculas, minúsculas, números e/ou símbolos.

- **Condições de inclusão:**

```
if incluir_maiusculas:
    caracteres += string.ascii_uppercase
if incluir_minusculas:
    caracteres += string.ascii_lowercase
if incluir_numeros:
    caracteres += string.digits
if incluir_simbolos:
    caracteres += string.punctuation
```

Para cada opção que o usuário escolher (incluir letras maiúsculas, minúsculas, números ou símbolos), os respectivos conjuntos de caracteres são adicionados à variável caracteres.

- **Validação:**

```
if caracteres == '':
    return "Erro: Nenhuma opção válida foi selecionada para gerar
a senha."
```

Caso o usuário não selecione nenhuma opção válida, o programa retorna um erro informando que é necessário escolher ao menos um tipo de caractere para gerar a senha.

- **Gerar a senha:**

```
senha = ''.join(random.choice(caracteres) for _ in
range(tamanho))
```

Aqui, usamos random.choice(caracteres) para escolher aleatoriamente um caractere do conjunto de caracteres e repetimos isso até que a senha tenha o comprimento desejado (definido pelo usuário). A função join() concatena esses caracteres em uma única string.

- **Função principal main():**

A função principal é onde ocorre a interação com o usuário. Ela solicita o comprimento da senha e as preferências de inclusão de caracteres.

```
tamanho = int(input("Digite o comprimento desejado para a senha: "))
```

O usuário insere o comprimento desejado para a senha.

- **Solicitar preferências:**

```
incluir_maiusculas = input("Incluir letras maiúsculas? (s/n): ").lower() == 's'  
incluir_minusculas = input("Incluir letras minúsculas? (s/n): ").lower() == 's'  
incluir_numeros = input("Incluir números? (s/n): ").lower() == 's'  
incluir_simbolos = input("Incluir símbolos? (s/n): ").lower() == 's'
```

Para cada opção, o usuário insere "s" para sim e "n" para não. O programa converte a resposta para minúsculas e verifica se é "s". Se for, a respectiva variável (incluir_maiusculas, incluir_minusculas, etc.) será True, caso contrário será False.

- **Gerar e exibir a senha:**

```
senha = gerar_senha(tamanho, incluir_maiusculas,  
incluir_minusculas, incluir_numeros, incluir_simbolos)  
print(f"Sua senha gerada é: {senha}")
```

O programa chama a função gerar_senha() com os parâmetros fornecidos pelo usuário e exibe a senha gerada na tela.

- **Executar o programa:**

```
main()
```

A função main() é chamada para iniciar o programa.

Este **Gerador de Senhas Aleatórias** é um projeto prático que demonstra como coletar entradas do usuário, manipular strings e usar bibliotecas padrão do Python para criar um programa útil e seguro. Ele também é uma ótima oportunidade para praticar a **manipulação de condições**, o uso de **funções** e a geração de dados aleatórios. Vamos agora aumentar mais um pouco a complexidade... Vamos para o projeto 5!

Projeto 5: Jogo da Forca

Descritivo:

No **Jogo da Forca**, o jogador tenta adivinhar uma palavra escolhida aleatoriamente pelo computador, uma letra por vez. O jogador tem um número limitado de tentativas e, a cada erro, uma parte do boneco da forca é "desenhada". O objetivo é adivinhar a palavra antes de esgotar todas as tentativas.

Detalhes:

- O programa escolhe aleatoriamente uma palavra de uma lista de palavras.
- O jogador insere letras uma a uma, e o programa revela as letras corretas na palavra ou adiciona uma parte ao boneco da forca para cada erro.
- O jogo termina quando o jogador acerta a palavra ou esgota as tentativas.
- O programa exibe uma mensagem de vitória ou derrota ao final.

Código:

Aqui está a implementação do projeto:

```
import random

# Lista de palavras possíveis
palavras = ['python', 'programacao', 'desenvolvimento', 'algoritmo', 'dados']

# Função para escolher uma palavra aleatória
def escolher_palavra():
    return random.choice(palavras)

# Função para exibir o estado atual da palavra
def exibir_palavra(palavra, letras_corretas):
    return ''.join([letra if letra in letras_corretas else '_' for letra in palavra])

# Função principal do Jogo da Forca
def jogo_da_forca():
    print("Bem-vindo ao Jogo da Forca!")

    # Escolher a palavra aleatória
    palavra = escolher_palavra()
    letras_corretas = []
    letras_erradas = []
    tentativas = 6 # Limite de erros
```



```
while True:
    # Exibir o estado atual da palavra
    estado_atual = exibir_palavra(palavra, letras_corretas)
    print(f"Palavra: {estado_atual}")

    # Exibir as letras erradas
    print(f"Letras erradas: {'', ' '.join(letras_erradas)}")
    print(f"Tentativas restantes: {tentativas}")

    # Verificar se o jogador venceu
    if '_' not in estado_atual:
        print("Parabéns! Você acertou a palavra!")
        break

    # Verificar se o jogador perdeu
    if tentativas == 0:
        print(f"Você perdeu! A palavra era: {palavra}")
        break

    # Solicitar uma letra do jogador
    letra = input("Digite uma letra: ").lower()

    # Verificar se a letra já foi usada
    if letra in letras_corretas or letra in letras_erradas:
        print("Você já tentou essa letra. Tente outra.")
        continue

    # Verificar se a letra está na palavra
    if letra in palavra:
        letras_corretas.append(letra)
    else:
        letras_erradas.append(letra)
        tentativas -= 1

# Executar o jogo da forca
jogo_da_forca()
```

Exemplo de Execução:



```
Bem-vindo ao Jogo da Forca!
Palavra: _____
Letras erradas:
Tentativas restantes: 6
Digite uma letra: a
Palavra: _a____
Letras erradas:
Tentativas restantes: 6
Digite uma letra: e
Palavra: _a____
Letras erradas: e
Tentativas restantes: 5
Digite uma letra: o
Palavra: _a_o_
Letras erradas: e
Tentativas restantes: 5
Digite uma letra: d
Palavra: dado_
Letras erradas: e
Tentativas restantes: 5
Digite uma letra: s
Palavra: dados
Letras erradas: e
Tentativas restantes: 5
Parabéns! Você acertou a palavra!
```

Explicação detalhada linha a linha:

- **Lista de palavras:**

```
palavras = ['python', 'programacao', 'desenvolvimento',
            'algoritmo', 'dados']
```

Aqui temos uma lista de palavras que o jogo poderá escolher aleatoriamente para o jogador tentar adivinhar.

- **Função escolher_palavra():**

```
def escolher_palavra():
    return random.choice(palavras)
```

A função `escolher_palavra()` utiliza a função `random.choice()` para escolher uma palavra aleatória da lista `palavras`.

- **Função exibir_palavra():**

```
def exibir_palavra(palavra, letras_corretas):
    return ''.join([letra if letra in letras_corretas else '_'
                    for letra in palavra])
```

Esta função exibe o estado atual da palavra, revelando as letras corretas já adivinhadas e substituindo as letras ainda não adivinhadas por "_".

Ela usa uma **list comprehension** para construir a string da palavra, colocando a letra correta se já foi adivinhada, ou um "_" caso contrário.

- **Função principal jogo_da_forca():**

Esta função contém a lógica principal do jogo.

- **Inicialização:**

```
palavra = escolher_palavra()
letras_corretas = []
letras_erradas = []
tentativas = 6 # Limite de erros
```

A palavra é escolhida aleatoriamente com `escolher_palavra()`.

`letras_corretas` e `letras_erradas` são listas vazias que armazenam as letras adivinhadas corretamente e as erradas, respectivamente.

`tentativas` define o número máximo de erros que o jogador pode cometer (neste caso, 6).

- **Loop principal:**

```
while True:
    estado_atual = exibir_palavra(palavra, letras_corretas)
    print(f"Palavra: {estado_atual}")
    print(f"Letras erradas: {'', '.join(letras_erradas)}")
    print(f"Tentativas restantes: {tentativas}")
```

Dentro do loop `while`, o estado atual da palavra é exibido usando `exibir_palavra()`.

As letras erradas e o número de tentativas restantes também são mostrados para o jogador.

- **Verificação de vitória:**

```
if '_' not in estado_atual:
    print("Parabéns! Você acertou a palavra!")
    break
```

Se não houver mais "_" no estado atual da palavra, significa que o jogador adivinhou todas as letras e venceu o jogo.

- **Verificação de derrota:**

```
if tentativas == 0:  
    print(f"Você perdeu! A palavra era: {palavra}")  
    break
```

Se o número de tentativas restantes for 0, o jogador perde, e o programa exibe a palavra correta.

- **Solicitação de letra ao jogador:**

```
letra = input("Digite uma letra: ").lower()
```

O jogador insere uma letra, e a função `input()` converte a letra para minúscula para evitar problemas de distinção entre maiúsculas e minúsculas.

- **Verificação de letra repetida:**

```
if letra in letras_corretas or letra in letras_erradas:  
    print("Você já tentou essa letra. Tente outra.")  
    continue
```

Se o jogador já tentou a letra anteriormente, o programa informa que a letra já foi usada e pede outra.

- **Adivinhação da letra:**

```
if letra in palavra:  
    letras_corretas.append(letra)  
else:  
    letras_erradas.append(letra)  
    tentativas -= 1
```

Se a letra estiver na palavra, ela é adicionada à lista `letras_corretas`. Caso contrário, é adicionada à lista `letras_erradas`, e o número de tentativas é decrementado.

- **Executar o jogo:**

```
jogo_da_forca()
```

A função `jogo_da_forca()` é chamada para iniciar o jogo.

O **Jogo da Forca** é um excelente projeto para praticar conceitos como **loops**, **condicionais**, **listas**, e **entrada de dados** do usuário. Além disso, ele permite trabalhar com **strings** e **manipulação de dados** em Python de uma forma interativa e divertida. Vamos avançar mais um pouco em complexidade no projeto 6!

Projeto 6: Classificação de Spam (Usando o Dataset de Emails)

Descritivo:

Neste projeto, vamos construir um modelo de **classificação de spam** para identificar se um email é spam ou não com base em seu conteúdo.

Utilizaremos o algoritmo **Naive Bayes**, que é amplamente utilizado para tarefas de classificação de textos devido à sua simplicidade e eficácia.

O objetivo do projeto é treinar o modelo usando um dataset de emails etiquetados como "spam" ou "não spam", e depois fazer previsões sobre novos emails.

Detalhes:

- O dataset de emails contém textos de mensagens e uma etiqueta indicando se o email é spam ou não.
- O Dataset está localizado no seguinte endereço: "https://raw.githubusercontent.com/justmarkham/scikit-learn-videos/master/data/sms_spam.csv"
- Usaremos técnicas de **processamento de linguagem natural (NLP)** para transformar o texto dos emails em dados que podem ser utilizados pelo modelo.
- Após a preparação dos dados, utilizaremos o algoritmo **Naive Bayes** para treinar o modelo e fazer previsões.
- O modelo será avaliado com base na acurácia de suas previsões sobre emails não vistos anteriormente.

Código, a execução e a explicação detalhada:

Aqui está a implementação do projeto e sua explicação detalhada, com a execução de cada célula:

```
# Carregando os pacotes, bibliotecas e funções necessárias
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score
from google.colab import files
```

pandas: Usado para carregar e manipular o dataset.

train_test_split: Função para dividir o dataset em conjuntos de treino e teste.

CountVectorizer: Converte o texto dos emails em uma matriz de contagem de palavras (usada para transformar o texto em uma forma que o modelo entenda).

MultinomialNB: O algoritmo Naive Bayes, adequado para problemas de classificação de texto.

accuracy_score: Função para avaliar a acurácia do modelo, ou seja, quão bem ele prevê o resultado correto.

files.upload(): Usado para carregar arquivos no Google Colab

```
[11] # 1. Carregar o dataset de emails (spam e não spam) diretamente de um upload no Colab
      uploaded = files.upload() # Isso abrirá uma janela para você enviar o arquivo CSV diretamente
```

Escolher arquivos sms_spam.csv

- **sms_spam.csv**(text/csv) - 483781 bytes, last modified: 15/10/2024 - 100% done

Saving sms_spam.csv to sms_spam (1).csv

Esta linha abre uma janela no Colab para você fazer upload de arquivos.

O arquivo de dados sms_spam.csv será carregado diretamente no ambiente de execução. Baixe esse arquivo antes para o seu computador por meio do seguinte link: https://piva.pro.br/pythonEssencial/sms_spam.csv

```
[13] # 2. Carregar o arquivo CSV no pandas, tentando diferentes abordagens para tratar erros
      # Tente carregar com ';' como separador e ignorar as linhas problemáticas
      df = pd.read_csv('sms_spam.csv', encoding='latin-1', sep=';', on_bad_lines='skip')

      # Verificar as primeiras linhas para confirmar o formato
      print(df.head())
```

	Column1	Column2
0	ham	Go until jurong point, crazy.. Available only ...
1	ham	Ok lar... Joking wif u oni...
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...
3	ham	U dun say so early hor... U c already then say...
4	ham	Nah I don't think he goes to usf, he lives aro...

pd.read_csv(): Carrega o arquivo CSV no formato de um DataFrame do pandas.

encoding='latin-1': Especifica a codificação do arquivo CSV, necessária para arquivos que contêm caracteres especiais.

sep=';': Define o separador como ;, utilizado no CSV para separar as colunas.

on_bad_lines='skip': Ignora linhas problemáticas que não seguem o formato esperado do CSV, como linhas mal formatadas.

print(df.head()) : Exibe as primeiras 5 linhas do DataFrame para verificar se os dados foram carregados corretamente.

```
# 3. Renomear colunas para facilitar o entendimento (ajustar se necessário)
df.columns = ['label', 'message']
```

Renomeia as colunas do DataFrame para label (que indica se o email é spam ou não) e message (que contém o texto da mensagem).

```
# 4. Converter labels (spam/ham) para valores numéricos
df['label'] = df['label'].map({'ham': 0, 'spam': 1})
```

Converte os rótulos de texto para valores numéricos:

- **ham** (não spam) é convertido para 0.
- **spam** é convertido para 1.

```
# 5. Dividir os dados em treino e teste
X_train, X_test, y_train, y_test = train_test_split(df['message'], df['label'], test_size=0.3, random_state=42)
```

train_test_split(): Divide os dados em dois conjuntos:

- **X_train** e **y_train**: Dados usados para treinar o modelo (70% dos dados).
- **X_test** e **y_test**: Dados usados para testar o modelo (30% dos dados).

test_size=0.3: Define que 30% dos dados serão usados para o teste e 70% para o treinamento.

random_state=42: Garante que a divisão dos dados seja reproduzível, para que os resultados sejam consistentes a cada execução.

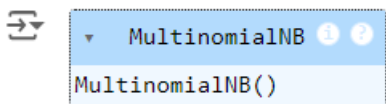
```
# 6. Transformar o texto dos emails em uma matriz de contagem de palavras (Bag of Words)
vectorizer = CountVectorizer()
X_train_counts = vectorizer.fit_transform(X_train)
X_test_counts = vectorizer.transform(X_test)
```

CountVectorizer(): Converte o texto dos emails em uma matriz de contagem de palavras. Essa técnica é conhecida como **Bag of Words**. Cada email é representado por um vetor de palavras, onde cada posição do vetor corresponde à frequência de uma palavra no email.

fit_transform(X_train): Treina o vectorizer nas mensagens de treino e transforma essas mensagens em vetores de contagem.

transform(X_test): Transforma o conjunto de teste em vetores, usando o mesmo vectorizer treinado no conjunto de treino.

```
✓ [18] # 7. Treinar o modelo Naive Bayes
Os  model = MultinomialNB()
      model.fit(X_train_counts, y_train)
```



The image shows a Jupyter Notebook cell with a green checkmark icon and the text 'Os' on the left. The code cell contains two lines: `model = MultinomialNB()` and `model.fit(X_train_counts, y_train)`. Below the code cell, there is a variable inspector showing a dropdown menu with 'MultinomialNB' selected, and a preview of the object: `MultinomialNB()`.

MultinomialNB(): Cria o modelo Naive Bayes, que é adequado para problemas de classificação de texto, como a identificação de spam.

fit(): Treina o modelo usando os vetores de contagem de palavras (`X_train_counts`) e os rótulos de spam (`y_train`).

```
# 8. Fazer previsões no conjunto de teste
y_pred = model.predict(X_test_counts)
```

predict(): Usa o modelo treinado para prever se os emails do conjunto de teste são spam ou não, com base nas características do conjunto de teste (`X_test_counts`).

```
✓ [20] # 9. Avaliar o modelo
0s accuracy = accuracy_score(y_test, y_pred)
print(f"Acurácia do modelo: {accuracy * 100:.2f}%")

⇒ Acurácia do modelo: 98.63%
```

accuracy_score(): Compara as previsões feitas pelo modelo (`y_pred`) com os rótulos reais (`y_test`) e calcula a acurácia, ou seja, a porcentagem de previsões corretas feitas pelo modelo.

print(): Exibe a acurácia do modelo em porcentagem. No caso ela foi acima de 98% de acurácia.

Este projeto simples de **classificação de spam** mostrou como aplicar **machine learning** em dados de texto utilizando o algoritmo **Naive Bayes**.

O processo inclui o pré-processamento dos dados textuais (convertendo para uma representação numérica), treinamento do modelo e avaliação de seu desempenho.

É um ótimo exemplo para entender como usar o **processamento de linguagem natural (NLP)** em machine learning e como trabalhar com classificadores simples em Python.

Agora vamos para o último projeto: O Projeto 7, sobre Ciência de Dados!

Projeto 7: Análise de Preços de Casas

Descritivo:

Neste projeto, vamos realizar uma análise de dados sobre preços de casas usando o dataset **House Prices** (Preços de Casas). O objetivo é construir um pipeline de ciência de dados completo, desde a carga e limpeza de dados até a modelagem e avaliação. Utilizaremos algoritmos de machine learning para prever o preço das casas com base em diversas características, como área total, número de quartos, número de banheiros, entre outros.

O pipeline de ciência de dados incluirá as seguintes etapas:

1. **Carregamento dos Dados:** Importar e visualizar os dados.
2. **Limpeza de Dados:** Tratar valores faltantes e remover colunas irrelevantes.
3. **Análise Exploratória de Dados (EDA):** Explorar as variáveis principais e suas correlações.
4. **Preparação dos Dados:** Converter variáveis categóricas em numéricas e dividir os dados em conjuntos de treino e teste.
5. **Modelagem:** Treinar um modelo de machine learning para prever os preços das casas.
6. **Avaliação do Modelo:** Avaliar o desempenho do modelo com métricas adequadas.

Código e explicação de cada parte:

Aqui está a implementação do projeto e sua explicação detalhada, com a execução de cada célula:

```
✓ [45] # 1. Carregar bibliotecas necessárias
0s
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.ensemble import RandomForestRegressor
```

pandas: Para manipulação de dados.

seaborn e **matplotlib**: Para visualização gráfica.

train_test_split: Para dividir os dados em conjunto de treino e teste.

LinearRegression e **RandomForestRegressor**: Algoritmos de regressão linear e de Regressão com Random Forest, utilizados para prever preços de casas. Vamos fazer uma comparação entre os dois, verificando qual gera um melhor modelos

mean_squared_error e **r2_score**: Métrica para avaliar a precisão do modelo.

```
[46] # 2. Carregar o dataset de preços de casas
url = 'https://raw.githubusercontent.com/ageron/handson-ml2/master/datasets/housing/housing.csv'
df = pd.read_csv(url)
# Exibir as primeiras linhas do dataset
print(df.head())
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	\
0	-122.23	37.88	41.0	880.0	129.0	
1	-122.22	37.86	21.0	7099.0	1106.0	
2	-122.24	37.85	52.0	1467.0	190.0	
3	-122.25	37.85	52.0	1274.0	235.0	
4	-122.25	37.85	52.0	1627.0	280.0	

	population	households	median_income	median_house_value	ocean_proximity
0	322.0	126.0	8.3252	452600.0	NEAR BAY
1	2401.0	1138.0	8.3014	358500.0	NEAR BAY
2	496.0	177.0	7.2574	352100.0	NEAR BAY
3	558.0	219.0	5.6431	341300.0	NEAR BAY
4	565.0	259.0	3.8462	342200.0	NEAR BAY

O dataset de preços de casas é carregado de uma URL usando **pd.read_csv()**. Esse dataset contém informações sobre casas na Califórnia e inclui variáveis como **latitude**, **longitude**, **número de quartos** e o **valor mediano das casas**.

Em seguida, o comando **print(df.head())** exibe as 5 primeiras linhas do dataset para que você possa ver uma amostra dos dados.

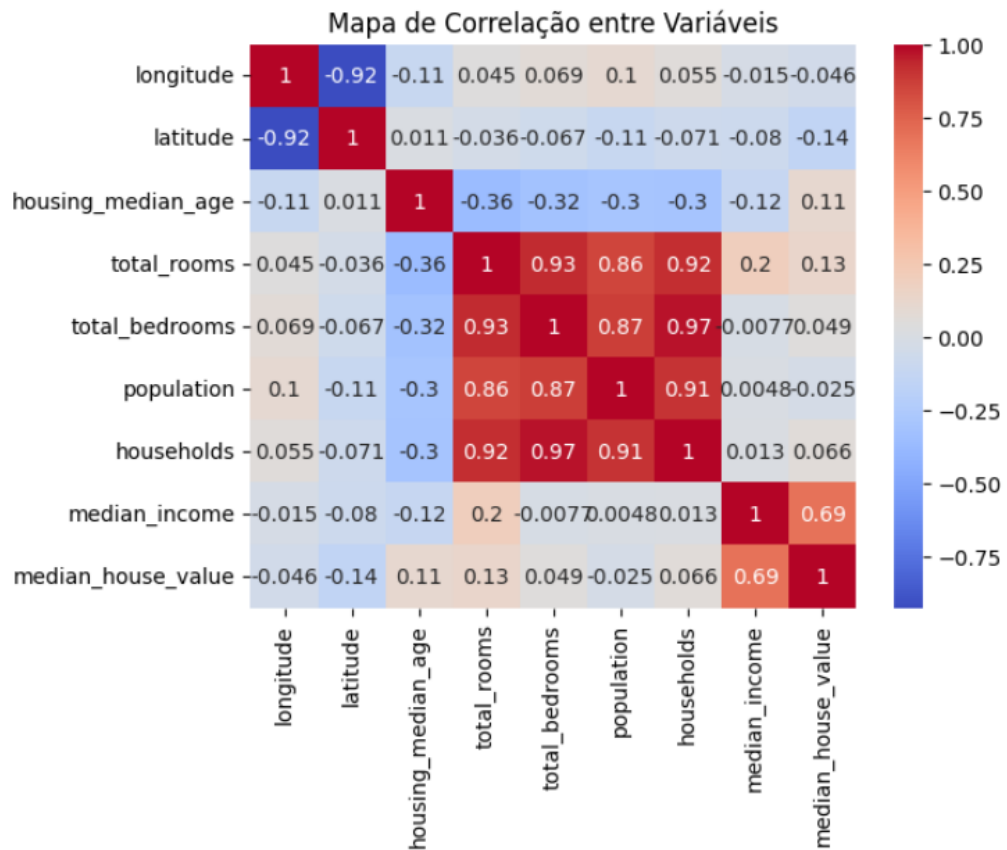
```
[47] # 3. Limpeza de Dados: Remover colunas irrelevantes e tratar valores faltantes

# Verificar se a coluna 'ocean_proximity' existe antes de remover
if 'ocean_proximity' in df.columns:
    df = df.drop(['ocean_proximity'], axis=1)

# Verificar se há valores faltantes em 'total_bedrooms' antes de preencher
if df['total_bedrooms'].isnull().sum() > 0:
    df['total_bedrooms'] = df['total_bedrooms'].fillna(df['total_bedrooms'].mean())
```

Aqui realizamos a limpeza dos dados... Removemos a coluna **ocean_proximity**, que não será utilizada no modelo e preenchemos valores faltantes em **total_bedrooms** com a média dessa coluna.

```
[48] # 4. Análise Exploratória de Dados (EDA): Visualizar correlações
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.title("Mapa de Correlação entre Variáveis")
plt.show()
```



Na análise exploratória dos dados, usamos um **heatmap** para visualizar a correlação entre as variáveis. Isso nos ajuda a entender quais variáveis estão mais fortemente relacionadas ao preço das casas.

```
[49] # 5. Separar as variáveis independentes (X) e a variável dependente (y)
X = df.drop('median_house_value', axis=1)
y = df['median_house_value']
```

No processo de separação das variáveis dependentes (y) das variáveis independentes (X) fizemos o seguinte:

X: Todas as colunas que contêm características das casas.

y: A variável que queremos prever, o valor mediano das casas.

```
✓ [50] # 6. Dividir o dataset em conjunto de treino e teste (80% treino, 20% teste)
0s X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

No processo de divisão do dataset em treino e teste, dividimos os dados em 80% para treino e 20% para teste.

```
✓ [51] # 7. Treinar o modelo de regressão linear
0s model = LinearRegression()
model.fit(X_train, y_train)
```

LinearRegression

```
LinearRegression()
```

```
✓ [52] # 7. Treinar o modelo2 também com Random Forest Regressor
16s model2 = RandomForestRegressor(n_estimators=100, random_state=42)
model2.fit(X_train, y_train)
```

RandomForestRegressor

```
RandomForestRegressor(random_state=42)
```

Treinamos o modelo utilizando dois algoritmos: Regressão Linear (**model**) e Regressão com Random Forest (**model2**).

```
✓ [53] # 8. Fazer previsões no conjunto de teste
0s y_pred = model.predict(X_test)
```

```
✓ [54] # 8. Fazer previsões com o modelo 2
0s y_pred2 = model2.predict(X_test)
```

Utilizamos os dois modelos para prever o preço das casas no conjunto de teste.

```
✓ [55] # 9. Avaliar o modelo usando o erro quadrático médio (Mean Squared Error)
0s mse = mean_squared_error(y_test, y_pred)
    print(f"Erro Quadrático Médio: {mse:.2f}")
    rmse = mse ** 0.5
    print(f"Raiz do Erro Quadrático Médio (RMSE): {rmse:.2f}")
```

```
⇒ Erro Quadrático Médio: 5052953703.90
   Raiz do Erro Quadrático Médio (RMSE): 71084.13
```

```
✓ [56] # 9. Avaliar o modelo2 usando o erro quadrático médio (Mean Squared Error)
0s mse2 = mean_squared_error(y_test, y_pred2)
    print(f"Erro Quadrático Médio - Modelo2: {mse2:.2f}")
    rmse2 = mse2 ** 0.5
    print(f"Raiz do Erro Quadrático Médio (RMSE) Modelo 2: {rmse2:.2f}")
```

```
⇒ Erro Quadrático Médio - Modelo2: 2495745711.19
   Raiz do Erro Quadrático Médio (RMSE) Modelo 2: 49957.44
```

Avaliamos o MSE (Erro Quadrático Médio) e o RMSE (a Raiz do Erro Quadrático Médio) de ambos os modelos.

O **MSE** mede a diferença média ao quadrado entre os valores reais e os previstos. Quanto menor o valor do MSE, melhor o modelo está prevendo o preço das casas.

Exemplo: Se o **MSE** for 10.000, isso significa que, em média, o modelo está errando o valor das casas em cerca de 100 dólares (já que é o quadrado do erro).

O **Erro Quadrático Médio (MSE)** é uma métrica que avalia a precisão de um modelo de regressão. Ele mede a média dos quadrados das diferenças entre os valores previstos e os valores reais.

- **Valores menores** indicam que o modelo está fazendo previsões mais próximas dos valores reais.
- Como o MSE é o quadrado das diferenças, ele amplifica grandes erros, sendo mais sensível a outliers (valores fora do padrão).
- O **MSE** é sempre positivo. Um **MSE** próximo de 0 indica um modelo mais preciso.

Exemplo:

- Se o **MSE** for 10.000, isso significa que, em média, o erro quadrático médio das previsões é 100. Portanto, o modelo está errando os valores em torno de 100 unidades monetárias.

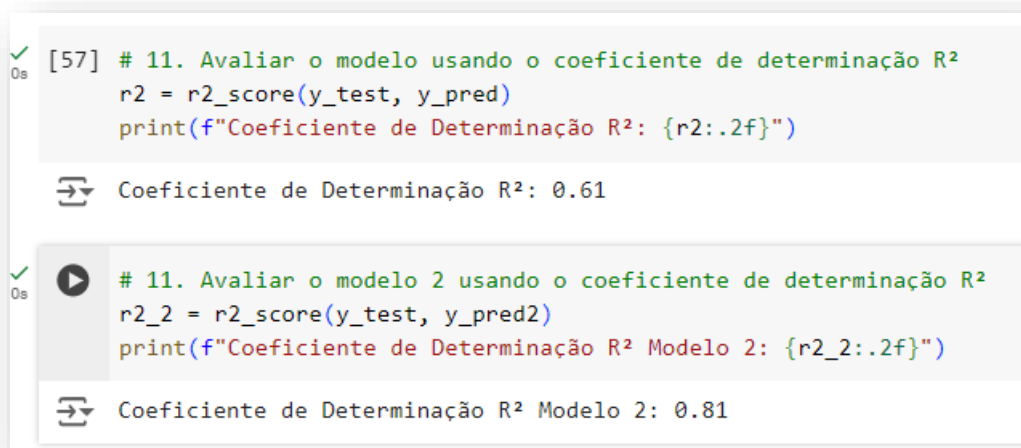
Já o **RMSE (ou Raiz Quadrada do Erro Quadrático Médio)**:

- O **RMSE** será na mesma unidade do valor previsto (neste caso, os preços das casas).
- Ele nos dirá o erro médio absoluto em termos de dólares (ou na unidade que o preço das casas é expresso), o que é mais fácil de interpretar. Um **RMSE** menor indica um modelo melhor.

Exemplo:

- Se o **RMSE** for **70.000**, isso significa que, em média, o modelo erra o valor da casa em cerca de **70.000 unidades monetárias** (provavelmente dólares). Isso pode indicar que o modelo precisa de melhorias.

Nos dois modelos, podemos ver que o segundo modelo (Regressão com Random Forest) apresenta valores menores... mas ainda muito elevados.



The screenshot shows two code cells from a Jupyter Notebook. The first cell, labeled [57], calculates the R-squared score for a linear regression model, resulting in 0.61. The second cell calculates the R-squared score for a Random Forest model, resulting in 0.81. Both cells include a comment in Portuguese: '# 11. Avaliar o modelo usando o coeficiente de determinação R²'.

```
[57] # 11. Avaliar o modelo usando o coeficiente de determinação R²
      r2 = r2_score(y_test, y_pred)
      print(f"Coeficiente de Determinação R²: {r2:.2f}")

      Coeficiente de Determinação R²: 0.61

# 11. Avaliar o modelo 2 usando o coeficiente de determinação R²
r2_2 = r2_score(y_test, y_pred2)
print(f"Coeficiente de Determinação R² Modelo 2: {r2_2:.2f}")

      Coeficiente de Determinação R² Modelo 2: 0.81
```

Avaliamos os dois modelos utilizando o **R²**, que mede a proporção da variância no valor das casas que é explicada pelas variáveis independentes no modelo. Ele varia entre 0 e 1:

- **R² = 1**: O modelo explica perfeitamente a variância nos dados.
- **R² = 0**: O modelo não explica nenhuma variância.
- Um valor próximo de 1 indica um bom ajuste do modelo.

No caso dos dois modelos criados, o primeiro, que utiliza Regressão Linear apresentou um $R^2 = 0.61$, isso significa uma acurácia de 61%.

Já o segundo modelo, que utilizou Regressão com Random Forest, apresentou um $R^2 = 0,81$, isso significa uma acurácia de aproximadamente 81%.

Com isso, concluímos que o segundo modelo é melhor, pois gerará respostas mais precisas.

E chegamos ao Final dessa Jornada ...

Ao longo deste capítulo, exploramos **sete projetos completos** que serviram como exemplos práticos para consolidar os conceitos aprendidos ao longo do livro.

Desde projetos simples, como o **jogo Pedra, Papel e Tesoura**, até projetos mais avançados, como **modelos de machine learning** e **pipelines de ciência de dados**, tivemos a oportunidade de ver como aplicar a programação em Python para resolver problemas do mundo real.

Esses projetos não apenas ilustraram diferentes aspectos da programação, mas também mostraram a evolução natural que ocorre ao longo da jornada de aprendizado.

Iniciamos com projetos básicos para desenvolver o pensamento lógico e a resolução de problemas, e gradualmente avançamos para tópicos mais complexos, como **machine learning** e **ciência de dados**, dois dos campos mais promissores da tecnologia atual.

O Futuro da Linguagem Python

Python se consolidou como uma das principais linguagens de programação devido à sua simplicidade, versatilidade e a vasta gama de bibliotecas que oferece.

Ao longo dos últimos anos, a popularidade da linguagem só cresceu, especialmente em áreas como **ciência de dados**, **machine learning**, **inteligência artificial**, **desenvolvimento web**, e até mesmo **automação de tarefas**.

Esse crescimento é impulsionado por uma comunidade ativa, que continua a desenvolver novos pacotes, frameworks e ferramentas que tornam Python cada vez mais poderoso e acessível.

Ferramentas como **pandas**, **NumPy**, **matplotlib**, **scikit-learn**, entre muitas outras, permitem que profissionais em diferentes setores usem Python para manipular dados, criar modelos preditivos, visualizar informações, e automatizar processos de maneira eficiente.

Python se destaca por sua capacidade de integrar-se com outras tecnologias, facilitando a criação de soluções completas, escaláveis e inovadoras.

Seja para o iniciante que está dando os primeiros passos na programação, ou para o especialista que desenvolve projetos complexos, Python oferece um caminho simples e eficiente.

O Próximo Passo

Agora que você tem uma base sólida em Python e compreende como utilizá-lo em diferentes contextos, o próximo passo é aplicar esses conhecimentos em **projetos reais**.

Explore novos datasets, experimente diferentes algoritmos e crie soluções que resolvam problemas do mundo real.

Cada projeto é uma oportunidade para melhorar suas habilidades e adquirir experiência prática.

A prática constante e o aprendizado contínuo são fundamentais para se manter atualizado em um mundo que está sempre evoluindo.

A comunidade Python é vasta e oferece uma infinidade de recursos para apoiar seu crescimento, desde fóruns de discussão até bibliotecas especializadas para qualquer área de interesse.

Acesse o site: <https://pypro.com.br> e mantenha-se atualizado(a). Aproveite para assinar as newsletters exclusivas para pessoas cadastradas.

Considerações Finais

Este livro foi projetado para oferecer um guia completo para iniciantes que desejam aprender Python e aplicar a linguagem nas áreas de ciência de dados, machine learning e automação.

Com o conhecimento adquirido, você está preparado para continuar sua jornada e explorar o imenso potencial que a linguagem oferece.

O futuro de Python é promissor, e o conjunto de ferramentas e bibliotecas disponíveis continuará a se expandir.

O que você aprendeu aqui é apenas o começo; Python é a chave para desbloquear soluções em diversas áreas e facilitar a criação de projetos inovadores.

Este é o final do livro, mas também é o **início** de sua jornada com Python.

Continue aprendendo, experimentando e desenvolvendo suas habilidades, porque o futuro pertence a quem domina o poder da programação e dos dados.

Referências Bibliográficas

Dilermando Piva Jr. e outros autores. **Detecção e Reconhecimento Facial: uma abordagem prática**. 1. ed. Salto: FoxTablet, 2023. 321p .

Dilermando Piva Jr. e outros autores. **Trabalhando de casa: o home office passado a limpo**. 1. ed. Salto, SP: FoxTable, 2021. v. 1. 185p .

Dilermando Piva Jr. e outros autores. **Algoritmos e Programação de Computadores**. 2. ed. Rio de Janeiro/RJ: Elsevier Editora Ltda, 2019. 528p .

Dilermando Piva Jr. e outros autores. **Estrutura de Dados e Técnicas de Programação**. 1. ed. Rio de Janeiro: Elsevier, 2014. v. 1. 416p .

Dilermando Piva Jr. e outros autores. **Organização Básica de Computadores e Linguagem de Montagem**. 1. ed. Rio de Janeiro: Elsevier, 2012. 408p .

Guido van Rossum. **Pep-8**. <https://peps.python.org/pep-0008/> (2024)

PyPRO. **Site do Projeto PyPro**. <https://pypro.com.br> (2024)

PythonDocumentation. **Python 3.13.0 documentation**. <https://docs.python.org/3/> (2024)

Tim Peters. **The zen of python**. <https://peps.python.org/pep-0020/> (2024)